



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

UN ACERCAMIENTO A LA GRAMÁTICA VÍA TEORÍA
DE CATEGORÍAS

T E S I S

QUE PARA OBTENER EL TÍTULO DE:

MATEMÁTICO

P R E S E N T A :

FERNANDO HERRERA FERREYRA

TUTOR

DR. OMAR ANTOLÍN CAMARENA



CIUDAD UNIVERSITARIA, CDMX, 2025

Índice general

Introducción	1
1. Gramáticas regulares	3
1.1. Categorías libres	3
1.2. Gramáticas simples	6
2. Gramáticas libres de contexto	15
2.1. Multicategorías	15
2.2. Gramáticas de multicategorías	20
3. Gramáticas recursivamente enumerables	23
3.1. Categorías monoidales	23
3.2. Gramáticas monoidales	31
4. Gramáticas categoriales	37
4.1. Categorías y gramáticas bicerradas	37
4.2. Reducciones gramaticales	39
4.3. Gramáticas AB	40
4.4. Gramáticas de Lambek	42
4.5. Gramáticas combinatoria	44
5. Pregrupos	47
5.1. Categorías rígidas	48
5.2. Pregrupos	53
6. DisCoPy	61
6.1. Categorías	61
6.2. Categorías monoidales	63
6.3. Multicategorías	65
6.4. Categorías bicerradas	68
6.5. Categorías rígidas	70
Apéndice. Demostraciones con diagramas	75

Introducción

El objetivo de esta tesis es estudiar una parte del marco teórico conocido como DisCoCat (*Distributional Compositional Category*), la premisa que establece que el estudio de la sintaxis de un lenguaje debe ser composicional, es decir, el análisis de una oración está determinado por sus componentes. Para tal objetivo, se introduce la teoría de categorías para estudiar la relación entre los componentes de una oración. La exposición se basará en el primer capítulo de la tesis doctoral de Giovanni Di Felice titulada *Categorical Tools for Natural Language Processing* (2022). Mostraremos los resultados principales del capítulo y detallaremos cuidadosamente las herramientas categóricas necesarias en cada capítulo.

En los primeros tres capítulos estudiaremos, como primer acercamiento a la gramática de los lenguajes naturales, la jerarquía de Chomsky de las clases de lenguajes formales. Al principio de cada capítulo, describiremos las categorías adecuadas sobre las cuales definiremos gramáticas cuyo poder expresivo sea exactamente el mismo que el de las clases de lenguajes formales en la jerarquía. La siguiente tabla resume los resultados que obtendremos.

Nivel	Lenguaje	Cómputo	Categorías
III	Regulares	Autómatas finitos	Categorías
II	Libres de contexto	Árboles de derivación	Multicategorías
0	Recursivamente enumerables	Máquinas de Turing	Categorías monoidales

Además, en el capítulo 3 introduciremos un lenguaje gráfico para categorías monoidales mediante cables y cajas, el cual no sólo nos permitirá definir la categoría monoidal libre y calcular ecuaciones dentro de la categoría, sino también implementar computacionalmente las categorías de este capítulo y los siguientes.

Adicionalmente, mencionaremos los inconvenientes del uso de cada una de estos lenguajes para la descripción de los lenguajes naturales, en particular, del español.

En el capítulo 4 buscaremos un nuevo enfoque al estudio formal de la gramática e introduciremos las gramáticas categoriales introducidas inicialmente por Kazimierz Ajdukiewicz en los 30 y continuado por Yehoshua Bar-Hillel y Joachim Lambek en los 50. Tales gramáticas surgen del estudio de la sintaxis por medio de los paradigmas de la lógica matemática, asociar a cada palabra un tipo y mediante reglas de inferencias generar derivaciones. Para su estudio, vamos a introducir las categorías bicerradas sobre las cuales definiremos gramáticas tan expresivas como las gramáticas categoriales.

En el capítulo 5 culminaremos nuestro estudio de gramáticas formales definiendo los pregrupos. Este enfoque, de índole algebraico, nos permite mantener cierto compromiso con el principio de composicionalidad —es decir, el análisis sintáctico depende fuertemente del orden y significado de las palabras— mientras que permite la validación de oraciones dentro de un lenguaje en tiempo polinomial. Los pregrupos por la vía categórica serán estudiados mediante categorías rígidas.

Finalmente, en el capítulo 6 mostraremos un breve manual de la biblioteca de código libre **DisCoPy** (*Distributional Compositional Python*), cuyo objeto es el cómputo de categorías mediante diagramas de cables para implementarlos, principalmente, para el procesamiento del lenguaje natural. Veremos cómo podemos implementar lo expuesto en los anteriores capítulos para poder analizar oraciones del español.

1 Gramáticas regulares

Nuestro objetivo en este capítulo es presentar una definición de los lenguajes regulares mediante la teoría de categorías, para los cuales existen tres maneras clásicas de definirlos:

1. con una gramática generativa, cuya ventaja es producir fácilmente expresiones válidas para un lenguaje regular $\mathcal{L}$. Estas gramáticas también son conocidas como gramáticas de tipo 3 en la jerarquía de Chomsky;
2. a través de expresiones regulares, usadas habitualmente para el reconocimiento de patrones simples; y
3. por medio de autómatas finitos deterministas, el modelo de cómputo básico asociado a la validación de cadenas perteneciente a un lenguaje $\mathcal{L}$.

Dado un lenguaje regular $\mathcal{L}$ definiremos una categoría cuyos objetos son las cadenas sobre un conjunto finito de símbolos, llamado el alfabeto, Σ y sus morfismos corresponden a las reglas de producción en $\mathcal{L}$. Para esta tarea es necesario saber cómo generar una categoría a partir de una colección de funciones (generadores o, en este caso, reglas de producción) sobre un conjunto, así que iniciaremos definiendo las herramientas categóricas necesarias, proseguiremos con la definición de un lenguaje regular por medio de una categoría y mostraremos su equivalencia con las definiciones anteriores.

1.1. Categorías libres

La noción de categoría libre es análoga a la de espacio vectorial generado por una base, es decir, dado un conjunto X y un campo k , sabemos cómo construir un k -espacio vectorial V que contenga a X .

En nuestro caso, los ingredientes de una categoría libre son un conjunto G_0 , cuyos elementos serán los objetos en la categoría libre, y un conjunto G_1 de aristas en G_0 cuyos elementos serán los generadores de los morfismos; así, introducimos los siguientes conceptos que servirán como los cimientos de una categoría libre.

Definición 1. Una **signatura simple** G es un conjunto de vértices G_0 y un conjunto de aristas G_1 tal que cada arista tiene asociado dos vértices, un dominio y un codominio

$$G_0 \xleftarrow{\text{dom}} G_1 \xrightarrow{\text{cod}} G_0$$

Notemos que una signatura simple induce una gráfica dirigida y viceversa, así que usaremos indistintamente ambos conceptos.

Dadas G, Γ signaturas simples, un **homomorfismo de signaturas** $\varphi: G \rightarrow \Gamma$ es un par de funciones $\varphi_0: G_0 \rightarrow \Gamma_0$ y $\varphi_1: G_1 \rightarrow \Gamma_1$ tal que el siguiente diagrama conmuta, es decir, $\text{dom} \circ \varphi_1 = \varphi_0 \circ \text{dom}$ y $\text{cod} \circ \varphi_1 = \varphi_0 \circ \text{cod}$.

$$\begin{array}{ccccc} G_0 & \xleftarrow{\text{cod}} & G_1 & \xrightarrow{\text{dom}} & G_0 \\ \downarrow \varphi_0 & & \downarrow \varphi_1 & & \downarrow \varphi_0 \\ \Gamma_0 & \xleftarrow{\text{cod}} & \Gamma_1 & \xrightarrow{\text{dom}} & \Gamma_0. \end{array}$$

Observemos que si consideramos dos homomorfismos de signaturas $\varphi = (\varphi_0, \varphi_1): G \rightarrow \Gamma$ y $\psi = (\psi_0, \psi_1): \Gamma \rightarrow \Phi$ podemos tomar su composición entrada a entrada, es decir, $\psi \circ \varphi = (\psi_0 \circ \varphi_0, \psi_1 \circ \varphi_1)$, la cuál resulta nuevamente un homomorfismo de signaturas ya que el siguiente diagrama conmuta, pues, el rectángulo inferior y superior conmutan.

$$\begin{array}{ccccc} G_0 & \xleftarrow{\text{cod}} & G_1 & \xrightarrow{\text{dom}} & G_1 \\ \downarrow \varphi_0 & & \downarrow \varphi_1 & & \downarrow \varphi_0 \\ \Gamma_0 & \xleftarrow{\text{cod}} & \Gamma_1 & \xrightarrow{\text{dom}} & \Gamma_1 \\ \downarrow \psi_0 & & \downarrow \psi_1 & & \downarrow \psi_0 \\ \Phi_0 & \xleftarrow{\text{cod}} & \Phi_1 & \xrightarrow{\text{dom}} & \Phi_1. \end{array}$$

Notemos, además, que la composición anterior es asociativa, pues lo es en cada entrada y que la identidad $\text{Id}_\Gamma = (\text{Id}_{\Gamma_0}, \text{Id}_{\Gamma_1})$ satisface que $\text{Id}_\Gamma \circ \varphi = \varphi$ y $\psi \circ \text{Id}_\Gamma = \psi$. Por lo anterior, tenemos una categoría cuyos objetos son las signaturas simples y sus flechas son los homomorfismos, la cual denotaremos **Sig** (en la literatura es usual denotarla **Graph**).

Consideremos ahora la categoría **Cat** cuyos objetos son categorías pequeñas, aquellas cuyas colecciones de objetos y de flechas son conjuntos, y las flechas son los funtores entre ellas. Definimos el funtor

$$U: \mathbf{Cat} \rightarrow \mathbf{Sig}$$

tal que para cada categoría $\mathcal{C}$, $U(\mathcal{C})$ es la gráfica dirigida cuyos vértices son los objetos $\mathcal{C}_0$, y las aristas corresponden a los morfismos, $\mathcal{C}_1$. Y, claramente, cada funtor $f: \mathcal{C} \rightarrow \mathcal{C}'$ induce un homomorfismo de gráficas $U(f): U(\mathcal{C}) \rightarrow U(\mathcal{C}')$.

Notemos que U solamente olvida la operación de composición de morfismos en una categoría para obtener la gráfica subyacente, es por ello que decimos que U es un funtor olvidadizo.

Ahora, nuestro objetivo es definir un funtor

$$F: \mathbf{Sig} \rightarrow \mathbf{Cat},$$

dada una gráfica dirigida G , los objetos de $F(G)$ serán los vértices de G , y sus morfismos son de alguna de las siguientes dos formas:

- Para cada v vértice de G , tenemos un morfismo $\text{Id}_v: v \rightarrow v$; y
- para cualquier $n \in \mathbb{N}$ y para cualesquiera $e_1, \dots, e_n$ aristas de G tales que $\text{cod}(e_i) = \text{dom}(e_{i+1})$ con $i \in \{1, \dots, n-1\}$ tenemos un morfismo $e_1 \dots e_n: \text{dom}(e_1) \rightarrow \text{cod}(e_n)$.

Si consideramos que Id_v es el camino de longitud $n = 0$, entonces los morfismos corresponden a todos los posibles caminos dirigidos dentro de la gráfica G . La composición de dos caminos está dada por la concatenación, salvo las identidades que se operan como la cadena vacía. Como la concatenación es siempre asociativa, concluimos que $F(G)$ es una categoría.

Ahora, tomemos $\varphi: G \rightarrow G'$ un homomorfismo de gráficas; definimos el funtor $F(\varphi): F(G) \rightarrow F(G')$ de la siguiente manera:

- Para cada v objeto en $F(G)$, $(F(\varphi))(v) = \varphi_0(v)$;
- Para cada Id_v morfismo identidad de $F(G)$, $(F(\varphi))(\text{Id}_v) = \text{Id}_{\varphi_0(v)}$; y
- Para cada $e_1 \dots e_n$ camino de G , $(F(\varphi))(e_1 \dots e_n) = \varphi_1(e_1) \dots \varphi_n(e_n)$

Hacemos notar que el último inciso está bien definido pues φ es un homomorfismo de gráficas. A F le llamamos **funtor libre**, en virtud de la siguiente proposición, pues dada una gráfica dirigida, F construye la categoría libre asociada.

Proposición 1. *Sea G una gráfica dirigida. Entonces existe un homomorfismo de gráficas $\eta: G \rightarrow UF(G)$ tal que para cualquier categoría $\mathcal{C}$ y un homomorfismo de gráficas $\varphi: G \rightarrow U(\mathcal{C})$ existe un único funtor $\tilde{\varphi}: F(G) \rightarrow \mathcal{C}$ que hace conmutar el siguiente diagrama:*

$$\begin{array}{ccc}
 F(G) & & G \xrightarrow{\eta} UF(G) \\
 \downarrow \tilde{\varphi} & & \searrow \varphi \quad \downarrow U(\tilde{\varphi}) \\
 \mathcal{C} & & U(\mathcal{C}).
 \end{array} \tag{1.1}$$

En otras palabras, nos dice que siempre que metamos una gráfica G en una categoría $\mathcal{C}$ por medio de un homomorfismo de gráficas elegido libremente, podemos extenderlo a un funtor de $F(G)$ en $\mathcal{C}$.

Demostración. Sea G una gráfica dirigida. Vamos a definir a $\eta: G \rightarrow UF(G)$, para ello observemos que $UF(G)$ es la gráfica cuyos vértices son los vértices de G y hay una arista de v a w en $UF(G)$ cada que hay un camino de v a w en G .

Definimos $\eta: G \rightarrow UF(G)$ como η_0 la identidad en vértices y dado una arista $e: v_1 \rightarrow v_2$ en G , por el comentario anterior es una arista en $UF(G)$, definimos $\eta_1(e) = e$. Así η es la inclusión de G en $UF(G)$ y, por lo tanto, un homomorfismo de gráficas.

Sean $\mathcal{C}$ una categoría y $\varphi: G \rightarrow U(\mathcal{C})$ un homomorfismo de gráficas. Queremos definir un funtor $\tilde{\varphi}: F(G) \rightarrow \mathcal{C}$. Sea v un objeto de $F(G)$, entonces es un vértice G , así $\varphi(c) \in U(\mathcal{C})$ y, por lo tanto, es un objeto en $\mathcal{C}$. Por lo tanto, $\tilde{\varphi}(v) = \varphi(v)$.

Sea f un morfismo de $F(G)$, tenemos los siguientes casos de acuerdo a la definición de morfismos en $F(G)$:

- Si $f = \text{Id}_v$ para algún v vértice de G , entonces $\tilde{\varphi}(\text{Id}_v) = \text{Id}_{\varphi(v)}$
- Si $f = e: v \rightarrow w$, como φ es un homomorfismo de gráficas, entonces hay una arista $\varphi_1(e): \varphi_0(v) \rightarrow \varphi_0(w)$ en $U(G)$ y, por lo tanto, hay un morfismo $\tilde{\varphi}(e): \tilde{\varphi}(v) \rightarrow \tilde{\varphi}(w)$ en $\mathcal{C}$.
- Si $f = e_1 \dots e_n$, definimos $\tilde{\varphi}(f) = \tilde{\varphi}(e_n) \circ \dots \circ \tilde{\varphi}(e_1)$

De las condiciones anteriores se sigue inmediatamente que $\tilde{\varphi}$ es funtor. Sigue ver que el diagrama 1.1 conmuta, pero es claro puesto que η y U actúan como inclusiones tanto en vértices como en aristas. $\square$

1.2. Gramáticas simples

A continuación definiremos la clase de lenguajes generados por una signatura simple y posteriormente mostraremos que son exactamente los lenguajes regulares.

A partir de esta sección, dada una signatura simple denotaremos como $\mathcal{C}(G)$ a la categoría libre generada por G . Sea V un conjunto no vacío de símbolos, llamado vocabulario, y G una signatura simple. A cada arista de G le asociaremos una palabra en V mediante una función $L: G_1 \rightarrow V$, o equivalentemente, un homomorfismo de gráficas $L: G \rightarrow V$ donde V es la gráfica con un vértice y cada palabra es una arista. Observemos que podemos extender L de la siguiente forma: para cada morfismo $f = f_n \circ \dots \circ f_1$ en $\mathcal{C}(G)$, $L^*(f) = L(f_1) \dots L(f_n) \in V^*$ y $L^*(\text{Id}_v) = \varepsilon$ para cualquier $v \in G_0$, donde V^* denota a la estrella de Kleene de V , es decir, el conjunto de cadenas finitas con símbolos en V , formalmente $V^* = \bigcup_{n \geq 0} V^n$.

Definición 2. Una **gramática simple** es una signatura finita simple G junto a un homomorfismo de gráficas $L: G \rightarrow V$ y símbolos distinguidos $s_0, s_1 \in G_0$, llamados el símbolo inicial y el terminal. Así, tenemos el siguiente diagrama:

$$\begin{array}{ccccc} G_0 & \xleftarrow{\text{dom}} & G_1 & \xrightarrow{\text{cod}} & G_0 \\ & & \downarrow L & & \\ & & V. & & \end{array}$$

El **lenguaje generado por G** se define como:

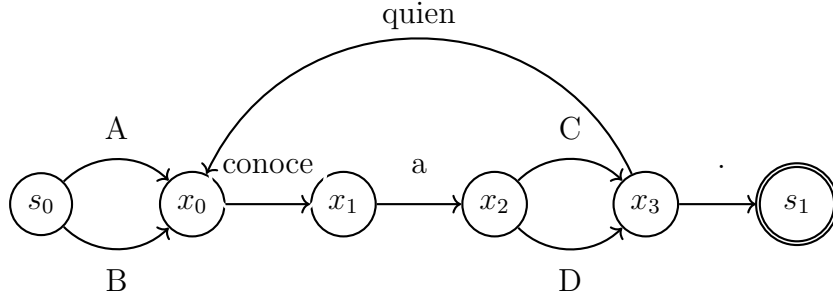
$$\mathcal{L}(G) = L^*(\mathcal{C}(G)(s_0, s_1)) \subset V^*,$$

es decir, es la imagen bajo el funtor L^* de todos los morfismos en $\mathcal{C}$ con dominio el símbolo inicial y codominio el símbolo terminal.

Un **morfismo de gramáticas simples** $\varphi: G \rightarrow H$ es un homomorfismo de gráficas tal que el siguiente diagrama conmuta

$$\begin{array}{ccc} G & \xrightarrow{\varphi} & H \\ & \searrow L_G \quad \swarrow L_H & \\ & V. & \end{array}$$

Ejemplo 1. Consideremos la siguiente gráfica G :



y notemos que induce claramente una gramática simple. Algunos elementos de su lenguaje generado son “A conoce a C”, “B conoce a D quien conoce a C” y “A conoce a C quien conoce a D quien conoce a D”.

Es necesario preguntarse cuál es la clase de lenguajes que son producidos por medio de estas signaturas; afortunadamente, la respuesta es sencilla: la clase de los lenguajes regulares. Para probar esta afirmación, introducimos la siguiente definición.

Definición 3. Una **gramática regular o de tipo 3** es una tupla (N, V, P, s_0) donde V es un vocabulario no vacío; N es un conjunto de símbolos no terminales con un símbolo de inicio s_0 ; y P un conjunto finito de reglas de producción, cada una de alguna de las siguientes formas:

$$A \rightarrow aB$$

$$A \rightarrow a$$

$$A \rightarrow \varepsilon$$

con A y B en N , y $a \in V$.

Diremos que β **se deriva de** α denotado $\alpha \Rightarrow \beta$ si y sólo si existen $\gamma, \theta, \kappa, \lambda$ tales que $\alpha = \gamma\kappa\theta$, $\beta = \gamma\lambda\theta$ y $\kappa \rightarrow \lambda$ es una regla de producción. Definimos $\Rightarrow^*$ como la cerradura transitiva¹ de $\Rightarrow$.

Definimos el lenguaje L generado por una gramática regular como

$$\mathcal{L} = \{w \in V^* | s_0 \Rightarrow^* w\}.$$

Decimos que un **lenguaje es regular** si es generado por una gramática regular.

Ejemplo 2. Consideremos la siguiente gramática regular con los símbolos no terminales $N = \{A, s_0\}$, el vocabulario $V = \{a, b, c\}$ y con las siguientes reglas P :

$$s_0 \rightarrow as_0$$

$$s_0 \rightarrow bA$$

$$A \rightarrow \varepsilon$$

$$A \rightarrow cA.$$

Observemos que el lenguaje generado por dicha gramática es $L = \{a^i b c^j | i, j \in \mathbb{N}\}$.

¹Sea $R \subset A \times A$ una relación. Definimos la cerradura transitiva de R^* como $x R^* y$ si y sólo si hay $a_1, \dots, a_n \in A$ tales que $x R a_1, a_1 R a_2, \dots, a_{n-1} R a_n$ y $a_n R y$.

Con la definición anterior, ya estamos en condiciones para demostrar el teorema principal de este capítulo:

Teorema 1. (a) *Todo lenguaje regular es generado por una gramática simple.*

(b) *Todo lenguaje generado por una gramática simple es regular.*

Demostración. Primero probemos (a).

Sea L un lenguaje regular generado por la gramática (N, V, P, s) .

Definimos una gramática simple $G = P \Rightarrow (V \cup \{s, \varepsilon\})$ donde para cada p regla de producción:

- Si p es de la forma $A \rightarrow aB$ con $A, B \in N$ y $a \in V$ entonces $p: A \rightarrow B$ en G y $L(p) = a$.
- Si p es de la forma $A \rightarrow a$ con $A \in N$ y $a \in V$, entonces $p: A \rightarrow s_1$ en G y $L(p) = a$
- Si p es de la forma $A \rightarrow \varepsilon$ con $A \in N$, entonces $p: A \rightarrow s_1$ en G y $L(p) = \varepsilon$

Notemos que el símbolo ε de nuestro vocabulario cubrirá el papel de la cadena vacía. Probemos que $L = L(G)$.

Sea $w \in L$. Entonces existe una derivación

$$s_0 \xRightarrow{p_1} \cdots \xRightarrow{p_n} w,$$

Observemos que si $n = 1$, entonces p_1 es de la forma $s \rightarrow w$, así tenemos el morfismo $p_1: s_0 \rightarrow s_1$ tal que $L(p_1) = w$. Ahora, si $n > 1$ veamos que la derivación es necesariamente de la forma:

$$s_0 \xrightarrow{p_1} aA_1 \xrightarrow{p_2} \cdots \xrightarrow{p_{n-1}} uA_{n-1} \xrightarrow{p_n} w,$$

donde $A_i \in N$ con $i \in \{1, \dots, n-1\}$, $a \in V$ y $u \in V^*$. Así tenemos los siguientes morfismos en G :

$$s_0 \xrightarrow{p_1} A_1 \xrightarrow{p_2} \cdots \xrightarrow{p_{n-1}} A_{n-1} \xrightarrow{p_n} s_1.$$

Entonces, el morfismo $p = p_n \circ p_{n-1} \circ \cdots \circ p_2 \circ p_1: s_0 \rightarrow s_1$ y

$$\begin{aligned} L(p) &= L(p_n \circ p_{n-1} \circ \cdots \circ p_2 \circ p_1) = (L(p_1)L(p_2) \dots L(p_{n-1}))L(p_n) \\ &= uL(p_n) = w. \end{aligned}$$

Observemos que es posible que $L(p_n) = \varepsilon$ pero en ese caso $u = u\varepsilon = w$.

Por lo tanto, $w \in L(G)$. Así, $L \subset L(G)$.

Ahora, sea $w \in L(G)$. Entonces existe $f: s_0 \rightarrow s_1$ en $\mathcal{C}(G)$ tal que $L(f) = w$. Como $\mathcal{C}(G)$ es una categoría libre, entonces f es un camino en G , así existen $p_1, \dots, p_n$ reglas de producción en P tal que $f = p_n \circ \dots \circ p_1$ que inducen una derivación:

$$s_0 \xRightarrow{p_1} \cdots \xRightarrow{p_n} w.$$

Y, por lo tanto, $w \in L$. Así $L(G) \subset L$. Con ello concluimos que $L = L(G)$ y terminamos la prueba del inciso (a).

Vayamos con la prueba de (b).

Sea G una gramática simple con vocabulario V y homomorfismo $L: G \rightarrow V$.

La estrategia será construir un autómata finito no determinista (AFND) que admite el lenguaje $L(G)$, lo cual implica que $L(G)$ es un lenguaje regular, cuya prueba se puede consultar en [4].

Definimos el AFND M de la siguiente manera: su conjunto de estados es G_1 , es decir, los vértices de G , donde s_0 es el símbolo de inicio y $F = \{s_1\}$ el conjunto de estados finales, y tomemos V como el conjunto de entradas. Definimos la función de transición $\delta: G_0 \times V \rightarrow 2^{G_0}$ tal que para cada $q \in G_0$ y $v \in V$,

$$\delta(q, v) = \{q' | \exists f: q \rightarrow q' \in G_1 \text{ tal que } L(f) = v\}.$$

Recordemos que δ se extiende recursivamente a V^* de la siguiente manera:

$$\begin{aligned} \delta^*(q, \varepsilon) &= \{q\}, \\ \delta^*(q, xa) &= \bigcup_{r \in \delta^*(q, x)} \delta(r, a). \end{aligned}$$

Así, definimos el lenguaje aceptado por M como:

$$L(M) = \{x \in V^* | s_1 \in \delta^*(s_0, x)\}.$$

Probemos que $L(G) = L(M)$.

Sea $w \in L(G)$. Entonces existe $f: s_0 \rightarrow s_1$ en $\mathbf{C}(G)$ tal que $L(f) = w$. Observemos que entonces existen $f_1, \dots, f_n$ en G_1 tales que el siguiente diagrama conmuta:

$$\begin{array}{ccccccc} a_0 = s_0 & \xrightarrow{f_1} & a_1 & \xrightarrow{f_2} & \dots & \xrightarrow{f_{n-1}} & a_{n-1} & \xrightarrow{f_n} & s_1 = a_{n+1}. \\ & & & & & & \searrow & & \nearrow \\ & & & & & & f & & \end{array}$$

Así, $w = L(f) = L(f_n) \cdots L(f_1)$, entonces observemos que para cada $i \in \{0, \dots, n\}$, $a_{i+1} \in \delta(a_i, L(f_i))$ y, por lo tanto, $s_1 = a_{n+1} \in \delta^*(a_0, L(f)) = \delta^*(s_0, w)$. Como $\{s_1\}$ es el conjunto terminal, concluimos que $w \in L(M)$. Por lo tanto, $L(G) \subset L(M)$.

Sea $w \in L(M)$. Entonces $s_1 \in \delta^*(s_0, w)$. Observemos que si $w = \varepsilon$, entonces $s_1 \in \delta(s_0, \varepsilon)$, así hay un morfismo $f: s_0 \rightarrow s_1$ tal que $L^*(f) = \varepsilon$, por la definición de L^* esto implica que $f = Id_{s_0}$, así $s_0 = s_1$ y $w \in L(G)$.

Por otro lado, si w no es la cadena vacía, entonces hay $w_1, \dots, w_n \in V$ tales que $w = w_1 \cdots w_n$ y estados $q_1, \dots, q_{n-1}$ que satisfacen:

$$q_1 \in \delta(s_0, w_1) \quad q_{i+1} \in \delta(q_i, w_i) \quad s_0 \in \delta(q_{n-1}, w_n)$$

para $i \in \{1, \dots, n-2\}$. Esto significa que hay $f_1, \dots, f_n$ en G_1 tales que

$$s_0 \xrightarrow{f_1} q_1 \xrightarrow{f_2} \dots \xrightarrow{f_{n-1}} q_{n-1} \xrightarrow{f_n} s_1$$

y $L(f_i) = w_i$ con $i \in \{1, \dots, n\}$. Por lo tanto, $f_n \circ \dots \circ f_1: s_0 \rightarrow s_1 \in \mathbf{C}(G)$ y

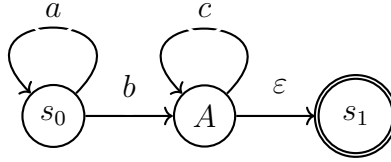
$$L^*(f_n \circ \dots \circ f_1) = L(f_1) \cdots L(f_n) = w_1 \cdots w_n = w$$

Por lo tanto, $w \in L(G)$ y así $L(M) \subset L(G)$. Por ello, $L(G) = L(M)$ como queríamos demostrar. $\square$

A partir de ahora dejaremos de usar el término gramáticas simples y, en virtud del teorema anterior, las llamaremos gramáticas regulares. Denotaremos como **Reg_V** a la categoría cuyos objetos son gramáticas regulares y los morfismos son los definidos en 2.

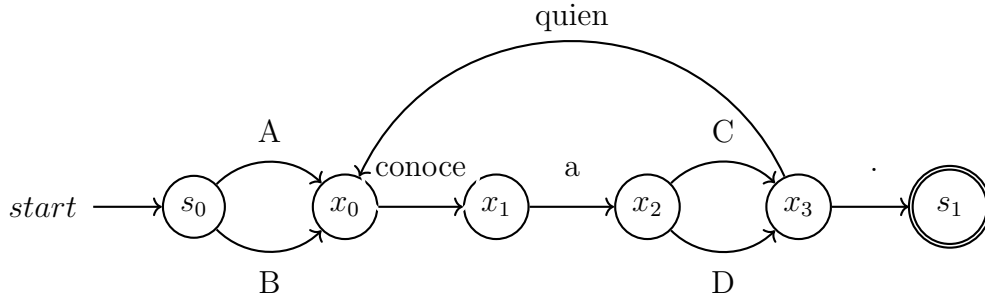
Para ilustrar las pruebas anteriores, veamos los siguientes ejemplos:

Ejemplo 3. Consideremos la gramática regular del segundo ejemplo, observemos que su lenguaje es generado por la gramática de la siguiente gráfica:



Vale la pena notar que esta gráfica induce inmediatamente un autómata finito con transiciones épsilon².

Ejemplo 4. Consideremos la gramática regular del primer ejemplo; es claro que induce el siguiente AFND:



Un resultado bien conocido sobre lenguajes regulares es que son cerrados respecto a uniones e intersecciones finitas. Existen diversas pruebas dependiendo de la definición preferida; mostremos entonces una en términos categóricos.

Proposición 2. *Los lenguajes regulares son cerrados bajo intersecciones y uniones finitas.*

²Una autómata finito con transiciones épsilon es un autómata finito no determinista donde es posible transitar entre estados sin consumir símbolos. Esta modificación no aumenta, ni disminuye el poder expresivo del autómata.

Demostración. Sean G y G' gramáticas regulares con símbolos iniciales s_0, s'_0 y símbolos finales s_1, s'_1 respectivamente.

Definimos la siguiente gramática regular $G \cap G'$: $H_1 \subset G_1 \times G'_1 \rightrightarrows H_0 \subset G_0 \times G'_0$ de la siguiente forma: (q_0, q'_0) y (q_1, q'_1) son el símbolo inicial y final respectivamente.

Además, existe una arista entre (a, b) y (a', b') en $G \cap G'$ si y sólo si existen $a \xrightarrow{f} b$ en G y $a' \xrightarrow{f'} b'$ en G' tales que $L_G(f) = L_{G'}(f')$, denotaremos a dicha arista como (f, f') y con ello definimos $L((f, f')) = L_G(f)$.

Afirmación. $L(G \cap G') = L(G) \cap L(G')$.

Tenemos que $w \in L(G \cap G')$ si y sólo si existe $(f, f') : (s_0, s'_0) \rightarrow (s_1, s'_1) \in \mathbf{C}(G \cap G')$ tal que $L^*(f, f') = w$, si y sólo existen $(f_1, f'_1), \dots, (f_n, f'_n)$ aristas de $G \cap G_1$ tales que el siguiente diagrama conmuta

$$(s_0, s'_0) \xrightarrow{(f_1, f'_1)} \dots \xrightarrow{(f_n, f'_n)} (s_1, s'_1)$$

y $w = L(f_1, f'_1) \cdots L(f_n, f'_n)$, pero esto ocurre si y sólo existen $f_1, \dots, f_n$ aristas en G y $f'_1, \dots, f'_n$ aristas en G' tales que los siguientes diagramas conmutan:

$$s_0 \xrightarrow{f_1} \dots \xrightarrow{f_n} s_1 \quad \quad \quad s'_0 \xrightarrow{f'_1} \dots \xrightarrow{f'_n} s'_1$$

y $w = L(f_1) \cdots L(f_n) = L(f'_1) \cdots L(f'_n)$. Notemos que podemos asumir sin pérdida de generalidad que ambas derivaciones tienen la misma longitud, si no, basta rellenar con transiciones épsilon.

Lo anterior sucede si y sólo si existen $f : s_0 \rightarrow s_1$ en $\mathbf{C}(G)$ y $f' : s'_0 \rightarrow s'_1$ en $\mathbf{C}(G')$ tales que $L^*(f) = w = L^*(f')$, así, $w \in L(G)$ y $w \in L(G')$. Por lo tanto $L(G \cap G') = L(G) \cap L(G')$.

Ahora, para probar el resultado en el caso de la unión definimos la gramática regular $G \cup G' : G_0 + G'_0 \rightrightarrows G_1 + G'_1$ donde el símbolo $+$ representa la unión disjunta y $+$ es el resultado de hacer las identificaciones $s_0 = s'_0$ y $s_1 = s'_1$ en la unión disjunta.

Afirmación. $L(G \cup G') = L(G) \cup L(G')$.

Notemos que $w \in L(G \cup G')$ si y sólo si existe $f : s_0 \rightarrow s_1$ en $\mathbf{C}(G \cup G')$ tal que $L^*(f) = w$.

Procediendo de manera análoga a la prueba anterior, veremos que esto ocurre si y sólo si existe $f : s_0 \rightarrow s_1$ en $\mathbf{C}(G)$ o $f : s'_0 \rightarrow s'_1$ en $\mathbf{C}(G')$ tal que $L^*(f) = w$, pero esto se da cuando w está en $L(G)$ o en $L(G')$.

Por lo tanto, $L(G \cup G') = L(G) \cup L(G')$.

Entonces, $L(G) \cap L(G')$ y $L(G) \cup L(G')$ son lenguajes regulares y procediendo inductivamente tenemos el resultado para uniones e intersecciones finitas. $\square$

Observemos que otra forma de interpretar lo anterior, es que dados dos objetos G, G' en $\mathbf{Reg}_V$ encontramos dos objetos $G \cap G'$ y $G \cup G'$ en $\mathbf{Reg}_V$. La siguiente proposición nos dirá la manera en que se relacionan en la categoría.

Proposición 3. $\mathbf{Reg}_V$ tiene productos y coproductos finitos. Más aún, si G, G' son objetos de $\mathbf{Reg}_V$, entonces $G \cap G'$ y $G \cup G'$ son su producto y coproducto respectivamente.

Demostración. Sean G y G' en $\mathbf{Reg}_V$.

Primero veamos que $G \cap G'$ es el producto donde las proyecciones $\pi: G \cap G' \rightarrow G$ y $\pi': G \cap G' \rightarrow G'$ como las inducidas por las proyecciones $G \times G' \rightarrow G$ y $G \times G' \rightarrow G'$. Probemos la propiedad universal. Sea H en $\mathbf{Reg}_V$ junto con $f: H \rightarrow G$ y $g: H \rightarrow G'$ morfismos de gramáticas regulares.

Vamos a definir el morfismo de gramáticas regulares $(f, g): H \rightarrow G \cap G'$ de la siguiente manera:

- Para cada vértice $v \in H_0$, $(f, g)_0(v) = (f_0(v), g_0(v))$, y
- para arista $h \in H_1$, $(f, g)_1(h) = (f_1(h), g_1(h))$

Veamos que es un morfismo de gramáticas regulares. Sea $h: v_1 \rightarrow v_2$ una arista de H . Como f, g son morfismos de gramáticas regulares, entonces $f(h): f(v_1) \rightarrow f(v_2)$ y $g(h): g(v_1) \rightarrow g(v_2)$ son aristas de G y G' respectivamente y, además, $L_G(f(h)) = L_H(h) = L_{G'}(g(h))$ pues el siguiente diagrama conmuta:

$$\begin{array}{ccccc} G & \xleftarrow{f} & H & \xrightarrow{g} & G' \\ & \searrow L_G & \downarrow L_H & \swarrow L_{G'} & \\ & & V & & \end{array}$$

Así $(f, g)(h): (f, g)(v_1) \rightarrow (f, g)(v_2)$ está en $G_1 \cap G_2$ y $L_H(h) = L_{G \cap G'}((f, g)(h))$ como queríamos ver.

Ya que definimos (f, g) entrada a entrada, tenemos que el siguiente diagrama conmuta:

$$\begin{array}{ccccc} & & H & & \\ & \swarrow f & \downarrow (f, g) & \searrow g & \\ G & \xleftarrow{\pi} & G \cap G' & \xrightarrow{\pi'} & G'. \end{array}$$

Supongamos entonces que existe un morfismo de gramáticas regulares $k: H \rightarrow G \cap G'$ tal que el siguiente diagrama también conmuta:

$$\begin{array}{ccccc} & & H & & \\ & \swarrow f & \downarrow k & \searrow g & \\ G & \xleftarrow{\pi} & G \cap G' & \xrightarrow{\pi'} & G'. \end{array}$$

Observemos que como π y π' las tomamos como las inducidas por las proyecciones del producto en aristas y vértices, entonces por unicidad en $\mathbf{Set}$ tenemos que $k_0 = (f, g)_0$ y $k_1 = (f, g)_1$ y, por lo tanto, $k = (f, g)$.

Concluimos entonces que $G \cap G'$ es el producto de G y G' en $\mathbf{Reg}_V$ pues se satisface la propiedad universal.

Probemos ahora que $G \cup G'$ es el coproducto donde las inyecciones $i: G \rightarrow G \cup G'$ y $i': G' \rightarrow G \cup G'$ como las inclusiones, es decir, $i(g) = g$ y $i'(g') = g'$ para $g \in G$ y $g' \in G'$.

Mostremos la propiedad universal. Sea H en $\mathbf{Reg}_V$ junto con $f: G \rightarrow H$ y $g: G' \rightarrow H$ morfismos de gramáticas regulares. Definimos $[f, g]: G \cup G' \rightarrow H$ tal que para cada $v \in (G \cup G')_0$ y $h \in (G \cup G')$ como

$$[f, g]_0(v) = \begin{cases} f_0(v) & \text{if } v \in G_0 \\ g_0(v) & \text{if } v \in G'_0 \end{cases} \quad [f, g]_1(h) = \begin{cases} f_1(h) & \text{if } h \in G_1 \\ g_1(h) & \text{if } h \in G'_1 \end{cases}$$

Puesto que f, g son morfismos de gramáticas, el siguiente diagrama conmuta:

$$\begin{array}{ccccc} G & \xrightarrow{f} & H & \xleftarrow{g} & G' \\ & \searrow L_G & \downarrow L_H & \swarrow L_{G'} & \\ & & V, & & \end{array}$$

por lo cual tenemos que $L_G(f(h)) = L_H(h) = L_{G'}(g(h))$, y como $[f, g](h) = f(h) \circ [f, g](h) = g(h)$, entonces $[f, g]$ es un morfismo de gramáticas regulares.

Por definición de las inyecciones es claro que el siguiente diagrama conmuta:

$$\begin{array}{ccccc} & & H & & \\ & \nearrow f & \uparrow [f, g] & \nwarrow g & \\ G & \xrightarrow{i} & G \cup G' & \xleftarrow{i'} & G'. \end{array}$$

Además, dado cualquier morfismo $k: G \cup G' \rightarrow H$ tal que el siguiente diagrama conmuta

$$\begin{array}{ccccc} & & H & & \\ & \nearrow f & \uparrow k & \nwarrow g & \\ G & \xrightarrow{i} & G \cup G' & \xleftarrow{i'} & G'. \end{array}$$

Entonces para cualesquiera vértice v y arista h en G tenemos que $k_0(v) = k_0 \circ i(v) = f_0(v)$ y $k_1(h) = k_1 \circ i(h) = f_1(h)$, en cambio si están en G' obtenemos que $k_0(v) = k_0 \circ i'(v) = g_0(v)$ y $k_1(h) = k_1 \circ i'(h) = g_1(h)$. Por lo tanto, $[f, g] = k$ como queríamos probar.

Por lo tanto, $G \cup G'$ es el coproducto de G y G' . $\square$

2 Gramáticas libres de contexto

Siguiendo la línea trazada en el capítulo anterior, en el presente exploraremos el siguiente nivel en la jerarquía de Chomsky, las gramáticas de tipo II o gramáticas libres de contexto.

Los lenguajes regulares, a pesar de su gran utilidad, su restricción a patrones simples los vuelve muy limitados y poco adecuados para la descripción del lenguaje natural. Siguiendo el ejemplo en [5], un lenguaje de la forma $\{a^i b c^i \mid i \in \mathbb{N}\}$ no es regular¹ y corresponde a la anidación de elementos, así oraciones del tipo “Si el pasto es verde, entonces el pasto es verde”, “Si si el pasto es verde, entonces el pasto es verde, entonces el pasto es verde” que se describen con la gramática $\{(Si)^i \text{ el pasto es verde, (entonces el pasto es verde)}^i \mid i \in \mathbb{N}\}$ no serían gramaticalmente válidas en el español, pero lo son (a pesar de su poca naturalidad).

Por ello, surge el estudio de los lenguajes libres de contexto, clásicamente definidos mediante gramáticas de nombre homónimo, árboles de derivación o por medio de autómatas con pila —autómatas que tienen memoria.

Por la vía categórica, en este capítulo mostraremos que gramáticas definidas mediante multicategorías, categorías donde los morfismos tienen como dominio más de un objeto, tienen exactamente el mismo poder expresivo que los lenguajes libres de contexto.

2.1. Multicategorías

Como en el capítulo anterior, comenzaremos dando los ingredientes para describir una multicategoría libre.

Definición 4. Una **signatura de multicategoría** H es un conjunto de nodos H_1 y uno de objetos H_0 junto a un par de funciones

$$H_0^* \xleftarrow{\text{dom}} H_1 \xrightarrow{\text{cod}} H_0$$

Notemos que a diferencia de una signatura simple, en las signaturas de multicategoría los morfismos tienen como dominio palabras en H_0 , es decir, sucesiones (finitas) de objetos.

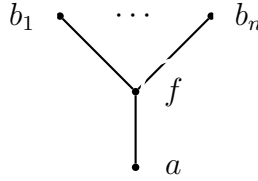
¹Esta es una consecuencia inmediata del lema del bombeo para lenguajes regulares cuyo enunciado y prueba puede consultarse en [4].

Un **morfismo de firmas de multicategorías** $\varphi: H \rightarrow \Gamma$ es un par de funciones $\varphi_0: H_0 \rightarrow \Gamma_0$ y $\varphi_1: H_1 \rightarrow \Gamma_1$ tales que el siguiente diagrama conmuta

$$\begin{array}{ccccc} H_0^* & \xleftarrow{\text{cod}} & H_1 & \xrightarrow{\text{dom}} & H_1 \\ \downarrow \varphi_0^* & & \downarrow \varphi_1 & & \downarrow \varphi_0 \\ \Gamma_0^* & \xleftarrow{\text{cod}} & \Gamma_1 & \xrightarrow{\text{dom}} & \Gamma_1. \end{array}$$

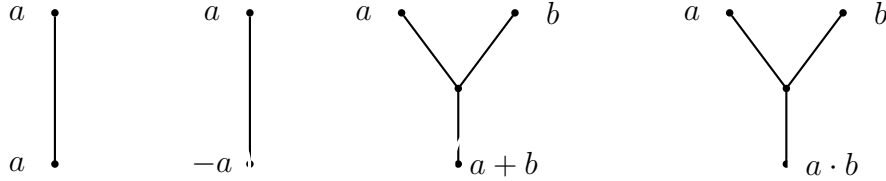
Con tales morfismos, las firmas de multicategorías forman la categoría **MultiSig**.

Dado una firma de multicategoría, denotaremos un nodo $b_1 \dots b_n \xrightarrow{f} a$ gráficamente como:



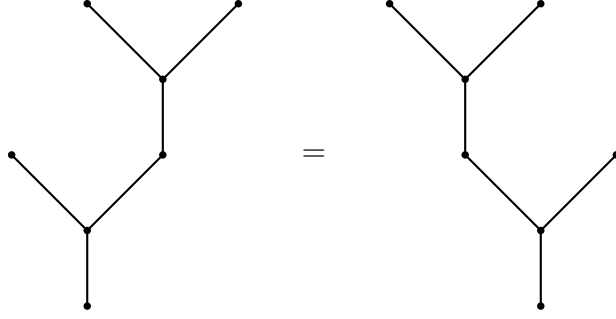
Además, diremos que f tiene aridad n .

Ejemplo 5. Consideremos la siguiente firma de multicategoría donde los objetos corresponderán a los números reales, $\mathbb{R}$. Luego, dado $a, b \in \mathbb{R}^*$ tenemos los siguientes nodos:

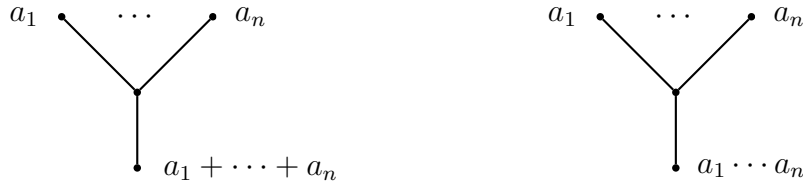


Notemos que los primeros dos nodos corresponden a las operaciones de aridad 1 que lleva un número así mismo o a su inverso aditivo respectivamente. Por otro lado, los últimos dos nodos corresponden a las operaciones binarias usuales de los números reales: la adición y la multiplicación.

Entonces notemos que podemos componer los diagramas anteriores de la manera obvia para poder generar todas las expresiones aritméticas (con sentido). Por lo anterior, podemos resaltar que para cualquier $abc \in \mathbb{R}^*$ tenemos que, dado que nuestras operaciones son asociativas, entonces $a + (b + c) = (a + b) + c$, así gráficamente tenemos la igualdad de los siguientes diagramas:



Por ello, siguiendo un razonamiento inductivo para cualesquiera $a_1 \dots a_n \in \mathbb{R}$ podemos agregar los nodos:



Así, todos los nodos son los representados por alguno de los diagramas anteriores y por sus composiciones sujetos a la identidad de la asociatividad. Con lo anterior, tenemos una signature de multicategoría $\mathbb{R}^* \xleftarrow{\text{dom}} N \xrightarrow{\text{cod}} \mathbb{R}$ donde N es el conjunto de nodos de la forma anterior o composición de los mismos.

Ahora, veamos la definición principal de esta sección:

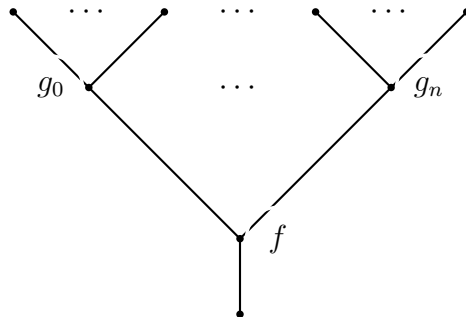
Definición 5. Un **multicategoría** es una signature de multicategoría H junto a una operación de composición

$$\prod_{b_i \in \vec{b}} H_1(\vec{c}_i, b_i) \times H_1(\vec{b}, a) \rightarrow H_1(\vec{c}, a)$$

con $a \in H_0$ y $\vec{c}_i, \vec{b} \in H_0^*$ donde dadas $f: \vec{b} \rightarrow a$ y $g_i: \vec{c}_i \rightarrow b_i$ definimos la composición como:

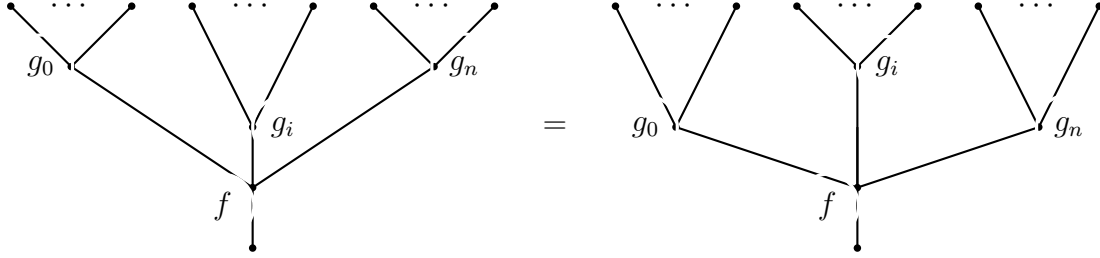
$$(c_1^1, c_2^1, \dots, c_{m_1}^1, c_2^1, \dots, c_1^n, \dots, c_{m_n}^n) \mapsto f(g_1(c_1^1, \dots, c_{m_1}^1), \dots, g_n(c_1^n, \dots, c_{m_n}^n))$$

que representamos gráficamente de la siguiente manera



Además, para cualquier $a \in H_0$ tenemos una identidad $a \xrightarrow{\text{Id}_a} a$ en donde se satisfacen las siguientes propiedades:

1. Para cualquier $f: \vec{b} \rightarrow a$ en H_1 , $f \cdot \text{Id}_a = f = \vec{\text{Id}}_{b_i} \cdot f$; y
2. Para cualesquiera $f: \vec{b} \rightarrow a$ y $g_i: \vec{c}_i \rightarrow b_i$ tenemos que:



Un **álgebra de firmas de multicategorías** $F: \mathbf{H} \rightarrow \mathbf{N}$ es un morfismo de firma de multicategorías compatible con la composición, es decir, cada que $\vec{c} \xrightarrow{\vec{g}} \vec{b} \xrightarrow{f} a$, $F(\vec{g} \cdot f) = F(\vec{g}) \cdot F(f)$ Con estos morfismos, denotamos como **MultiCat** a la categorías de multicategorías.

Es importante resaltar que la propiedad 2 nos indica intuitivamente que no importa el orden en que hagamos las composiciones, es decir, se respeta cierta noción de asociatividad. Aunque todavía no definimos formalmente el significado del cálculo en diagramas, lo analizaremos con precisión en el siguiente capítulo.

La firma de multicategoría del ejemplo 5 es propiamente un multicategoría con la composición dada de la manera natural.

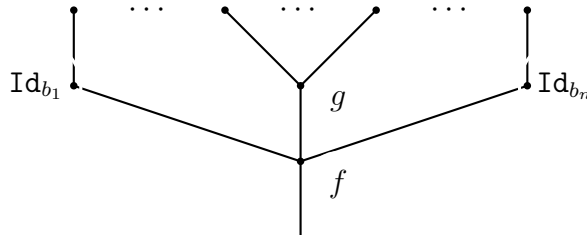
De manera análoga al capítulo anterior, nuestra intención a continuación es definir una multicategoría libre. Antes, veamos la siguiente definición

Definición 6. Sea H una firma de multicategoría. Sean $f: \vec{b} \rightarrow a$ y $g: \vec{c} \rightarrow b_i$ en H_1 con $i \in \{1, \dots, n\}$ con n es la aridad de f .

Definimos la **composición parcial en la i -ésima entrada** como:

$$g \circ_i f = (\text{Id}_{b_1} \cdots g \cdots \text{Id}_{b_n}) \cdot f$$

Gráficamente



Una observación importante es que definimos la composición parcial a partir de la composición, pero también pudimos definirlo inversamente de la siguiente forma:

$$[g_1 \cdots g_n] \cdot f = g_n \circ_n (g_{n-1} \circ_{n-1} \cdots \circ_2 (g_1 \circ_1 f) \cdots)$$

Donde la propiedad 2 de la composición para multicategorías nos garantiza que efectivamente la composición anterior coincide con la usual.

Con ello, ya estamos casi en condiciones de mostrar la construcción de una multicategoría libre, pero antes veamos una definición más.

Definición 7. Sea H una signature de multicategoría y sea $a \in H_0$, definimos recursivamente un **árbol con raíz a de H** , como:

- Un nodo de la forma $w: \varepsilon \rightarrow a$ es un árbol (a los nodos de esta forma, les llamamos hojas);
- Un par $(f, (g_1, \dots, g_n))$ donde f es un nodo de aridad n , $f: a_1 a_2 \cdots a_n \rightarrow a$, y $g_1, \dots, g_n$ son nodos con codominios $a_1, \dots, a_n$ es un árbol;
- Son todos.

Sea H una signature de multicategoría. **Multi**(H) es la multicategoría cuyos objetos son exactamente los mismos que H con los morfismos los árboles en H y para cada objeto $a \in G_0$ tenemos la Id_a . La i -ésima composición parcial para morfismos f, g en **Multi**(H), $g \circ_i f$ está por reemplazar la i -ésima hoja de f por una copia de g .

Decimos que dos árboles en **Multi**(H) son iguales si puede ser obtenido uno a partir de otro mediante la aplicación consecutiva de las propiedades 1 y 2 para la composición en multicategorías.

De acuerdo presentado previamente, la composición parcial anterior induce correctamente la composición y, por lo tanto, **Multi**(H) efectivamente es un multicategoría. Más aún, la construcción anterior es libre en virtud de la siguiente proposición.

Proposición 4. Sean H una signature de multicategorías, $\mathbf{G}$ un multicategoría y $f: G_1 \rightarrow O_1$ una función que preserve la aridad de los nodos. Entonces existe un único morfismo de multicategorías $\varphi: \mathbf{Multi}(H) \rightarrow \mathbf{G}$ tal que el siguiente diagrama conmuta:

$$\begin{array}{ccc} H & \xrightarrow{\eta} & \mathbf{G} \\ \downarrow i & \nearrow \varphi & \\ \mathbf{Multi}(H) & & \end{array}$$

Omitimos la demostración de esta proposición, ya que su argumento es conceptualmente análogo al de la Proposición 1, salvo por los detalles técnicos ya descritos.

2.2. Gramáticas de multicategorías

Sea G una signatura finita de multicategoría. Consideremos sus objetos como símbolos, los nodos como reglas de producción y los morfismos en $\mathbf{Multi}(G)$ como derivaciones, entonces obtenemos la noción de una gramática, más precisamente:

Definición 8. Un **gramática de multicategorías** es una signatura finita de multicategoría de la siguiente forma:

$$(B + V)^* \xleftarrow{\text{dom}} G \xrightarrow{\text{cod}} B$$

donde V es un vocabulario y B es un conjunto de símbolos no terminales con un símbolo distinguido $s \in B$ y G un conjunto de reglas de producciones.

El lenguaje generado por G es:

$$\mathcal{L}(G) = \{u \in V^* \mid \exists g : u \rightarrow s \in \mathbf{Multi}(G)\}.$$

Para comprender la definición anterior, consideremos los siguientes ejemplos.

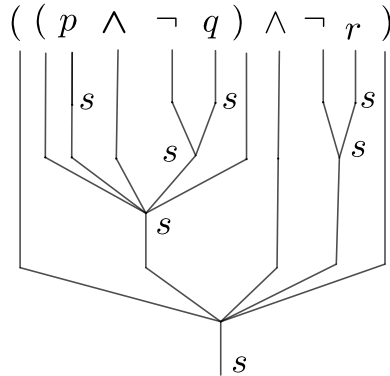
Ejemplo 6. El lenguaje de la lógica proposicional. Sea el conjunto de símbolos terminales vacío, el vocabulario $V = Var \cup \{\wedge, \neg, (,)\}$ donde Var es el conjunto de letras proposicionales con las asignaciones:

$$\text{para todo } p \in Var, \quad p \rightarrow s$$

y las reglas de producción

$$(s \wedge s) \rightarrow s \quad \neg s \rightarrow s$$

Con lo anterior, podemos obtener, por ejemplo, la siguiente derivación.



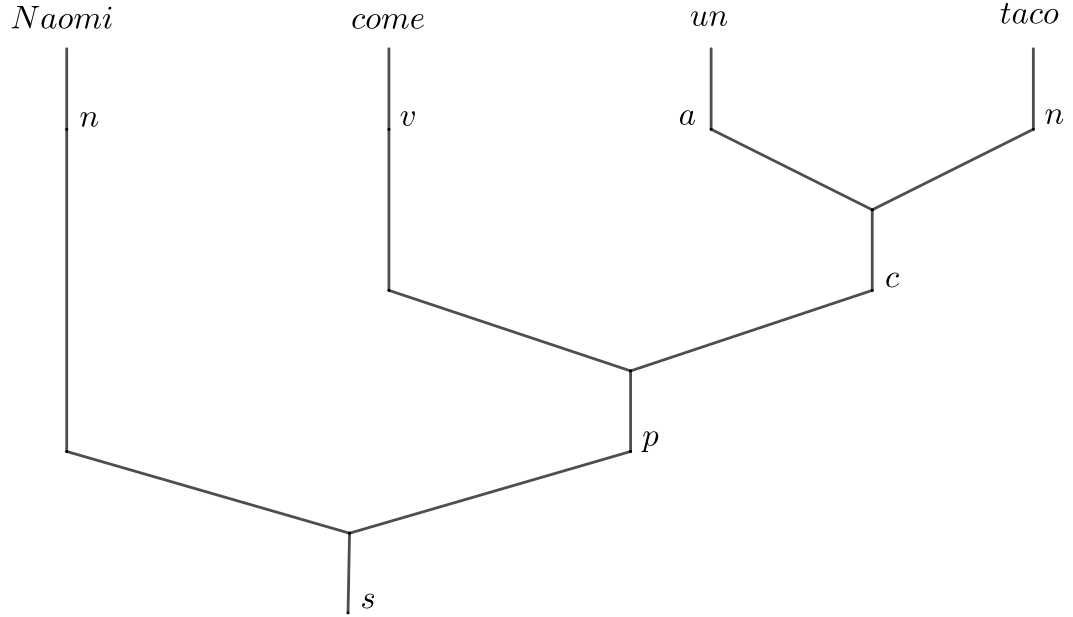
Ejemplo 7. Consideremos el siguiente conjunto de símbolos $B = \{n, v, a, c, p\}$ que corresponde a sustantivo, verbo, artículo, complemento y predicado; el vocabulario $V = \{Naomi, come, un, taco\}$; las asignaciones

$$Naomi \rightarrow n \quad come \rightarrow v \quad un \rightarrow n \quad taco \rightarrow n$$

y las reglas de producción

$$(a, n) \rightarrow c \quad (v, c) \rightarrow p \quad (n, p) \rightarrow s$$

Tenemos la siguiente derivación



Ahora, como anunciamos al inicio del capítulo, mostraremos la relación entre los lenguajes anteriores con la jerarquía de Chomsky. Para ello, veamos las siguientes definiciones y resultados.

Definición 9. Una gramática libre de contexto es una tupla $G = (V, B, P, s)$ donde V es un vocabulario, B es un conjunto de símbolos no terminales con un símbolo distinguido s y P es un conjunto de reglas de producción de la forma $A \rightarrow \alpha$ donde A es un símbolo no terminal y $\alpha \in (V + B)^*$.

Si $A \rightarrow \alpha$ es una regla de producción, para cualquier $\beta, \gamma \in (V + B)^*$ definimos la derivación $\beta A \gamma \Rightarrow \beta \alpha \gamma$. Consideramos $\Rightarrow^*$ como la cerradura transitiva de $\Rightarrow$.

Definimos el lenguaje generado por G como:

$$\mathcal{L}(G) = \{w \in V^* \mid s \Rightarrow^* w\}.$$

Decimos que un lenguaje L es libre de contexto si existe una gramática G libre de contexto tal que $L = \mathcal{L}(G)$.

Observemos que una oración en un lenguaje libre de contexto corresponde a una cadena en V^* que puede ser derivada a partir de s en G .

Ahora, la manera anterior de describir el lenguaje es eficiente para la generación de nuevas expresiones, sin embargo, puede no ser muy útil para el propósito de discriminar si una expresión dada pertenece, o no, al lenguaje. Con ese propósito surgen los árboles de derivación.

Definición 10. Sea $G = (V, B, P, s)$ una gramática libre de contexto.

Un árbol es un árbol de derivación para G si:

- Cada vértice está etiquetado con un símbolo de $V \cup B \cup \{\varepsilon\}$;
- la raíz del árbol es s ;
- si un vértice interior está etiquetado con A , entonces $A \in B$, es decir, no es un símbolo terminal;
- si un vértice A tiene n hijos, $A_1, \dots, A_n$, entonces:

$$A \rightarrow A_1 A_2 \cdots A_n$$

es una regla de producción en P ; y

- si un vértice está etiquetado con ε , entonces es un hoja y, además, es el único hijo de su padre.

Notemos que el diagrama del ejemplo uno corresponde a un árbol de derivación en la gramática $A \rightarrow \neg A \mid A \wedge A \mid A \vee A \mid A \rightarrow A \mid A \leftrightarrow A \mid a_0 \mid a_1 \mid a_2$.

Con esto, podemos enunciar la siguiente proposición cuya prueba podemos consultar en [4].

Proposición 5. *Sea $G = (V, B, P, s)$ una gramática libre de contexto. Entonces $s \Rightarrow^* w$ si y solo si existe un árbol de derivación para G que produce a w .*

Dada la definición de morfismos en una multicategoría libre, es inmediato notar que los morfismos de la forma $f: u \rightarrow s$ con $u \in V^*$ en una gramática de multicategoría son árboles de derivaciones y viceversa. Así, tenemos el siguiente teorema con el cual terminamos el presente capítulo.

Teorema 2. *Sea G una signatura de multicategoría y G' una gramática libre de contexto, entonces*

1. $L(G)$ es un lenguaje libre de contexto, y
2. existe una signatura de multigráfica que genera a $L(G')$.

3 Gramáticas recursivamente enumerables

La introducción de los lenguajes libres de contexto aumenta ampliamente la clase de lenguajes que podemos describir formalmente; sin embargo, consideremos las siguientes oraciones presentadas en [7]: “Ayer una mujer llegó que conocía”, “Ayer un mujer y un hombre llegaron a los que cononocía y admiraba”. Notemos que la concordancia entre los sujetos “hombre y mujer” y el verbo “llegaron” y el pronombre relativo “los que” marca la estructura de oración $xa^ib^ic^id^i$, el cual no es un lenguaje regular.

En este capítulo daremos por terminado el estudio de la jerarquía de Chomsky por medio de la teoría de categorías, presentando los lenguajes recursivamente enumerables que permiten representar oraciones como las anteriores, los cuales son comúnmente definidos de la siguiente forma:

- son los lenguajes producidos por una gramática de tipo 0, las cuales son las gramáticas generativas cuyas reglas de producción tienen cero restricciones; y
- son los lenguajes reconocidos por una máquina de Turing.

En esta ocasión, las categorías adecuadas para trabajar los lenguajes recursivamente enumerables son las categorías monoidales que, como su nombre lo indica, son categorías dotadas de una operación binaria que cumple propiedades análogas a la estructura de un monoide algebraico. Además, tienen asociado un cálculo gráfico, es decir, existen diagramas que nos permiten representar los morfismos en la categoría y, más aún, nos permiten efectuar operaciones en ellos mediante deformaciones. Estos diagramas nos permitirán definir las categorías monoidales libres de forma combinatoria.

Finalmente, mostraremos que los lenguajes generados por una categoría monoidal son exactamente los lenguajes recursivamente enumerables.

3.1. Categorías monoidales

Definición 11. Una **signatura monoidal** G es una signatura de la siguiente forma:

$$G_0^* \xleftarrow{\text{dom}} G_1 \xrightarrow{\text{cod}} G_0^*$$

Un **morfismo de firmas monoidales** $\varphi : G \rightarrow \Gamma$ es un par de funciones $\varphi_0 : G_0 \rightarrow \Gamma_0$ y $\varphi_1 : G_1 \rightarrow \Gamma_1$ tales que el siguiente diagrama conmuta:

$$\begin{array}{ccccc} G_0^* & \xleftarrow{\text{cod}} & G_1 & \xrightarrow{\text{dom}} & G_0^* \\ \downarrow \varphi_0 & & \downarrow \varphi_1 & & \downarrow \varphi_0 \\ \Gamma_0^* & \xleftarrow{\text{cod}} & \Gamma_1 & \xrightarrow{\text{dom}} & \Gamma_0^* \end{array}$$

Con tales morfismos, las firmas monoidales forman la categoría **MonSig**.

Definición 12. Una **categoría monoidal estricta** $(\mathcal{C}, \otimes, 1)$ es una categoría $\mathcal{C}$ con un bifunctor¹ $\otimes : \mathcal{C} \times \mathcal{C} \rightarrow \mathcal{C}$ y un objeto distinguido 1 en $\mathcal{C}$ en la que se satisfacen las siguientes ecuaciones:

1. $1 \otimes f = f = f \otimes 1$
2. $f \otimes (g \otimes h) = (f \otimes g) \otimes h$

para cualesquiera f, g, h morfismos en $\mathcal{C}$.

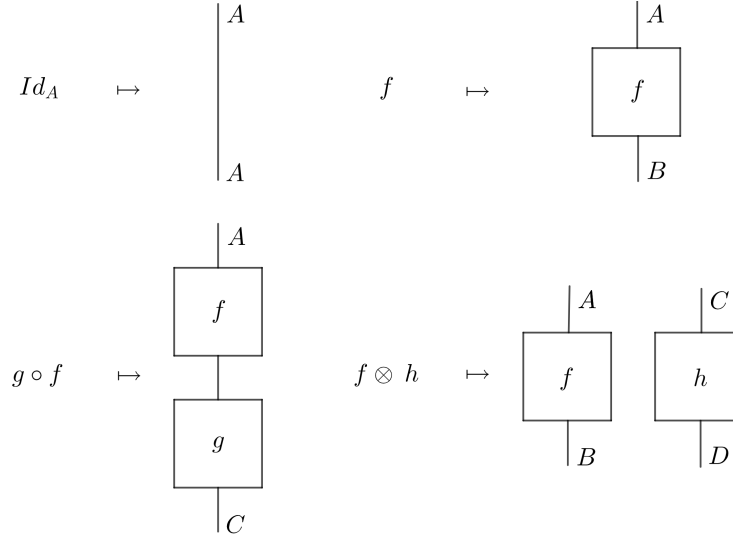
Un funtor monoidal estricto es un funtor entre categorías monoidales $F : \mathcal{C} \rightarrow \mathcal{D}$ que preserve el producto tensorial, es decir, $F(g \otimes_{\mathcal{C}} f) = F(g) \otimes_{\mathcal{D}} F(f)$. A la categoría de categorías monoidales con morfismos los funtores monoidales la denotamos **MonCat**.

Notemos que en una categoría monoidal nuestros morfismos pueden componerse de dos maneras. Dado $f : A \rightarrow B$ y $g : B \rightarrow C$ morfismos podemos construir la composición $g \circ f : A \rightarrow C$. Pero también dados $u : A \rightarrow C$ y $v : B \rightarrow D$ podemos usar nuestro funtor para formar el morfismo $u \otimes v : A \otimes B \rightarrow C \otimes D$.

Es usual interpretar ambas composiciones de la siguiente manera: la composición usual es secuencial, es decir, primero se ejecuta el primer morfismo y después aplicamos el segundo; y la composición monoidal es paralela, es decir, ejecutamos ambos morfismos de manera simultánea.

Las consideraciones anteriores motivan un lenguaje gráfico para categorías monoidales mediante cables y cajas. Definimos recursivamente el lenguaje gráfico de la siguiente manera: Sean A, B, C, D objetos y $f : A \rightarrow B$, $g : B \rightarrow C$ y $h : C \rightarrow D$ morfismos en una categoría monoidal, tenemos la siguiente representación gráfica. Estos diagramas son llamados **diagramas de cables**.

¹Un bifuntor es un funtor en cada entrada



Como es de esperarse, las dos operaciones entre los morfismos de nuestra categorías se “comportan bien” entre sí, como veremos en la siguiente proposición.

Proposición 6. Sea $(\mathcal{C}, \otimes, 1)$ una categoría monoidal y sean $f : A \rightarrow B$, $g : B \rightarrow C$, $h : D \rightarrow E$ y $j : E \rightarrow F$ morfismos de $\mathcal{C}$, entonces

$$(g \circ f) \otimes (j \circ h) = (g \otimes j) \circ (f \otimes h), \quad (3.1)$$

$$(Id_B \otimes h) \circ (f \otimes Id_D) = f \otimes h = (f \otimes Id_E) \circ (Id_A \otimes h). \quad (3.2)$$

Demostración. Sean $f : A \rightarrow B$, $g : B \rightarrow C$, $h : D \rightarrow E$ y $j : E \rightarrow F$ morfismos en una categoría monoidal $\mathcal{C}$.

Primero probemos 3.1.

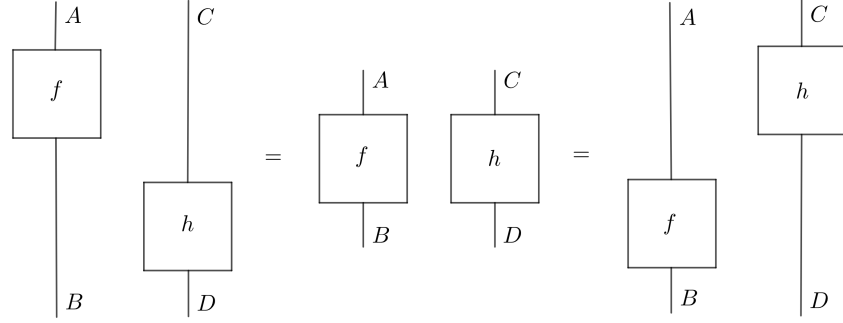
$$\begin{aligned} (g \circ f) \otimes (j \circ h) &= \otimes(g \circ f, j \circ h) \\ &= \otimes((g, j)) \circ (f, h) \\ &= (\otimes(g, j)) \circ (\otimes(f, h)) && \text{por la bifuntorialidad de } \otimes \\ &= (g \otimes j) \circ (f \otimes h). \end{aligned}$$

Ahora, veamos 3.2

$$\begin{aligned} (Id_B \otimes h) \circ (f \otimes Id_D) &= (Id_B \circ f) \otimes (h \circ Id_D) && \text{Por 3.1} \\ &= f \otimes h \\ &= (f \circ Id_A) \otimes (Id_E \circ h) \\ &= (f \otimes Id_E) \circ (Id_A \otimes h) && \text{Por 3.1} \end{aligned}$$

□

Podemos expresar en el lenguaje gráfico de categorías monoidales la ecuación 3.2 de la siguiente forma:



La ecuación anterior sugiere que siempre podemos encontrar diagramas donde cada caja este en un nivel distinto al resto, así que probémoslo, pero antes veamos la siguiente definición para poder enunciar nuestra proposición.

Definición 13. Un diagrama de cables está en **posición general** si no hay otra caja a la derecha o a la izquierda de cada caja.

Proposición 7. *Todo diagrama de cables es equivalente a un diagrama de cables en posición general.*

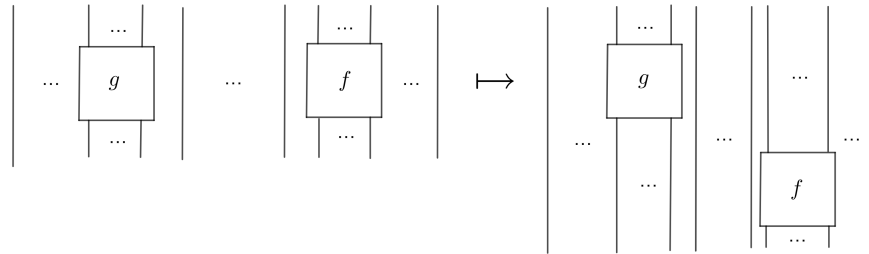
Demostración. Sea D un diagrama de cables. Haremos la demostración por inducción sobre el número de cajas en el diagrama.

La base de inducción es trivial, pues si D tiene una caja ya acabamos.

Supongamos que el resultado es cierto para algún n y que D tiene $n + 1$ cajas.

Sea f la caja ubicada más a la derecha y abajo en el diagrama y consideremos D' el diagrama obtenido al sustituir f por $id_{dom(f)}$. Entonces D' tiene n cajas y por hipótesis es equivalente a un diagrama de cajas G' en posición general; consideremos ahora el diagrama G que se obtiene al pegarle f en el cable al que se lo habíamos cortado.

Observemos que la caja que colocamos sigue siendo la más derecha y abajo, y a su izquierda hay a lo más una caja g (si no, ya acabamos), pues G' está en posición general. Entonces basta hacer la siguiente deformación o, en otras palabras, aplicar 3.1.



□

Notemos que el argumento de la prueba anterior fue expresado de dos formas distintas, por medio de la igualdad 3.1 y por su representación en el diagrama de cables. Sin embargo, es válido si el lector considera que fue una coincidencia muy afortunada y tenga sus reservas. Sin embargo, el siguiente teorema debido a A. Joyal y R. Street

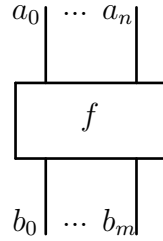
nos iluminará en el uso de diagramas de cables, cuya demostración puede consultarse en [10]

Teorema 3 (Teorema de coherencia del cálculo gráfico en categorías monoidales). *Una ecuación $f = g$ bien formada² entre morfismos de una categoría monoidal es válida en la categoría monoidal si y sólo si los diagramas de cables de f y g son equivalentes salvo isotopías planas.*

Dos diagramas de cables son equivalentes salvo isotopías planas si es posible transformar uno en el otro moviendo continuamente las cajas y cables sin permitir que estos se crucen entre sí.

La proposición y teorema anterior serán fundamentales en nuestro objetivo: generar una categoría monoidal libre a partir de una signatura monoidal. Para ello, primero describiremos un lenguaje gráfico para las signaturas monoidales ya que lo usaremos en la descripción de categorías monoidales.

Sea G una signatura monoidal y sea $f : \vec{a} \rightarrow \vec{b} \in G_1$. Diremos que f es una caja y la representaremos como:



Además, tenemos dos casos distinguidos que corresponden a morfismos de la forma $u : \varepsilon \rightarrow a$ y $w : b \rightarrow \varepsilon$ cuya representación es la siguiente.



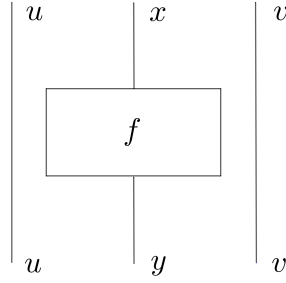
Usaremos la descripción anterior para construir la categoría monoidal libre $\mathbf{MC}(G)$ asociada a una signatura monoidal; para ello, primero veamos qué es una signatura de niveles.

Dada G una signatura monoidal definimos la **signatura de niveles** de G como

$$G_0^* \xleftarrow{\text{cod}} L(G) = G_0^* \times G_1 \times G_0^* \xrightarrow{\text{dom}} G_0^*$$

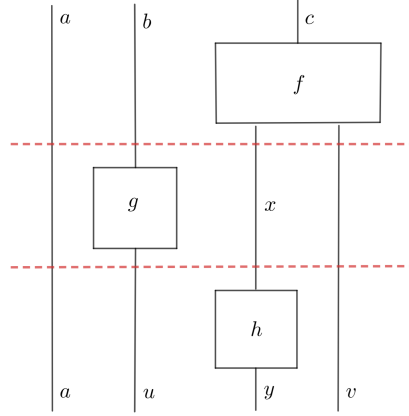
²Una ecuación entre morfismos es bien formada si los morfismos efectivamente son parte la categoría y poseen tanto el mismo dominio como codominio

donde para cada nivel $l = (u, f : x \rightarrow y, v)$ en $L(G)$ definimos $\text{dom}(l) = uxv$ y $\text{cod}(l) = uyv$, gráficamente representamos a l de la siguiente manera



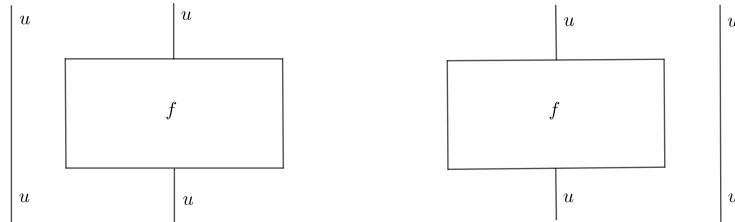
Sea $\mathbf{PMC}(G)$, la categoría libre cuyos objetos son los diagramas premonoidales y el conjunto de morfismos generadores son secuencias de niveles de la forma $(d_1, \dots, d_n)$ tales que $\text{cod}(d_i) = \text{dom}(d_{i+1})$ para cada $i \in \{1, \dots, n-1\}$.

Veamos el siguiente ejemplo de un diagrama premonoidal generado con la signatura $G_0 = \{a, b, c, u, v, y, x\}$ y $G_1 = \{f : c \rightarrow xv, g : b \rightarrow u, h : x \rightarrow y\}$.



Supongamos que queremos construir el diagrama a partir de la signatura, ¿qué información necesitamos? En principio, conocer el número de niveles que conforma el diagrama, en nuestro caso tres, luego conocer el dominio y codominio del diagrama, abc y $auyv$ respectivamente, además de la lista de cajas en cada nivel, $[f, g, h]$. Finalmente, necesitamos conocer la posición en la que se encuentra la caja en cada nivel, lo cual lo identificamos señalando el número de cables a la izquierda de la caja, $(2, 1, 2)$.

Aunque la última condición es innecesaria en el diagrama anterior, es importante en casos como el siguiente, donde los primeros datos no determinan un único diagrama premonoidal.



La explicación anterior motiva nuestra siguiente proposición, una manera eficiente de representar la información contenida en un diagrama premonoidal:

Proposición 8 (Codificación de diagramas premonoidales). *Sea G una signatura monoidal. Un diagrama premonoidal d en $\mathbf{PMC}(G)$ está únicamente determinada de la siguiente forma:*

1. un dominio: $\text{dom}(d) \in G_0^*$
2. un codominio: $\text{cod}(d) \in G_0^*$
3. una lista de cajas: $\text{cajas}(d) \in G_1^n$
4. una lista de números identificadores: $\text{id}(d) \in \mathbb{N}^n$

donde n es el número de niveles del diagrama d y el n -ésimo identificador señala la posición de la caja en el n -ésimo nivel, indicando el número de “cables” a la izquierda de la caja.

Más aún, la información anterior define un diagrama premonoidal válido si para cada $i \in \{1, \dots, n\}$ tenemos que

$$\text{peso}(d)_i \geq \text{id}(d)_i + |\text{dom}(b_i)|$$

donde

$$\text{peso}(d)_1 = |\text{dom}(d)| \quad \text{peso}(d)_{i+1} = \text{peso}(d)_i + |\text{cod}(b_i)| - |\text{dom}(b_i)|$$

con $b_i = \text{boxes}(d)_i$.

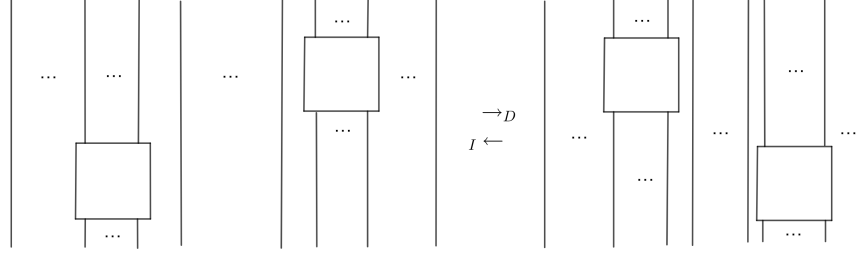
Notemos que la función **peso** indica el número de cables con los que inicia cada nivel; así, la desigualdad significa que este número debe ser mayor que el número de cables que estarán a la izquierda de la caja menos el número de cables que necesita la caja del nivel. Bajo estas condiciones, es evidente que el diagrama generado será válido.

Ahora bien, notemos que los elementos de $\mathbf{PMC}(G)$ se parecen mucho a diagramas de cables en posición general; sin embargo, necesitamos establecer cuándo dos diagramas premonoidales son equivalentes. Para ello, veamos las siguientes definiciones.

Definición 14. Sea G una signatura y d un diagrama premonoidal de G . Decimos que d admite un intercambio izquierdo en el nivel n si $\text{id}(d)_{n+1} \geq \text{id}(d)_n + |\text{cod}(b_n)|$ y admite un intercambio derecho en el nivel $n + 1$ si $\text{id}(d)_n \geq \text{id}(d)_{n+1} + |\text{dom}(b_n)|$.

En otras palabras, un intercambio es posible cuando las cajas a intercambiar no tienen cables en común. Ahora sí, veamos qué es un intercambio.

Definición 15. Sea G una signatura y d un diagrama premonoidal de G . Si d admite un intercambio izquierdo (o derecho) en el nivel n , entonces generamos un nuevo diagrama idéntico en todos los niveles a d salvo en n y $n + 1$ donde hay un cambio de la siguiente forma:



Decimos que el nuevo diagrama es el **intercambio izquierdo (o derecho) en el nivel n de d** .

Claramente los diagramas producidos tras un intercambio son también un diagrama válido. Definimos una reducción de diagramas como una secuencia de intercambios izquierdos o derechos. Con ello, definimos la siguiente relación: dos diagramas d y d' están $\sim$ -relacionados si existe una reducción que lleve d a d' .

Proposición 9. *Sea G una signatura monoidal. La relación $\sim$ sobre el conjunto de diagramas premonoidales de G es de equivalencia.*

Demostración. Sea G una signatura monoidal. Sea d una diagrama premonoidal. Consideramos la secuencia de reducciones vacía, entonces $d \sim d$. Por lo tanto, $\sim$ es reflexiva.

Sea d y d' diagramas tales que $d \sim d'$. Sea $p_1^{r_1}, p_2^{r_2}, \dots, p_n^{r_n}$ la secuencia de reducciones que lleva d a d' con $r_i \in \{d, i\}$ para todo $i \in \{1, \dots, n\}$, es decir, indica si el intercambio fue izquierdo o derecho. Notemos que un intercambio izquierdo es el inverso del derecho y viceversa, entonces consideremos r'_i como el opuesto a r_i . Entonces la reducción $p_n^{r'_n}, p_{n-1}^{r'_{n-1}}, \dots, p_1^{r'_1}$ lleva d' a d . Por lo tanto, $d' \sim d$, así tenemos que $\sim$ es simétrica.

Por último, sean d, d' y d'' tales que $d \sim d'$ y $d' \sim d''$. Entonces hay una reducción que lleva d a d' y una que lleva d' a d'' , así hay una reducción que lleva d a d'' , es decir, $d \sim d''$. Por lo tanto, $\sim$ es transitiva. Por lo tanto, $\sim$ es de equivalencia. $\square$

Otra forma de expresar la proposición anterior es señalando que cualesquiera dos diagramas premonoidales tales que pueda deformarse, sin cruzar o cortar cables, uno en el otro están $\sim$ -relacionados. Con todo lo anterior, ya estamos en condiciones de conocer a nuestra categoría monoidal libre, $\mathbf{MC}(G)$.

Proposición 10. *Sea G una signatura monoidal, entonces definimos la categoría monoidal libre de G como*

$$\mathbf{MC}(G) \cong \mathbf{PMC}(G) / \sim$$

donde $\mathbf{PMC}(G) / \sim$ es la categoría premonoidal libre con la identificación en morfismos: si $f \sim g$, entonces $f = g$.

Demostración. Sea G una signatura monoidal. Mostremos que $\mathbf{PMG}(G) / \sim$ es una categoría monoidal.

Notemos que cualquier combinación con sentido de elementos de G_1 con $\circ$ y $\otimes$ puede expresarse como un diagrama de cables, el cual a su vez puede transformarse a un

diagrama equivalente en posición general en virtud de la proposición 7. A su vez, todo diagrama en posición general puede ser expresado como un diagrama premonoidal, por lo tanto, cualquier morfismo en $\mathbf{MC}(G)$ se encuentra en $\mathbf{PMC}(G)$.

Por otro lado, la proposición 9 nos dice que cualquiera dos diagramas premonoidales tales que pueda deformarse, sin cruzar o cortar cables, son iguales. Pero de acuerdo al teorema de coherencia para el cálculo gráfico podemos concluir que $\mathbf{PMC}(G)/\sim$ satisface los axiomas de una categoría monoidal. $\square$

3.2. Gramáticas monoidales

Veamos como podemos darle una interpretación lingüística a una signatura monoidal G . Podemos considerar los objetos de $\mathbf{MC}(G)$ como cadenas de símbolos de G_0 , las cajas (o flechas generadores) de G_1 como reglas de producción y los morfismos en $\mathbf{MC}(G)$ como derivaciones. Formalmente podemos definir un lenguaje monoidal de la siguiente manera:

Definición 16. Una **gramática monoidal** es una signatura monoidal finita G de la siguiente forma

$$(V + B)^* \xleftarrow{\text{dom}} G \xrightarrow{\text{cod}} (V + B)^*$$

donde V es un conjunto de palabras llamada vocabulario y B es un conjunto de símbolos con un símbolo inicial distinguido $s \in B$ llamado el símbolo de oración.

Una oración $u \in V^*$ es gramaticalmente correcta si hay un morfismo $g : u \rightarrow s$ en $\mathbf{MC}(G)$. Definimos el lenguaje generado por G como:

$$\mathcal{L}(G) = \{u \in V^* \mid \exists g : u \rightarrow s \text{ en } \mathbf{MC}(G)\}.$$

A los lenguajes generados por gramáticas monoidales les llamamos lenguajes monoidales.

Ejemplo 8. Sabemos que los lenguajes libres de contextos son producidos por gramáticas de hipergráficas de la siguiente forma:

$$(V + B)^* \leftarrow G \rightarrow B$$

Ya que $B \subset (V + B)^*$, entonces son signaturas monoidales y, por lo tanto, es un lenguaje monoidal.

Tal como se prometió al inicio del capítulo, veremos que los lenguajes monoidales son exactamente los lenguajes recursivamente enumerables. Pero antes convendrá recordar las dos definiciones equivalentes, la primera en términos de las gramáticas que los generen y la segunda en términos del modelo de cómputo que los aceptan.

Definición 17. Una gramática de tipo 0 (o sin restricciones) es una tupla $G = (V, B, P, s)$ donde V es un vocabulario, B es un conjunto de símbolos no terminales con un símbolo distinguido s y P es un conjunto de reglas de producción de la forma $\alpha \rightarrow \beta$ donde $\alpha, \beta \in (V + B)^*$.

Si $\alpha \rightarrow \beta$ es una regla de producción, para cualquier $\gamma, \delta \in (V + B)^*$ definimos la derivación

$$\gamma\alpha\delta \Rightarrow \gamma\beta\delta.$$

Definimos el lenguaje generado por G como:

$$\mathcal{L}(G) = \{v \in V^* | s \Rightarrow^* v\}$$

donde $\Rightarrow^*$ es la cerradura reflexiva y transitiva de $\Rightarrow$.

Teorema 4. *Los lenguajes monoidales son exactamente los generados por una gramática de tipo 0.*

Demostración. Primero veamos que todo lenguaje monoidal es producido por una gramática de tipo 0.

Sea G una gramática monoidal. Para cada $f : u \rightarrow v$ en G_1 definimos la regla de producción $p_f : v \rightarrow u$ en P con $v, u \in (V + B)^*$. Notemos que si existen dos flechas generadoras $f, g : u \rightarrow v$, entonces $p_f = p_g$.

Sea $w \in \mathcal{L}(G)$, entonces existe $f : w \rightarrow s$ en $\mathbf{MC}(G)$, eso implica existen flechas generadoras tal que el siguiente diagrama conmuta

$$\begin{array}{ccc} w & \xrightarrow{f_1} \dots \xrightarrow{f_n} & s. \\ & \searrow f & \nearrow \end{array}$$

Entonces tenemos la siguiente derivación de P :

$$s \Rightarrow \dots \Rightarrow w$$

es decir, tenemos que $s \Rightarrow^* w$ y, por lo cual, w está en el lenguaje generado por la gramática de tipo 0 (V, B, P, s) . Por lo tanto, $\mathcal{L}(G) \subset \mathcal{L}(V, B, P, s)$. La otra contención se da leyendo el argumento anterior al revés y, por lo tanto, $\mathcal{L}(G) = \mathcal{L}(V, B, P, s)$. Resta probar que todo lenguaje generado por una gramática de tipo 0 es monoidal, aunque podríamos dar una demostración análoga, pospondremos su prueba. $\square$

Como hemos mencionado anteriormente, cada nivel dentro de la jerarquía de Chomsky tiene asociado un modelo de cómputo capaz de reconocerlo, introduzcamos el modelo asociado a los lenguajes generados por una gramática de tipo 0.

Definición 18. Sea $M = (Q, \Sigma, \Gamma, \delta, q_0, \sqcup, F)$ una máquina de Turing.

Definimos un movimiento de la siguiente manera: Dado $X_1X_2 \dots X_{i-1}qX_i \dots X_n$ en la banda y $\delta(q, X_i) = (p, Y, \leftarrow)$ tenemos que:

$$X_1X_2 \dots X_{i-1}qX_i \dots X_n \vdash_M X_1X_2 \dots X_{i-2}pX_{i-1}Y \dots X_n.$$

Pero si $\delta(q, X_i) = (p, Y, \rightarrow)$, entonces

$$X_1X_2 \dots X_{i-1}qX_i \dots X_n \vdash_M X_1X_2 \dots X_{i-1}YpX_{i+1} \dots X_n.$$

Definimos $\vdash_M^*$ como la cerradura reflexiva y transitiva de $\vdash_M$.

El lenguaje M aceptado por una máquina de Turing se define como

$$\mathcal{L}(M) = \{w \in \Sigma^* | q_0w \vdash_M^* \alpha s \beta \text{ con } s \in F \text{ y } \alpha, \beta \in \Gamma^*\},$$

es decir, el lenguaje producido por una máquina de Turing es el conjunto de cadenas del vocabulario tales que al inicializarse la banda con esas cadenas logramos alcanzar algún estado final. Ya estamos en condiciones de dar la siguiente definición:

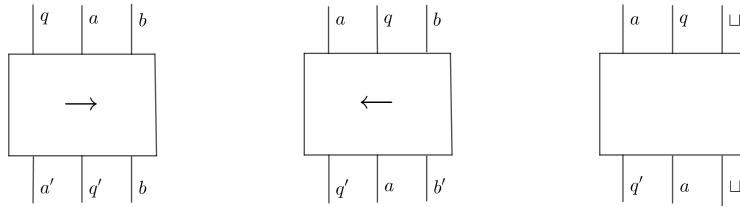
Definición 19. Un lenguaje L es recursivamente enumerable si existe una máquina de Turing M que acepta L , es decir, $L = L(M)$.

Un resultado bien conocido es que los lenguajes recursivamente enumerables tienen el mismo poder expresivo que los lenguajes generados por una gramática de tipo 0 [4]. Probemos que los lenguajes recursivamente enumerables son lenguajes monoidales.

Proposición 11. Para todo lenguaje recursivamente enumerable L , existe una gramática monoidal G tal que $L = \mathcal{L}(G)$.

Demostración. Sea L un lenguaje recursivamente enumerable generado por la máquina de Turing $M = (Q, \Sigma, \Gamma, \delta, q_0, \sqcup, F)$.

Definimos $B = (\Gamma \setminus \Sigma) \cup Q$ y $V = \Sigma$ con $s \in B$ el símbolo de oración. Las reglas de producción son las siguientes:



donde $a, a', b, b' \in \Sigma$, $q, q' \in Q$ tales que $\delta(q, a) = (q', a', \rightarrow)$, $\delta(q, b) = (q', b', \leftarrow)$ y $\delta(q, \sqcup) = (q', \sqcup, \rightarrow)$. Además, agregamos las reglas de producción

$$x \rightarrow q_0 x \quad y \quad x s y \rightarrow s$$

necesarias para inicializar la banda y para limpiarla.

Veamos que $L = \mathcal{L}(G)$. Sea $w \in L$, entonces $q_0 w \vdash_M^* \alpha s \beta$. Pero notemos que para cada aplicación $\vdash_M$ tenemos un morfismo que actúa de la misma manera, entonces tenemos morfismos tales que

$$q_0 w \xrightarrow{f_1} \dots \xrightarrow{f_n} \alpha s \beta$$

Además como también tenemos las reglas de producción $w \rightarrow q_0 w$ y $\alpha s \beta \rightarrow s$ tenemos una derivación $w \rightarrow s$ en G . Por lo tanto, $w \in \mathcal{L}(G)$.

Para el regreso, sea $w \in \mathcal{L}(G)$, entonces existe una derivación $w \rightarrow s$. Observemos que por las reglas producción, tenemos la derivación es necesariamente de la siguiente forma

$$w \longrightarrow q_0 w \longrightarrow \dots \longrightarrow \alpha s \beta \longrightarrow s,$$

pero la derivación intermedia solo puede lograrse con reglas de producción de la forma $\leftarrow$ y $\rightarrow$, las cuales corresponden con derivaciones en $\vdash_M$, por lo tanto, $q_0 w \vdash_M^* \alpha w \beta$. Por lo tanto, $w \in L$. Por lo tanto, $L = \mathcal{L}(G)$. $\square$

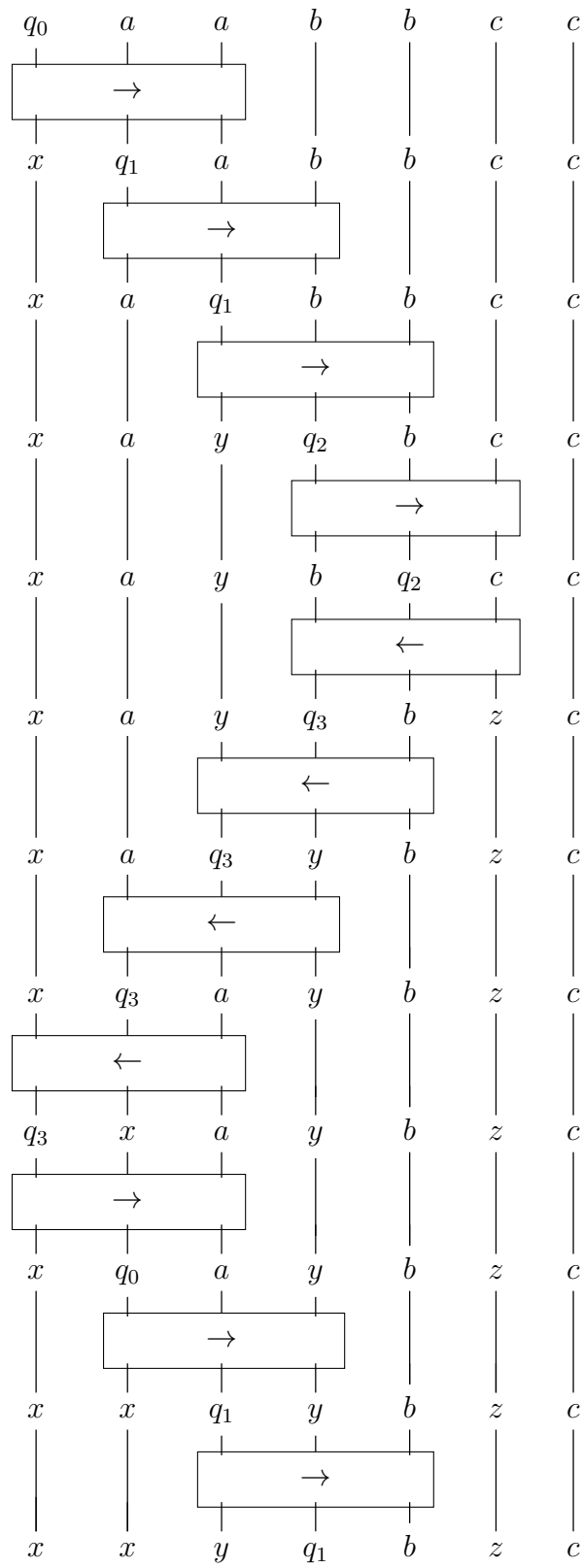
Terminemos este capítulo con el siguiente ejemplo.

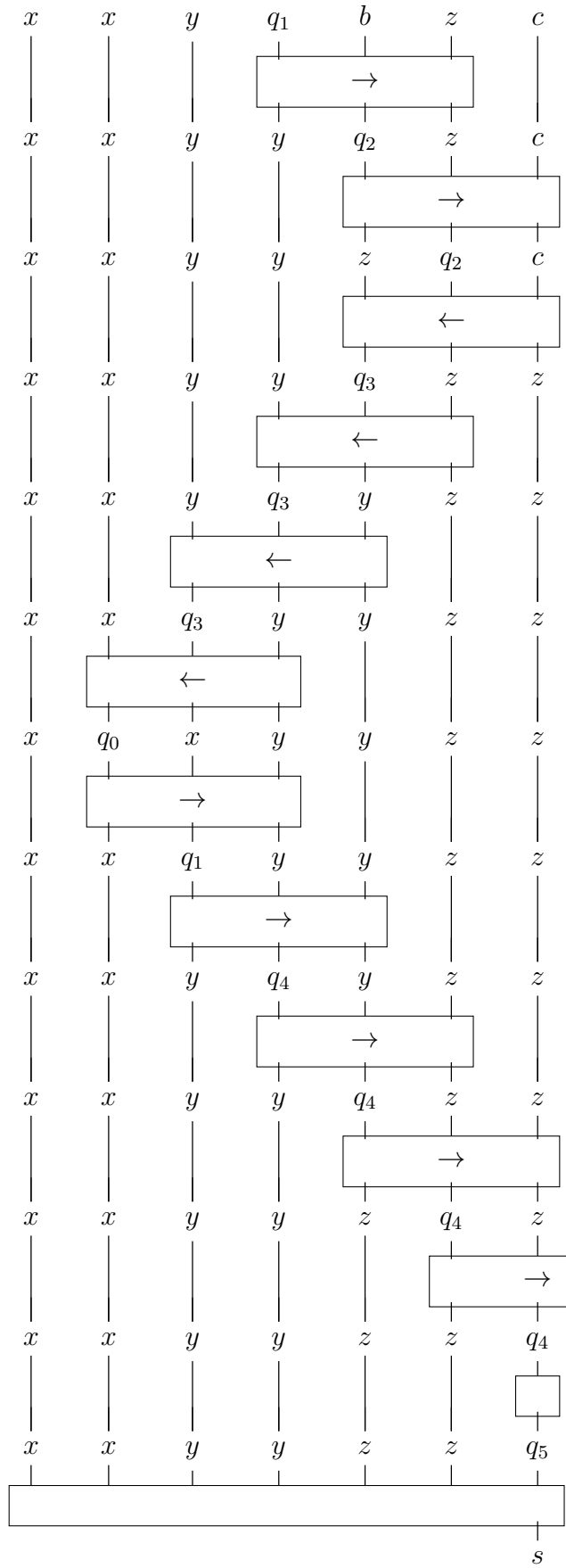
Ejemplo 9. El lenguaje $L = \{a^i b^i c^i | i \in \mathbb{N}\}$ no es regular, pero sí es recursivamente enumerable.

Una máquina de Turing que acepte el lenguaje anterior es la siguiente:

δ	a	b	c	x	y	z	$\sqcup$
q_0	$q_1, x, \rightarrow$				$q_4, y, \rightarrow$		$q_5, \sqcup, \rightarrow$
q_1	$q_1, a, \rightarrow$	$q_2, y, \rightarrow$			$q_1, y, \rightarrow$		
q_2		$q_2, b, \rightarrow$	$q_3, z, \leftarrow$			$q_2, z, \rightarrow$	
q_3	$q_3, a, \leftarrow$	$q_3, b, \leftarrow$	$q_3, c, \leftarrow$	$q_0, x, \rightarrow$	$q_3, y, \leftarrow$	$q_3, z, \leftarrow$	
q_4					$q_4, y, \rightarrow$	$q_4, z, \rightarrow$	$q_5, \sqcup, \rightarrow$
q_5							$q_5, s, \rightarrow$

En las siguientes dos páginas presentamos la derivación en la gramática monoidal asociada de la palabra $aabbcc$.





4 Gramáticas categoriales

El gran poder expresivo con el que cuentan los lenguajes descritos en el capítulo anterior se paga con un precio bastante caro: la eficiencia computacional.

El problema del paro nos dice lo siguiente: dada una máquina de Turing M y una entrada w no hay garantía de que la ejecución de M con la entrada w termine. Se puede consultar en [4] que no es posible dada una gramática monoidal G y $w \in \Sigma^*$ determinar siempre si existe o no un morfismo $f: w \rightarrow s$ reduciendo este problema al del paro.

Por ello, en este capítulo introducimos un enfoque al estudio de las gramáticas de los lenguajes de naturales, basadas en la asociación de elementos gramaticales con tipos y funciones de tipos que permite realizar las derivaciones de manera funcional. Por ejemplo, las palabras “Octavio”, “envió”, “una” y “carta” adquieren los tipos $n, (n \setminus s)/n, n/n$ y n con lo cual se tiene la siguiente derivación, en el estilo deductivo.

$$\frac{\frac{\frac{(n \setminus s)/n}{n} \quad \frac{n/n \quad n}{n}}{n \setminus s}}{s}$$

Donde los tipos son, intuitivamente, tratados como fracciones con orientación. Otra ventaja de las gramáticas categoriales que buscan preservar el principio de composicionalidad del lenguaje, es decir, que la sintaxis y el significado de las oraciones están estrechamente relacionados y se deben estudiar simultáneamente.

Para el estudio categórico de las gramáticas categoriales introducimos el concepto de categoría bicerrada, categorías donde formalizaremos el concepto de fracciones con orientación que mencionamos anteriormente.

4.1. Categorías y gramáticas bicerradas

Definición 20. Una **signatura bicerrada** G es una colección de generadores G_1 y tipos básicos G_0 junto a un par de funciones

$$T(G_0) \xleftarrow{\text{dom}} G_1 \xrightarrow{\text{cod}} T(G_0)$$

donde $T(G_0)$ es el conjunto de tipos bicerrados dados por la siguiente definición

$$T(G_0) ::= a, b \in G_0 | a \otimes b | a/b | a \setminus b.$$

Un **morfismo de signaturas bicerradas** $\varphi: G \rightarrow \Gamma$ es un par de funciones $\varphi_0: G_0 \rightarrow \Gamma_0$ y $\varphi_1: G_1 \rightarrow \Gamma_1$ tales que hacen conmutar el diagrama de signaturas. A las categorías de signaturas bicerradas le llamamos **BcSig**.

Definición 21. Una **categoría monoidal bicerrada** $\mathcal{C}$ es una categoría monoidal equipada con dos bifuntores¹ $-\backslash -: \mathcal{C}^{op} \times \mathcal{C} \rightarrow \mathcal{C}$ y $-/-: \mathcal{C} \times \mathcal{C}^{op} \rightarrow \mathcal{C}$ tales que para cualquier objeto a , $a \otimes - \dashv a \backslash -$ y $- \otimes a \dashv -/a$. O equivalentemente, tenemos los siguientes isomorfismos naturales para cualesquiera objetos a, b, c

$$\mathcal{C}(a, c/b) \cong \mathcal{C}(a \otimes b, c) \cong \mathcal{C}(b, a \backslash c). \quad (4.1)$$

Los isomorfismos anteriores son usualmente llamados *curry* —cuando reemplazamos $\otimes$ — o *uncurry* —en el otro caso—. Con los funtores monoidales como morfismos, las categorías bicerradas forman la categoría **BcCat**.

El desarrollo de las gramáticas que veremos este capítulo estuvo intimamente relacionado con la lógica. Es por esto, que vale la pena mencionar que los morfismos en una categoría bicerrada pueden ser pensados como deducciones en un sistema formal. En particular, los axiomas de una categoría monoidal pueden ser representados como reglas de inferencia de la siguiente manera

$$\frac{a}{a \rightarrow a} \text{ id}, \quad \frac{a \rightarrow b \quad b \rightarrow c}{a \rightarrow c} \circ, \quad \frac{a \rightarrow b \quad c \rightarrow d}{a \otimes c \rightarrow c \otimes d} \otimes.$$

Por su parte, el curry y uncurry en una categoría bicerrada puede ser visto de la siguiente manera

$$a \rightarrow c/b \quad \text{sii} \quad a \otimes b \rightarrow c \quad \text{sii} \quad b \rightarrow a \backslash c.$$

Interpretamos el curry como una manera de transformar una función que toma dos argumentos a una función que toma sólo uno y arroja como resultado otra función. Dada G una signatura bicerrada, la categoría libre bicerrada **BC**(G) es la categoría monoidal que contiene todos los morfismos que pueden ser generados mediante las cuatro reglas de inferencia anteriores por los generadores de G .

Ahora bien, veamos la gramática que puede generar este tipo de categorías.

Definición 22. Una **gramática bicerrada** G es un signatura bicerrada de la siguiente forma:

$$T(B + V) \xleftarrow{\text{dom}} G \xrightarrow{\text{cod}} T(B + V)$$

donde B es un vocabulario y V es un conjunto de tipos básicos con un tipo distinguido s . El lenguaje generado por G es

$$\mathcal{L}(G) = \{u \in V^* \mid \exists f: u \rightarrow s \text{ en } \mathbf{BC}(G)\}.$$

A continuación presentaremos tres tipos de gramáticas importantes, conocidas usualmente como gramáticas categoriales: gramáticas AB, gramáticas de Lambek y gramáticas categoriales combinatorias que pueden ser definidas mediante gramáticas bicerradas. Antes de ello, debemos saber a qué nos referimos a encontrar una gramática en otras.

¹Dada $\mathcal{C}$ una categoría, definimos $\mathcal{C}^{op}$ la categoría opuesta de $\mathcal{C}$ cuyos objetos son los mismo que $\mathcal{C}$ y $f: a \rightarrow b$ es un morfismo de $\mathcal{C}^{op}$ si $f: b \rightarrow a$ es un morfismo de $\mathcal{C}$. En otras palabras, invertimos flechas.

4.2. Reducciones gramaticales

Definición 23. Sean G y G' gramáticas sobre el mismo vocabulario. Decimos que G se **reduce débilmente** a G' , $G \leq G'$, si $\mathcal{L}(G) \subset \mathcal{L}(G')$. G es débilmente equivalente a G' si $\mathcal{L}(G) = \mathcal{L}(G')$.

Notemos que aunque dos gramáticas sean débilmente equivalente, las derivaciones entre ambas pueden ser completamente distintas. Por ello, necesitamos una definición más fuerte de reducción entre gramáticas.

Definición 24. Sean G y G' gramáticas monoidales sobre el mismo vocabulario. Decimos que G se **reduce fuertemente** a G' si hay un morfismo de signaturas monoidales $\varphi: G \rightarrow G'$ tal que $\varphi_0(v) = v$ para todo $v \in V$ y $\varphi_0(s) = s'$. Las gramáticas monoidales con reducciones fuertes como morfismos forman la subcategoría $\mathbf{Grammar}_V$. Dos gramáticas son fuertemente equivalentes si son isomorfas en $\mathbf{Grammar}_V$.

Como es de esperarse, tenemos el siguiente resultado.

Proposición 12. Si G se reduce fuertemente a G' , entonces $G \leq G'$.

Demostración. Sean G y G' gramáticas monoidales tales que G se reduce fuertemente a G' . Veamos que $\mathcal{L}(G) \subset \mathcal{L}(G')$.

Sea $w \in \mathcal{L}(G)$. Entonces existe un morfismo $f: w \rightarrow s$ en $\mathbf{MC}(G)$.

Ya que G se reduce fuertemente a G' existe el morfismo de signatura monoidales $\varphi: G \rightarrow G'$ tal que $\varphi_0(v) = v$ para todo $v \in V$ y $\varphi_0(s) = s'$. Además, al ser un morfismo de signaturas monoidales el siguiente diagrama conmuta

$$\begin{array}{ccccc} G_0^* & \xleftarrow{\text{cod}} & G_1 & \xrightarrow{\text{dom}} & G_0^* \\ \downarrow \varphi_0 & & \downarrow \varphi_1 & & \downarrow \varphi_0 \\ \Gamma_0^* & \xleftarrow{\text{cod}} & \Gamma_1 & \xrightarrow{\text{dom}} & \Gamma_0^* \end{array}$$

Entonces

$$\text{dom}(\varphi_1(f)) = \varphi_0(\text{dom}(f)) = \varphi_0(w) = w$$

y

$$\text{cod}(\varphi_1(f)) = \varphi_0(\text{cod}(f)) = \varphi_0(s) = s'.$$

Por lo tanto, $\varphi_1(f): w \rightarrow s' \in \mathbf{MC}(G')$. Por lo tanto, $w \in \mathcal{L}(G')$ y así $\mathcal{L}(G) \subset \mathcal{L}(G')$ como queríamos demostrar. $\square$

Además, nos brinda una manera de comparar nuestras gramáticas descritas con anterioridad.

Proposición 13. Toda gramática simple se reduce fuertemente a una gramática de multicategorías y toda gramática de multicategorías se reduce fuertemente a una gramática monoidal.

Demostración. De las definiciones, es claro que tenemos el siguiente diagrama conmutativo dado por inclusiones

$$\begin{array}{ccccc}
 \mathbf{Sig} & \hookrightarrow & \mathbf{MultiSig} & \hookrightarrow & \mathbf{MonSig} \\
 \downarrow c & & \downarrow \text{Multi} & & \downarrow \text{MC} \\
 \mathbf{Cat} & \hookrightarrow & \mathbf{MultiCat} & \hookrightarrow & \mathbf{MonCat}.
 \end{array}$$

Por lo tanto, toda gramática simple y de multicategorías inducen gramáticas monoidales de manera funtorial. $\square$

Notemos que una reducción fuerte es solo re etiquetar las palabras y el tipo de oración, es decir, hacemos esencialmente lo mismo. Así, esta noción puede resultar demasiado restrictiva, por ello necesitamos un punto medio entre ambas.

Definición 25. Sean G y G' gramáticas monoidales sobre el mismo vocabulario V . Una **reducción funtorial de G a G'** es un funtor $F: \mathbf{MC}(G) \rightarrow \mathbf{MC}(G')$ tales que $F_0(v) = v$ para todo $v \in V$ y $F_0(s) = s'$. Una equivalencia funtorial entre G y G' es una reducción funtorial de G a G' y una de G' a G .

4.3. Gramáticas AB

En 1953, Ajdiuciewicz y Bar-Hillel introducen la primera de las gramáticas categoriales, las gramáticas AB. Sea B un conjunto de tipos básicos, definimos los tipos de una gramática AB como

$$T_{AB}(B) ::= a, b \in B | a \backslash b | a / b.$$

A los últimos dos tipos se les conoce como fracciones.

La idea de las gramáticas categoriales, como su nombre lo indica, es asignar a cada palabra una categoría determinada cuyo objetivo es indicar la manera en que las palabras se pueden relacionar entre sí de una manera significativa.

Por ejemplo, consideremos la oración “Valentina come pastel”. Notemos que las palabras “Valentina” y “pastel” son sustantivos, así asignémosles el mismo tipo n . Por otro lado, la palabra “come” es el núcleo de la oración, así que debería tener el tipo s , pero para que tenga sentido a su izquierda necesita tener un sustantivo como sujeto; el tipo $n \backslash s$ representa esto, y a su derecha también tiene otro sustantivo, así asignémosle finalmente el tipo $(n \backslash s) / n$. Entonces la derivación correspondiente a la oración “Valentina come pastel” es

$$n \quad (n \backslash s) / n \quad n \quad \rightarrow \quad n \quad n \backslash s \quad \rightarrow s.$$

Formalmente, definimos una gramática AB de la siguiente manera

Definición 26. Una **gramática AB** es una tupla $G = (V, B, \Delta, s)$ donde B es un vocabulario, B es un conjunto de tipos básicos, $\Delta \subset V \times T_{AB}(B)$ es un léxico. Las reglas de las gramáticas AB están dadas por la siguiente signatura monoidal

Proposición 14. *Las gramáticas AB se reducen funtorialmente a una gramática bicerrada.*

Demostración. Ya que ambas son categorías monoidales libres, para exhibir un funtor basta de las gramáticas AB a las bicerradas, basta asignar a los generadores $R[AB]$ en la bicerrada, pero veamos que estos se pueden obtener de la definición de categoría bicerrada. Sean a, b, c objetos, entonces

$$\frac{\frac{a \backslash b}{a \backslash b \rightarrow a \backslash b} \text{ id}}{a \otimes (a \backslash b) \rightarrow b} \quad 4.1$$

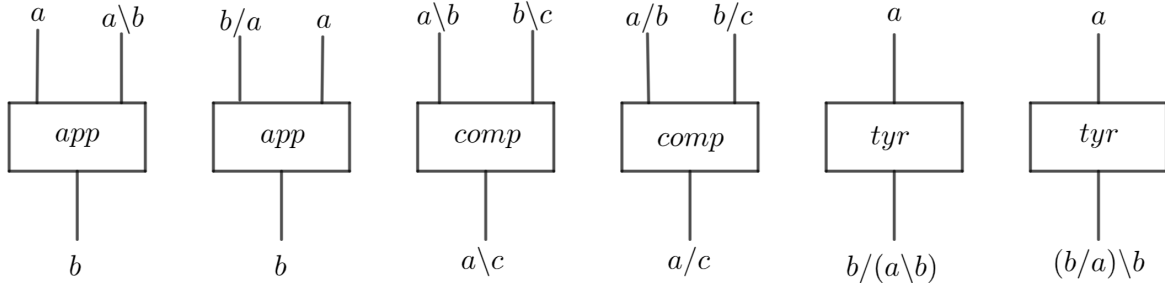
$$\frac{\frac{a/b}{a/b \rightarrow a/b} \text{ id}}{(a/b) \otimes b \rightarrow a} \quad 4.1$$

como queríamos probar. $\square$

4.4. Gramáticas de Lambek

Un par de años más tarde, Joachim Lambek introdujo dos reglas más a las gramáticas categoriales: la composición y el *type raising*. Aunque ambas reglas no le agregan poder expresivo a las gramáticas, sí permiten realizar las derivaciones de una manera más literal.

Definición 27. Un **gramática de Lambek** es una tupla $G = (V, B, \Delta, s)$ donde V es un vocabulario, B es un conjunto finito de tipos básicos y $\Delta \subset V \times T(B)$ es un léxico con la siguiente signatura R_L



El lenguaje generado por G se define como

$$\mathcal{L}(G) = \{u \in V^* \mid \exists g: u \rightarrow s \in \mathbf{MC}(\Delta + R_L)\}.$$

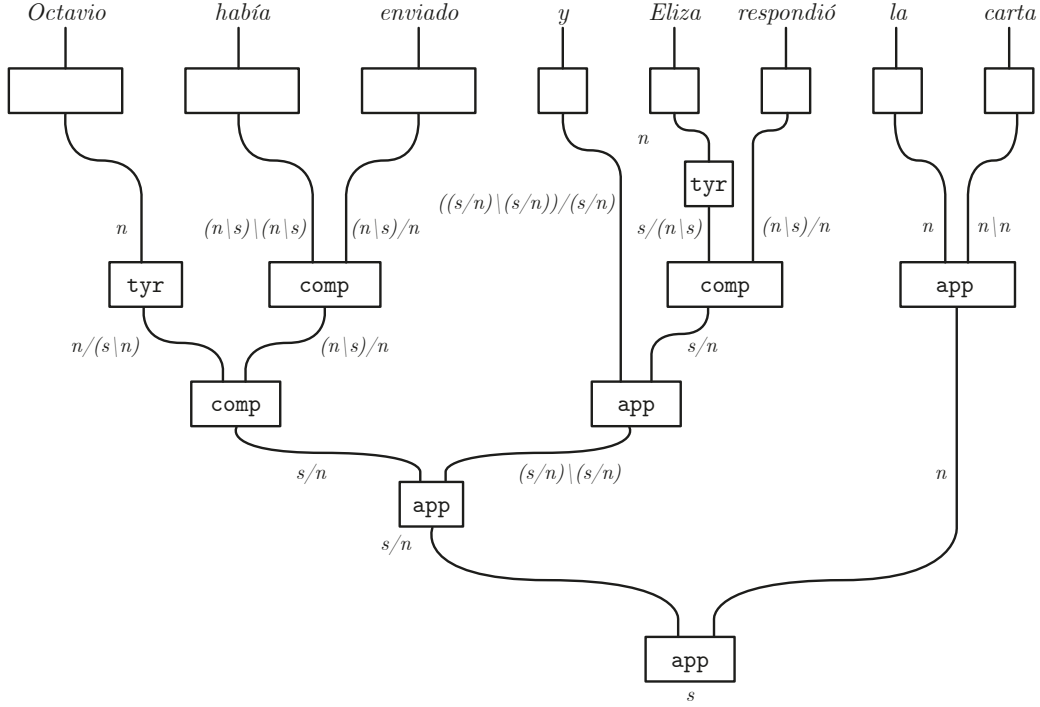
Ejemplo 11. Consideremos el siguiente léxico

$$\Delta(\text{Octavio}) = n, \quad \Delta(\text{enviado}) = (n \backslash s)/s, \quad \Delta(\text{había}) = (n \backslash s)/(n \backslash s),$$

$$\Delta(y) = ((s/n)/(s/n))/(s/n), \quad \Delta(\text{Eliza}) = n,$$

$$\Delta(\text{respondió}) = (n \backslash s)/n, \quad \Delta(\text{la}) = n/n, \quad \Delta(\text{carta}) = n.$$

Entonces tenemos la siguiente derivación de la oración “Octavio había enviado y Eliza recibió una carta”



Proposición 15. *Las gramáticas AB se reducen funtorialmente a una gramática bicerrada.*

Demostración. Como en la proposición anterior, basta ver que podemos generar R_L en una categoría bicerrada. Ya sabemos que las dos *app* son posibles, veamos el resto. Sean a, b objetos. Construyamos las derivaciones correspondientes a *comp*.

$$\begin{array}{c}
 \frac{a \backslash c}{a \backslash c \rightarrow a \backslash c} \text{ id} \\
 \frac{a \otimes a \backslash b \rightarrow b}{a \otimes a \backslash b \otimes b \backslash c \rightarrow c} \text{ 4.1} \\
 \frac{b \backslash c}{b \backslash c \rightarrow b \backslash c} \text{ id} \\
 \frac{a \otimes a \backslash b \otimes b \backslash c \rightarrow c}{a \backslash b \otimes b \backslash c \rightarrow a \backslash c} \otimes \text{ y } \circ \text{ 4.1}
 \end{array}$$

$$\begin{array}{c}
 \frac{b/c}{b/c \rightarrow b/c} \text{ id} \\
 \frac{a/b}{a/b \rightarrow a/b} \text{ id} \\
 \frac{b/c \otimes c \rightarrow b}{a/b \otimes b/c \otimes c \rightarrow a} \text{ 4.1} \\
 \frac{a/b \otimes b/c \rightarrow a/c}{a/b \otimes b/c \rightarrow a/c} \otimes \text{ y } \circ \text{ 4.1}
 \end{array}$$

Concluamos con las derivaciones de *tyr*.

$$\begin{array}{c}
 \frac{a \backslash b}{a \backslash b \rightarrow a \backslash b} \text{ id} \\
 \frac{a \otimes (a \backslash b) \rightarrow b}{a \rightarrow b/(a \backslash b)} \text{ 4.1} \\
 \text{ 4.1}
 \end{array}$$

$$\begin{array}{c}
 \frac{b/a}{b/a \rightarrow b/a} \text{ id} \\
 \frac{(b/a) \otimes a \rightarrow b}{a \rightarrow (b/a) \backslash b} \text{ 4.1} \\
 \text{ 4.1}
 \end{array}$$

con lo cual terminamos la prueba. $\square$

4.5. Gramáticas combinatoria

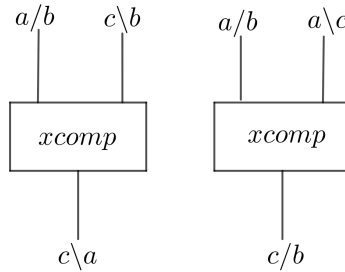
En los ochentas, Stuart Schieber demostró que hay lenguas que no pueden ser generadas por gramáticas libres de contexto, esto es porque ciertas oraciones involucran de manera natural dependencias cruzadas, fenómeno que ocurre cuando dos series de palabras se mezclan entre sí. Aunque el fenómeno no es usual en español o inglés, es muy común en el suizo-alemán. Veamos el siguiente ejemplo

“... *mer em Hans es huss halfed aastrüiche*”

que se traduce como “... nosotros *ayudamos a Hans* a *pintar la casa*”.

Ejemplos sencillos en el español son las oraciones “*Ayer una mujer llegó que conocía*” y “Octavio envió a su amada una carta”. Para analizar estos casos, se extienden las gramáticas anteriores a la gramática categorial combinatoria mediante la inclusión de dos reglas de composición cruzada.

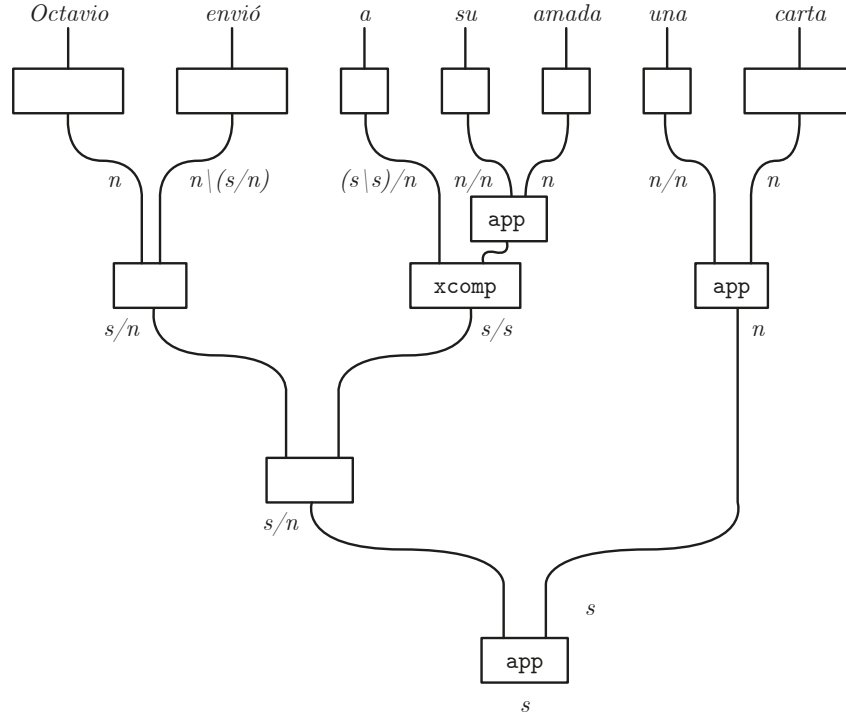
Definición 28. Una **gramática categorial combinatoria (GCC)** es una tupla $G = (V, B, \Delta, s)$ donde V es un vocabulario, B es un conjunto de tipos y $\Delta \subset V \times T(B)$ es un léxico. La signatura R_{GCC} es la signatura R_L más las siguientes dos reglas para todo a, b, c objeto



El lenguaje generado por G se define como

$$\mathcal{L}(G) = \{u \in V^* \mid \exists g: u \rightarrow s \in \mathbf{MC}(\Delta + R_{GCC})\}.$$

Ejemplo 12. Consideremos el vocabulario del ejemplo 10. Tenemos la siguiente derivación de la oración “Octavio envió a su amada una carta”.



A diferencia de las dos gramáticas categoriales anteriores, la signatura de CCG no puede ser deducida de los axiomas de categoría bicerrada, pues todos los axiomas preservan el orden de los tipos. Así, nuestro mejor intento sería agregar directamente las reglas $xcomp$ a la signatura de las gramáticas monoidales, o bien, trabajar en categorías cerradas.

Definición 29. Una **categoría monoidal simétrica** $\mathcal{C}$ es una categoría monoidal junto a una transformación natural $\sigma_{a,b}: a \otimes b \rightarrow b \otimes a$ que satisface para cualesquiera $a, b, c \in \mathcal{C}_0$ y $f: a \rightarrow b \in \mathcal{C}_1$

$$\begin{array}{c} a & b \\ & \searrow \quad \swarrow \\ & \text{X} \\ & \swarrow \quad \searrow \\ a & b \end{array} = \begin{array}{c} a & b \\ | & | \\ a & b \end{array}, \quad \begin{array}{c} a & c \\ | & | \\ \boxed{f} & \text{X} \\ & | \\ c & b \end{array} = \begin{array}{c} a & c \\ & \searrow \quad \swarrow \\ & \text{X} \\ & \swarrow \quad \searrow \\ c & b \end{array} \begin{array}{c} \boxed{f} \\ | \\ c & b \end{array}, \quad \begin{array}{c} c & a & b \\ & \searrow \quad \swarrow \quad \swarrow \\ & \text{X} & \text{X} \\ & \swarrow \quad \searrow \quad \searrow \\ a & b & c \end{array} = \begin{array}{c} c & a & b \\ | & | & | \\ a & b & c \end{array}.$$

Definición 30. Una categoría cerrada es una categoría simétrica bicerrada.

Proposición 16. Una categoría cerrada tiene las reglas $xcomp$ como morfismos.

Demostración. Sean a, b, c objetos una categoría cerrada libre. Basta mostrar las derivaciones para $xcomp$.

$$\begin{array}{c}
\frac{a/b}{a/b \rightarrow a/b} \text{ id} \quad \frac{\frac{c \backslash b}{c \backslash b \rightarrow c \backslash b} \text{ id}}{c \otimes c \backslash b \rightarrow b} \text{ 4.1} \\
\hline
\frac{a/b \otimes c \otimes c \backslash b \rightarrow a}{c \otimes a/b \otimes c \backslash b \rightarrow a} \sigma_{a/b, c} \text{ 4.1} \\
\hline
a/b \otimes c \backslash b \rightarrow c \backslash a
\end{array}$$

$$\begin{array}{c}
\frac{a/b}{a/b \rightarrow a/b} \text{ id} \quad \frac{a \backslash c}{a \backslash c \rightarrow a \backslash c} \text{ id} \\
\hline
\frac{a/b \otimes b \rightarrow a}{a/b \otimes b \otimes a \backslash c \rightarrow c} \sigma_{b, a \backslash c} \text{ 4.1} \\
\hline
a/b \otimes a \backslash c \rightarrow c/b
\end{array}$$

□

Una última observación es que podría resultar una mala idea trabajar gramáticas sobre categorías cerradas sin restricciones, pues entonces corremos el riesgo de perder el orden de las palabras en una oración, por lo cual restringimos los morfismos $f: u \rightarrow s$ a la aplicación exclusiva de $xcomp$.

5 Pregrupos

En este capítulo definiremos un tercer acercamiento a las gramáticas de los lenguajes naturales desarrollado por Joachim Lambek en 1999 [6].

Un pregrupo es una estructura algebraica $(P, \leq, 1, -^r, -^l)$ que satisface las siguientes propiedades

- $(P, \geq)$ es un orden parcial;
- $(P, \cdot, 1)$ es un monoide;
- Para cualesquiera $x, y, z \in P$, si $x \leq y$, entonces $x \cdot z \leq y \cdot z$; y
- para cualquier objeto $t \in P$,
 - $t \cdot t^r \leq 1$ y $t^l \cdot t \leq 1$ (contracciones), y
 - $1 \leq t^r \cdot t$ y $1 \leq t \cdot t^l$ (expansiones).

Dado un conjunto de tipos gramaticales, por ejemplo, $B = \{n, s\}$ podemos considerar el pregrupo libre generado por B y dada la asignación de tipos “Octavio”, “envió”, “una” y “carta” dada por $n, n^r \cdot s \cdot n^l, n \cdot n^l$ y n respectivamente, tenemos que

$$\begin{aligned}
 n \cdot (n^r \cdot s \cdot n^l) \cdot (n \cdot n^l) \cdot n &= (n \cdot n^r) \cdot s \cdot (n^l \cdot n) \cdot (n^l \cdot n) \\
 &= 1 \cdot s \cdot 1 \cdot 1 && \text{aplicando contracciones} \\
 &= s.
 \end{aligned}$$

En estas gramáticas determinar si una oración es o no válida se puede decidir mediante operaciones algebraicas. Su uso es conveniente¹ ya que son tan expresivas como las gramáticas Lambek, no pierden compromiso con el principio de composicionalidad y determinar si una palabra pertenece a la gramática es no sólo decidible, sino que es requiere tiempo polinomial.

Para la descripción de los pregrupos en el marco de la teoría de categorías, introduciremos el concepto de elementos adjuntos derechos e izquierdos en una categoría, que se traducen como *cups* y *caps* en el lenguaje gráfico de categorías, para definir el concepto de una categoría rígida y las gramáticas que producen.

¹Lambek además mencionaría que son prácticas, escribir operaciones algebraicas ocupan menos espacio y tiempo, que árboles de derivación o diagramas de autómatas

5.1. Categorías rígidas

Definición 31. Una signatura rígida G es una colección de generadores G_1 y tipos básicos G_0 junto a un par de funciones:

$$P(G_0) \xleftarrow{\text{dom}} G_1 \xrightarrow{\text{cod}} P(G_0)$$

donde $P(G_0)$ es el conjunto de tipos de pregrupos dados por la siguiente definición

$$T(G_0) ::= a, b \in G_0 | a^r | a^l.$$

Un morfismo de signaturas rígidas $\varphi: G \rightarrow \Gamma$ es un par de funciones $\varphi_0: G_0 \rightarrow \Gamma_0$ y $\varphi_1: G_1 \rightarrow \Gamma_1$ tales que hacen conmutar el diagrama de signaturas. A la categoría de signaturas rígidas le llamamos **RgSig**.

Definición 32. Sea $(\mathcal{C}, \otimes, 1)$ una categoría monoidal. Un objeto a^l es un adjunto izquierdo de un objeto a y a^r es adjunto derecho de a si existen morfismos

$$\eta_r: 1 \rightarrow a^r \otimes a, \quad \varepsilon_r: a \otimes a^r \rightarrow 1,$$

$$\eta_l: 1 \rightarrow a \otimes a^l, \quad \varepsilon_l: a^l \otimes a \rightarrow 1.$$

llamados las unidades y evaluaciones respectivamente, tales que hacen conmutar los siguientes triángulos

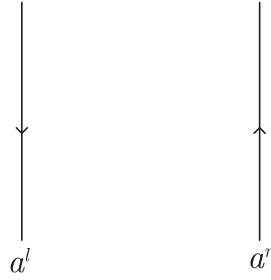
$$\begin{array}{ccc} a & \xrightarrow{\eta_l \otimes Id} & a \otimes a^l \otimes a \\ & \searrow Id_a & \downarrow Id \otimes \varepsilon_l \\ & & a, \end{array} \quad \begin{array}{ccc} a & \xrightarrow{Id_a \otimes \eta_r} & a \otimes a^r \otimes a \\ & \searrow Id_a & \downarrow \varepsilon_r \otimes Id \\ & & a. \end{array} \quad (5.1)$$

Con base a la noción de adjuntos, podemos definir la clase de categorías que protagonizarán el presente capítulo.

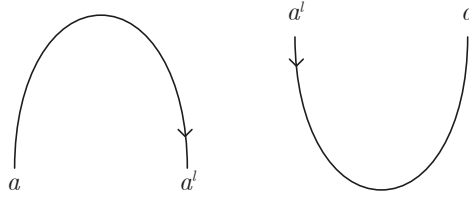
Definición 33. Una categoría monoidal $(\mathcal{C}, \otimes, 1)$ es una categoría rígida si cada objeto a tiene un adjunto izquierdo a^l y un adjunto derecho a^r .

Antes de continuar, desarrollemos un lenguaje gráfico para las categorías rígidas. En el capítulo 4, establecimos como representar cada objeto y morfismo de una categoría monoidal por medio de diagramas, entonces basta ver cómo lo haremos con adjuntos, unidades y evaluaciones.

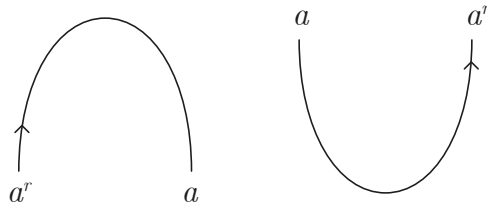
El adjunto izquierdo lo representaremos mediante un cable con una flecha hacia abajo, mientras que el adjunto derecho también será un cable pero con una flecha hacia arriba, tal como mostramos a continuación.



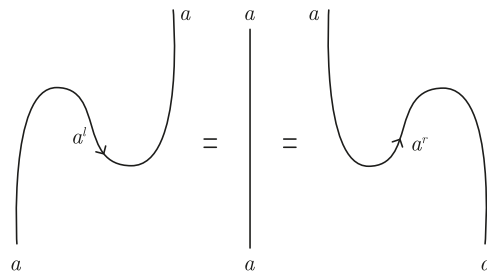
Con esta convención, resulta natural dibujar los morfismos $\eta_l: 1 \rightarrow a \otimes a^l$ y $\varepsilon_l: a^l \otimes a \rightarrow 1$ de la siguiente manera.



Dada esta representación gráfica es común referirse a la unidad y evaluación con los nombres *caps* y *cups*. Por su parte, la unidad y evaluación derecha se representan de esta forma.



Con la cual las ecuaciones (5.1) adquieren la siguiente forma en el cálculo gráfico.



Debido a esta representación, estas ecuaciones son conocidas como las ecuaciones de la serpiente.

Como en los capítulos anteriores, describiremos el poder expresivo de las gramáticas que pueden definirse sobre una categoría rígida. Un primer acercamiento a la respuesta es que tienen, a lo más, tanto capacidad como las gramáticas categoriales de acuerdo a la siguiente proposición.

Proposición 17. *Sea $(\mathcal{C}, \otimes, 1)$ una categoría rígida, entonces $\mathcal{C}$ es bicerrada.*

Demostración. Sea $(\mathcal{C}, \otimes, 1)$ una categoría rígida.

Para a y b objetos de $\mathcal{C}$ definimos $a \backslash b = a^r \otimes b$ y $b / a = b \otimes a^l$.

Queremos mostrar que existen isomorfismos naturales $\mathcal{C}(a, c \otimes b^l) \cong \mathcal{C}(a \otimes b, c) \cong \mathcal{C}(b, a^r \otimes c)$.

Probaremos solamente que $\mathcal{C}(a, c \otimes b^l) \cong \mathcal{C}(a \otimes b, c)$ pues son análogos.

Sean a, b y c objetos en $\mathcal{C}$, y sea $f: a \otimes b \rightarrow c$ morfismo. Consideremos la siguiente composición

$$a \xrightarrow{\cong} a \otimes 1 \xrightarrow{\text{Id}_a \otimes \eta_b} a \otimes b \otimes b^l \xrightarrow{f \otimes \text{Id}_{b^l}} c \otimes b^l$$

y denotémosla $\Phi(f): a \rightarrow c \otimes b^l$, es decir, tenemos un morfismo $\Phi: \mathcal{C}(a \otimes b, c) \rightarrow \mathcal{C}(a, c \otimes b^l)$.

Busquemos $\Psi: \mathcal{C}(a, c \otimes b^l) \rightarrow \mathcal{C}(a \otimes b, c)$ la inversa de Φ . Sea $g: a \rightarrow c \otimes b^l$ y definimos $\Psi(g)$ como la composición

$$a \otimes b \xrightarrow{g \otimes \text{Id}_b} c \otimes b^l \otimes b \xrightarrow{\text{Id}_c \otimes \epsilon_b} c \otimes 1 \xrightarrow{\cong} c$$

Notemos que por la ecuación de la serpiente es claro que son inversas. Falta ver que es natural tanto en a como en c .

Para la naturalidad en a , sea $h: a' \rightarrow a$ morfismo, queremos que el siguiente diagrama conmute

$$\begin{array}{ccc} \mathcal{C}(a' \otimes b, c) & \xrightarrow{\Phi} & \mathcal{C}(a', c \otimes b^l) \\ \downarrow h \otimes \text{Id}_b & & \downarrow h \\ \mathcal{C}(a \otimes b, c) & \xrightarrow{\Phi} & \mathcal{C}(a, c \otimes b^l). \end{array}$$

Sea $f: a' \otimes b$, tenemos que ver que las siguiente composiciones son iguales

$$\begin{array}{c} a' \xrightarrow{h} a \xrightarrow{\cong} a \otimes 1 \xrightarrow{\text{Id}_a \otimes \eta_b} a \otimes b \otimes b^l \xrightarrow{f \otimes \text{Id}_{b^l}} c \otimes b^l \\ \underbrace{\hspace{15em}}_{\Phi(f)} \\ a' \xrightarrow{\cong} a' \otimes 1 \xrightarrow{\text{Id}_{a'} \otimes \eta_b} a' \otimes b \otimes b^l \xrightarrow{h \otimes \text{Id}_{b \otimes b^l}} a \otimes b \otimes b^l \xrightarrow{f \otimes \text{Id}_{b^l}} c \otimes b^l. \\ \underbrace{\hspace{15em}}_{\Phi(f \circ (h \otimes \text{Id}_{b^l}))} \end{array}$$

Por la ecuación 3.2 tenemos la siguiente igualdad

$$(\text{Id}_a \circ \eta_b) \circ (h \otimes \text{Id}_1) = (h \otimes \text{Id}_{b \otimes b^l}) \circ (\text{Id}_{a'} \otimes \eta_b)$$

y aplicando $f \otimes b^l$ tenemos la igualdad deseada y, por lo tanto, el diagrama conmuta. Para la naturalidad en b sea $k: c \rightarrow c'$, veamos que el siguiente diagrama conmuta

$$\begin{array}{ccc} \mathcal{C}(a \otimes b, c) & \xrightarrow{\Phi} & \mathcal{C}(a, c \otimes b^l) \\ \downarrow k & & \downarrow k \otimes \text{Id}_{b^l} \\ \mathcal{C}(a \otimes b, c') & \xrightarrow{\Phi} & \mathcal{C}(a, c' \otimes b^l). \end{array}$$

Sea $g: a \otimes b \rightarrow c$, queremos ver que la composición $(k \otimes \text{Id}_{b^l}) \circ \Phi(g) = \Phi(k \circ g)$, lo cual es cierto pues

$$\begin{aligned} (k \otimes \text{Id}_{b^l}) \circ \Phi(g) &= (k \otimes \text{Id}_{b^l}) \circ ((g \text{Id}_{b^l}) \circ (\text{Id}_a \otimes \eta_b)) \\ &= ((k \otimes \text{Id}_{b^l}) \circ (g \text{Id}_{b^l})) \circ (\text{Id}_a \otimes \eta_b) \\ &= ((k \circ g) \otimes \text{Id}_{b^l}) \circ (\text{Id}_a \otimes \eta_b) \\ &= \Phi(k \circ g) \end{aligned}$$

y, por lo tanto, el diagrama conmuta.

Por lo tanto, tenemos el isomorfismo natural $\mathcal{C}(a, c \otimes b^l) \cong \mathcal{C}(a \otimes b, c)$ y, de manera análoga, $\mathcal{C}(b, a^r \otimes c) \cong \mathcal{C}(a \otimes b, c)$.

Por lo tanto, $\mathcal{C}$ es una categoría bicerrada. $\square$

Dada G una signatura rígida, definimos la categoría rígida libre como

$$\mathbf{RC}(G) = \mathbf{MC}(G \cup \{\text{cups}, \text{caps}\}) / \sim_{\text{serpiente}}$$

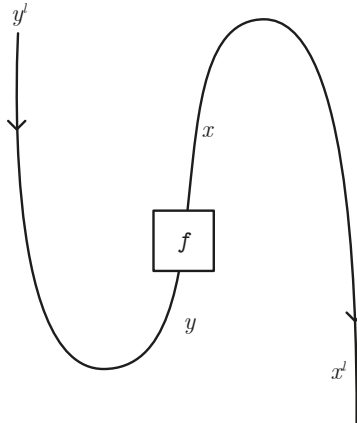
donde $\sim_{\text{serpiente}}$ es la relación inducida por las ecuaciones de la serpiente 5.1. En otras palabras, $\mathbf{RC}(G)$ es la categoría monoidal libre donde añadimos como generadores las unidades y evaluaciones, e igualamos los morfismos de acuerdo a las ecuaciones de la serpiente para que se satisfaga la definición de categoría rígida.

Antes de mostrar las gramáticas generadas en las categorías rígidas, hablaremos de una clase de morfismos que tendrán un rol importante en la siguiente sección.

Sea $f: x \rightarrow y$ un morfismo en $\mathbf{RC}(G)$, consideremos la siguiente composición

$$y^r \xrightarrow{\cong} 1 \otimes y^r \xrightarrow{\eta_x \otimes \text{Id}_{y^r}} x^r \otimes x \otimes y^r \xrightarrow{\text{Id}_{x^r} \otimes f \otimes \text{Id}_{y^r}} x^r \otimes y \otimes y^r \xrightarrow{\text{Id}_{x^r} \otimes \varepsilon_y} x^r$$

Definimos $f^r: y^r \rightarrow x^r$ como $f^r = (\text{Id}_{x^r} \otimes \varepsilon_y) \circ (\text{Id}_{x^r} \otimes f \otimes \text{Id}_{y^r}) \circ (\eta_x \otimes \text{Id}_{y^r})$ y gráficamente lo representamos como



Y, además, probemos que f^r hace conmutar los siguientes diagramas

$$\begin{array}{ccc}
 x \otimes y^r & \xrightarrow{f \otimes \text{Id}_{y^r}} & y \otimes y^r \\
 \text{Id}_x \otimes f^r \downarrow & \searrow \varepsilon_f & \downarrow \varepsilon_y \\
 x \otimes x^r & \xrightarrow{\varepsilon_x} & 1
 \end{array}
 \qquad
 \begin{array}{ccc}
 1 & \xrightarrow{\eta_x} & x^r \otimes x \\
 \eta_y \downarrow & \searrow \eta_f & \downarrow \text{Id}_{x^r} \otimes f \\
 y^r \otimes y & \xrightarrow{f^r \otimes \text{Id}_y} & x^r \otimes y
 \end{array}$$

Primero veamos que el primer cuadrado conmuta

$$\begin{aligned}
 \varepsilon_x \circ (\text{Id}_x \otimes f^r) &= \varepsilon_x \circ (\text{Id}_x \otimes ((\text{Id}_{x^r} \otimes \varepsilon_y) \circ (\text{Id}_{x^r} \otimes f \otimes \text{Id}_{y^r}) \circ (\eta_x \otimes \text{Id}_{y^r}))) \\
 &= \varepsilon_x \circ (\text{Id}_x \otimes ((\text{Id}_{x^r} \otimes (\varepsilon_y \circ (f \otimes \text{Id}_{y^r}))) \circ (\eta_x \otimes \text{Id}_{y^r}))) \\
 &= \varepsilon_x \circ (\text{Id}_x \otimes \text{Id}_{x^r} \otimes (\varepsilon_y \circ (f \otimes \text{Id}_{y^r}))) \circ (\text{Id}_x \otimes \eta_x \otimes \text{Id}_{y^r}) \\
 &= (\varepsilon_y \circ (f \otimes \text{Id}_{y^r})) \circ (\varepsilon_x \otimes \text{Id}_x \otimes \text{Id}_{y^r}) \circ (\text{Id}_x \otimes \eta_x \otimes \text{Id}_{y^r}) \\
 &= (\varepsilon_y \circ (f \otimes \text{Id}_{y^r})) \circ ((\varepsilon_x \otimes \text{Id}_x) \circ (\text{Id}_x \otimes \eta_x)) \otimes \text{Id}_{y^r} \\
 &= \varepsilon_y \circ (f \otimes \text{Id}_{y^r}) \circ (\text{Id}_x \otimes \text{Id}_{y^r}) \\
 &= \varepsilon_y \circ (f \otimes \text{Id}_{y^r}).
 \end{aligned}$$

Por lo tanto, el diagrama conmuta.

Probemos ahora que el segundo cuadrado conmuta

$$\begin{aligned}
 (f^r \otimes \text{Id}_y) \circ \eta_y &= (((\text{Id}_{x^r} \otimes \varepsilon_y) \circ (\text{Id}_{x^r} \otimes f \otimes \text{Id}_{y^r}) \circ (\eta_x \otimes \text{Id}_{y^r})) \otimes \text{Id}_y) \circ \eta_y \\
 &= (((\text{Id}_{x^r} \otimes \varepsilon_y) \circ ((\text{Id}_{x^r} \otimes f) \circ \eta_x) \otimes \text{Id}_{y^r}) \otimes \text{Id}_y) \circ \eta_y \\
 &= (\text{Id}_{x^r} \otimes \varepsilon_y \otimes \text{Id}_y) \circ (((\text{Id}_{x^r} \otimes f) \circ \eta_x) \otimes \text{Id}_y \otimes \text{Id}_{y^r}) \circ \eta_y \\
 &= (\text{Id}_{x^r} \otimes \varepsilon_y \otimes \text{Id}_y) \circ (\text{Id}_{x^r} \otimes \text{Id}_y \otimes \eta_y) \circ ((\text{Id}_{x^r} \otimes f) \circ \eta_x) \\
 &= (\text{Id}_{x^r} \otimes ((\varepsilon_y \otimes \text{Id}_y) \circ (\text{Id}_y \otimes \eta_y))) \circ ((\text{Id}_{x^r} \otimes f) \circ \eta_x) \\
 &= (\text{Id}_{x^r} \otimes \text{Id}_y) \circ ((\text{Id}_{x^r} \otimes f) \circ \eta_x) \\
 &= (\text{Id}_{x^r} \otimes f) \circ \eta_x.
 \end{aligned}$$

Por lo tanto el diagrama conmuta. Invitamos al lector a revisar los diagramas correspondientes a estos diagramas en el [Apéndice](#). Con base a lo anterior, introducimos la siguiente definición.

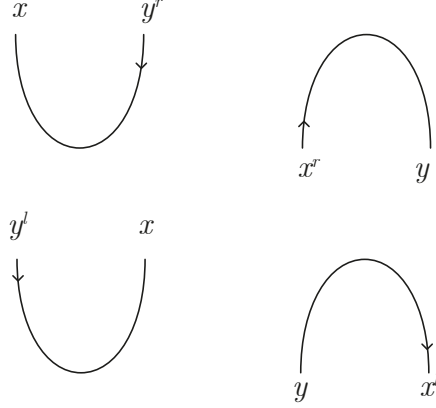
Definición 34. Sea $f: x \rightarrow y$ un morfismo en $\mathbf{MC}(G)$, definimos la **contracción y expansión generalizada derecha de f** como $\varepsilon_f: x \otimes y^r \rightarrow 1$ y $\eta_f: 1 \rightarrow x^r \otimes y$ dadas por

$$\varepsilon_f = \varepsilon_y \circ (f \otimes \text{Id}_{y^r}) = \varepsilon_x \circ (\text{Id}_x \otimes f^r) \qquad \text{y} \qquad \eta_f = (\text{Id}_{x^r} \otimes f) \circ \eta_x = (f^r \otimes \text{Id}_y) \circ \eta_y.$$

Análogamente definimos la **contracción y expansión generalizada izquierda de f** como $\varepsilon_f: y^l \otimes x \rightarrow 1$ y $\eta_f: 1 \rightarrow y \otimes x^l$ dadas por

$$\varepsilon_f = \varepsilon_y \circ (\text{Id}_{y^l} \otimes f) \qquad \text{y} \qquad \eta_f = (f \otimes \text{Id}_{x^l}) \circ \eta_x.$$

Los representamos gráficamente de la siguiente manera.



Finalmente llamaremos **paso inducido** a un morfismo de la forma $\text{Id}_u \otimes s \otimes \text{Id}_v : u \otimes x \otimes v \rightarrow u \otimes y \otimes v$ donde $s : x \rightarrow y$ es un morfismo generador.

5.2. Pregrupos

Definamos la noción de gramáticas sobre una categoría rígida, una subclase de las gramáticas bicerradas.

Definición 35. Una **gramática rígida** G es una signatura rígida de la siguiente forma

$$P(B + V) \xleftarrow{\text{dom}} P(B + V) \xrightarrow{\text{cod}} P(B + V)$$

donde V es un vocabulario y B un conjunto de tipos básicos. El lenguaje generado por G se define como $\mathcal{L}(G) = \{u \in V^* \mid \exists g : u \rightarrow s \in \mathbf{RC}(G)\}$ donde $\mathbf{RC}(G)$ es la categoría rígida generada por G .

Con base a la definición anterior, podemos establecer una gramática de pregrupo, una gramática rígida donde cada palabra en el vocabulario tiene asignados tipos.

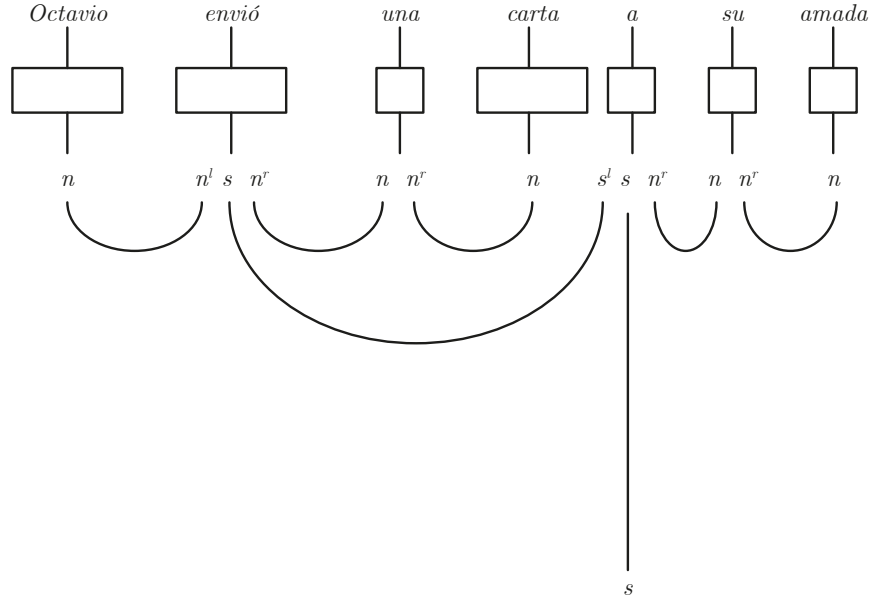
Definición 36. Una **gramática de pregrupo** es una tupla $G = (V, B, \Delta, I, s)$ donde V es un vocabulario, B es un conjunto finito de tipos básicos, $\Delta \subset V \times P(B)$ es un léxico que asigna a cada palabra un posible tipo de pregrupo, y $I \subset B \times B$ un conjunto finito de reglas de producción. El lenguaje generado por G está dado por $\mathcal{L}(G) = \{u \in V^* \mid \exists g : u \rightarrow s \in \mathbf{RC}(G)\}$ donde $\mathbf{RC}(G) = \mathbf{RC}(\Delta + I)$

Ejemplo 13. Sea $B = \{s, n\}$ el conjunto de tipos básicos correspondiente oración y sustantivo con el siguiente léxico

$$\Delta(\text{Octavio}) = n, \quad \Delta(\text{envió}) = n^l \otimes n \otimes n, \quad \Delta(\text{una}) = n \otimes n^r, \quad \Delta(\text{carta}) = n,$$

$$\Delta(a) = s^l \otimes s \otimes n^r, \quad \Delta(su) = n^r \otimes n, \quad \Delta(\text{amada}) = n.$$

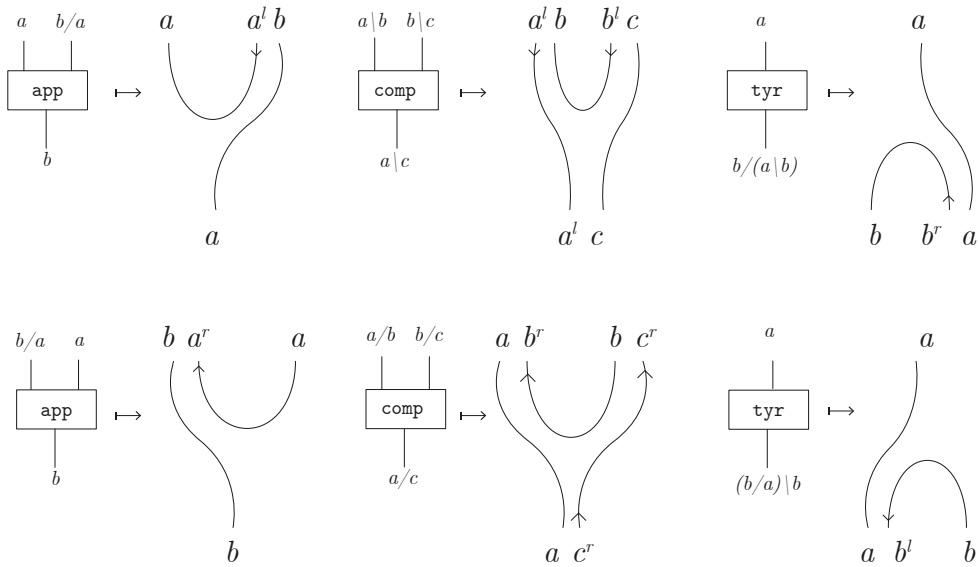
Entonces la oración “Octavio envió una carta a su amada” es gramaticalmente correcta dado el siguiente morfismo.



Anteriormente mencionamos que las gramáticas de pregrupos son, a lo más, tan expresivas como las gramáticas bicerradas. Pero el ejemplo anterior basado en 10 sugiere que poseen exactamente el mismo poder expresivo y, más aún, existe una reducción funtorial canónica como lo veremos a continuación.

Proposición 18. *Sea G una gramática de Lambek, entonces existe una gramática de pregrupo G' con una reducción funtorial $\mathbf{MC}(G) \rightarrow \mathbf{RC}(G)$*

Demostración. Sea G una gramática de Lambek. Consideremos G' la gramática de pregrupos con el vocabulario, tipos básicos y léxico que G con las reglas de producción dadas por la siguiente traducción entre gramáticas



□

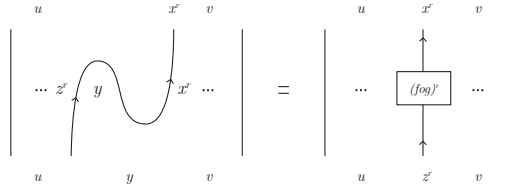
El estudio de gramáticas de pregrupos es entonces el estudio de las gramáticas descritas en el capítulo anterior, sin embargo, la verificación computacional de oraciones aceptadas por el lenguaje es computacionalmente más eficiente de acuerdo al *Switching lemma* probado por Lambek y Teller en [6]. Antes de enunciarlo, veamos los siguientes movimientos válidos en una categoría rígida.

1. Los generadores de $f: x \rightarrow y$ y $g: a \rightarrow b$ no comparten elementos en el codominio y dominio respectivamente, entonces podemos intercambiar el orden de composición de f y g según convenga, de acuerdo a la ley de intercambio.
2. Reemplazar dos pasos inducidos por un solo paso inducido.

Supongamos $d_1 = \text{Id}_u \otimes g \otimes \text{Id}_v$ y $d_2 = \text{Id}_u \otimes h \otimes \text{Id}_v$ con $f: x \rightarrow y$ y $h: y \rightarrow z$, entonces $d_2 \circ d_1 = (\text{Id}_u \otimes h \otimes \text{Id}_v) \circ (\text{Id}_u \otimes g \otimes \text{Id}_v) = \text{Id}_u \otimes (h \circ g) \otimes \text{Id}_v$.

3. Reemplazar una expansión generalizada seguida por una contracción generalizada por un paso inducido. Tenemos dos casos:

- a) La contracción generalizada está a la derecha. Gráficamente tenemos el siguiente caso



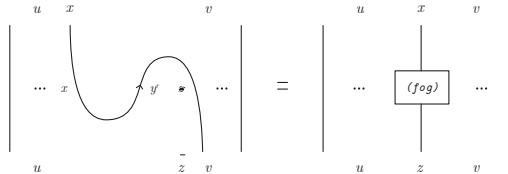
Sea $f: y \rightarrow x$ y $g: z \rightarrow y$, entonces tenemos que

$$\begin{aligned}
 (\text{Id}_{z^r} \otimes \varepsilon_f) \circ (\eta_g \otimes \text{Id}_{x^r}) &= (\text{Id}_{z^r} \otimes (\varepsilon_x \circ (f \otimes \text{Id}_{x^r}))) \circ (((\text{Id}_{z^r} \otimes g) \circ \eta_z) \otimes \text{Id}_{x^r}) \\
 &= (\text{Id}_{z^r} \otimes \varepsilon_x) \circ (\text{Id}_{z^r} \otimes f \otimes \text{Id}_{x^r}) \circ (\text{Id}_{z^r} \otimes g \otimes \text{Id}_{x^r}) \circ (\eta_{z^r} \otimes \text{Id}_{x^r}) \\
 &= (\text{Id}_{z^r} \otimes \varepsilon_x) \circ (\text{Id}_{z^r} \otimes (f \circ g) \otimes \text{Id}_{x^r}) \circ (\eta_{z^r} \otimes \text{Id}_{x^r}) \\
 &= (f \circ g)^r.
 \end{aligned}$$

Véase [Apéndice](#).

Por lo tanto, $\text{Id}_u \otimes (\text{Id}_{z^r} \otimes \varepsilon_f) \circ (\eta_g \otimes \text{Id}_{x^r}) \otimes \text{Id}_v = \text{Id}_u \otimes (f \circ g)^r \otimes \text{Id}_v$.

- b) La contracción generalizada está a la izquierda. Gráficamente tenemos el siguiente caso



Sea $f: y \rightarrow x$ y $g: x \rightarrow y$, entonces

$$\begin{aligned}
(\varepsilon_g \circ \text{Id}_z) \circ (\text{Id}_x \circ \eta_f) &= ((\varepsilon_y \circ (g \otimes \text{Id}_{y^r})) \otimes \text{Id}_z) \circ (\text{Id}_x \otimes ((\text{Id}_{y^r} \otimes f) \circ \eta_y)) \\
&= (\varepsilon_y \otimes \text{Id}_z) \circ (g \otimes \text{Id}_{y^r} \otimes \text{Id}_z) \circ (\text{Id}_x \otimes \text{Id}_{y^r} \otimes f) \circ (\text{Id}_x \otimes \eta_y) \\
&= (\varepsilon_y \otimes \text{Id}_z) \circ (\text{Id}_y \otimes \text{Id}_{y^r} \otimes f) \circ (g \otimes \text{Id}_{y^r} \otimes \text{Id}_y) \circ (\text{Id}_x \otimes \eta_y) \\
&= f \circ (\varepsilon_y \otimes \text{Id}_y) \circ (\text{Id}_y \otimes \eta_y) \circ g \\
&= f \circ \text{Id}_y \circ g \\
&= f \circ g.
\end{aligned}$$

Véase [Apéndice](#).

Por lo tanto, $\text{Id}_u \otimes ((\varepsilon_g \circ \text{Id}_z) \circ (\text{Id}_x \circ \eta_f)) \otimes \text{Id}_v = \text{Id}_u \otimes f \circ g \otimes \text{Id}_v$.

Notemos que estos dos casos corresponden a generalizaciones de las ecuaciones de la serpiente.

4. Reemplazar un paso inducido seguido por una contracción generalizada por una contracción generalizada. Tenemos dos subcasos:

a) El paso inducido está a la izquierda.

$$\left| \begin{array}{c} u \quad z \quad x' \quad v \\ \vdots \quad y \quad \vdots \quad \vdots \\ u \quad \quad v \end{array} \right| = \left| \begin{array}{c} u \quad z \quad x' \quad v \\ \vdots \quad \quad \vdots \quad \vdots \\ u \quad \quad v \end{array} \right|$$

Sea $f: y \rightarrow x$ y $g: z \rightarrow y$. Entonces

$$\begin{aligned}
\varepsilon_{f \circ g} &= \varepsilon_x \circ ((f \circ g) \otimes \text{Id}_{x^r}) \\
&= \varepsilon_x \circ ((f \otimes \text{Id}_{x^r}) \otimes (g \otimes \text{Id}_{x^r})) \\
&= (\varepsilon_x \circ (f \otimes \text{Id}_{x^r})) \circ (g \otimes \text{Id}_{x^r}) \\
&= \varepsilon_f \circ (g \otimes \text{Id}_{x^r}).
\end{aligned}$$

Véase [Apéndice](#).

Por lo tanto, $\text{Id}_u \otimes \varepsilon_{f \circ g} \otimes \text{Id}_v = \text{Id}_u \otimes (\varepsilon_f \circ (g \otimes \text{Id}_x)) \otimes \text{Id}_v$.

b) El paso inducido está a la derecha.

$$\left| \begin{array}{c} u \quad y \quad z' \quad v \\ \vdots \quad y \quad \vdots \quad \vdots \\ u \quad \quad v \end{array} \right| = \left| \begin{array}{c} u \quad y \quad z' \quad v \\ \vdots \quad \quad \vdots \quad \vdots \\ u \quad \quad v \end{array} \right|$$

Sea $f: z \rightarrow x$ y $g: y \rightarrow z$.

$$\begin{aligned}
\varepsilon_{f \circ g} &= \varepsilon_x \circ ((f \circ g) \otimes \text{Id}_{x^r}) \\
&= \varepsilon_x \circ ((f \circ \text{Id}_y \circ g) \otimes \text{Id}_{x^r}) \\
&= \varepsilon_x \circ (f \otimes \text{Id}_x) \circ (\text{Id}_y \otimes \text{Id}_{x^r}) \circ (g \otimes \text{Id}_{x^r}) \\
&= \varepsilon_x \circ (f \otimes \text{Id}_{x^r}) \circ [((\varepsilon_z \otimes \text{Id}_z) \circ (\text{Id}_z \otimes \eta_z)) \otimes \text{Id}_{x^r}] \circ (g \otimes \text{Id}_{x^r}) \\
&= \varepsilon_x \circ (f \otimes \text{Id}_{x^r}) \circ (\varepsilon_z \otimes \text{Id}_z \otimes \text{Id}_{x^r}) \circ (\text{Id}_z \otimes \eta_z \otimes \text{Id}_{x^r}) \circ (g \otimes \text{Id}_{x^r}) \\
&= \varepsilon_x \circ (\text{Id}_x \otimes \text{Id}_{x^r}) \circ (\varepsilon_z \otimes f \otimes \text{Id}_{x^r}) \circ (g \otimes \eta_z \otimes \text{Id}_{x^r}) \circ (\text{Id}_y \otimes \text{Id}_{x^r}) \\
&= \varepsilon_x \circ (\varepsilon_z \otimes f \otimes \text{Id}_{x^r}) \circ (g \otimes \eta_z \otimes \text{Id}_{x^r}) \\
&= \varepsilon_x \circ (\varepsilon_z \otimes \text{Id}_x \otimes \text{Id}_{z^r}) \circ (\text{Id}_z \otimes \text{Id}_{z^r} \otimes f \otimes \text{Id}_{x^r}) \circ (g \otimes \eta_z \otimes \text{Id}_{x^r}) \\
&= \varepsilon_z \circ (\text{Id}_z \otimes \text{Id}_{z^r} \otimes \varepsilon_x) \circ (\text{Id}_z \otimes \text{Id}_{z^r} \otimes f \otimes \text{Id}_{x^r}) \circ (g \otimes \eta_z \otimes \text{Id}_{x^r}) \\
&= \varepsilon_z \circ (\text{Id}_z \otimes \text{Id}_{z^r} \otimes \varepsilon_x) \circ (g \otimes \text{Id}_{z^r} \otimes f \otimes \text{Id}_{x^r}) \circ (\text{Id}_y \otimes \eta_z \otimes \text{Id}_{x^r}) \\
&= \varepsilon_z \circ (g \otimes \text{Id}_{z^r} \otimes \varepsilon_x) \circ (\text{Id}_y \otimes \text{Id}_{z^r} \otimes f \otimes \text{Id}_{x^r}) \circ (\text{Id}_y \otimes \eta_z \otimes \text{Id}_{x^r}) \\
&= \varepsilon_z \circ (g \otimes \text{Id}_{z^r}) \circ (\text{Id}_y \otimes \text{Id}_{z^r} \otimes \varepsilon_x) \circ (\text{Id}_y \otimes \text{Id}_{z^r} \otimes f \otimes \text{Id}_{x^r}) \circ (\text{Id}_y \otimes \eta_z \otimes \text{Id}_{x^r}) \\
&= \varepsilon_g \circ [\text{Id}_y \otimes ((\text{Id}_{z^r} \otimes \varepsilon_x) \circ (\text{Id}_{z^r} \otimes f \otimes \text{Id}_{x^r}) \circ (\eta_z \otimes \text{Id}_{x^r}))] \\
&= \varepsilon_g \circ (\text{Id}_y \otimes f^r).
\end{aligned}$$

Véase [Apéndice](#).

Y, por lo tanto, $\text{Id}_u \otimes \varepsilon_{f \circ g} \otimes \text{Id}_v = \varepsilon_g \circ (\text{Id}_y \otimes f^r)$.

5. Reemplazar una expansión generalizada seguida por un paso inducido por una expansión generalizada. Tenenemos nuevamente dos subcasos:

a) El paso inducido está a la derecha.

b) El paso inducido está a la izquierda.

Teorema 5. Sea G una gramática de pregrupo y f un morfismo en $\mathbf{RC}(G)$, entonces f puede expresarse como

$$f = (e_1 \circ \cdots \circ e_r) \circ (p_1 \circ \cdots \circ p_s) \circ (c_1 \circ \cdots \circ c_t)$$

donde los e_i son expansiones generalizadas, los p_j son pasos inducidos y c_k son contracciones generalizadas.

Demostración. Sea G una gramática de pregrupo y $f: u \rightarrow v$ un morfismo en $\mathbf{RC}(G) = \mathbf{RC}(\Delta_G + I_G)$, entonces de acuerdo a la proposición 10, f se expresa como un diagrama en posición general, $f = d_n \circ d_{n-1} \circ \cdots \circ d_2 \circ d_1$, hagamos la prueba por inducción sobre n .

Si $n = 1$ terminamos. Supongamos que si tenemos un diagrama con n niveles, entonces satisface el teorema y supongamos $f = d_{n+1} \circ \cdots \circ d_1$.

Aplicamos la hipótesis de inducción al morfismo $d_n \circ \cdots \circ d_1$, entonces

$$d_n \circ \cdots \circ d_1 = c_1 \otimes \cdots \otimes c_r \otimes p_1 \otimes \cdots \otimes p_s \otimes e_1 \otimes \cdots \otimes e_t$$

donde los e_i son expansiones generalizadas, los p_j son pasos inducidos y c_k son contracciones generalizadas.

Notemos que si d_n es una expansión generalizada, entonces ya terminamos. Supongamos que no y tenemos los siguientes casos.

- Caso 1. El dominio de d_n no es codominio de expansiones, contracciones generalizadas o pasos inducidos. Entonces, podemos aplicar consecutivamente el movimiento 1. para llevarlo a la forma deseada.
- Caso 2. El dominio de d_n es codominio de una expansión generalizada c_i . Entonces podemos aplicar el movimiento 1. para poner consecutivamente $d_n \circ c_i$.
 - Caso 2.1. d_n es paso inducido. Entonces podemos aplicar el movimiento 5.a) o 5.b) para poner una expansión generalizada.
 - Caso 2.2. d_n es una contracción generalizada. Entonces podemos aplicar 3.a) o 3.b) para sustituir $d_n \circ c_i$ por un paso inducido y, de ser necesario, aplicar el caso 1 o el caso 2.1
- Caso 3. El dominio de d_n es codominio de un paso inducido p_j . Entonces podemos aplicar consecutivamente el movimiento 1 hasta obtener $d_n \circ p_j$
 - Caso 3.1. d_n es paso inducido, entonces ya terminamos.
 - Caso 3.2 d_n es contracción generalizada, entonces podemos aplicar el movimiento 4.a) o 4.b) para obtener una contracción generalizada, y aplicar este paso consecutivamente hasta poder aplicar el movimiento 1 y mover la contracción según la configuración deseada.

Notemos finalmente que el dominio d_n no puede ser el codominio de una contracción generalizada, pues sería la identidad en 1 o una expansión generalizada.

Por lo tanto, en cualquier caso f es expresable de la forma deseada como queríamos demostrar. $\square$

Un corolario inmediato de este lema es el siguiente

Corolario 1. *Sea $G = (V, B, \Delta, I, s)$ una gramática de pregrupo y $u \in \mathcal{L}(G)$, entonces existe una derivación de $f: u \rightarrow s$ usando exclusivamente pasos inducidos y contracciones generalizadas.*

Demostración. Sea $G = (V, B, \Delta, I, s)$ una gramática de pregrupo y $u \in \mathcal{L}(G)$, entonces existe un morfismo $f: u \rightarrow s$, por el teorema anterior,

$$f = (e_1 \circ \cdots \circ e_r) \circ (p_1 \circ \cdots \circ p_s) \circ (c_1 \circ \cdots \circ c_t): u \rightarrow s$$

donde los e_i son expansiones generalizadas, los p_j son pasos inducidos y c_k son contracciones generalizadas.

Supongamos que $r > 0$, entonces ya que $\text{cod}(e_1) = v \otimes a \otimes b^r \otimes w$ con $a, b \in V \cup B$ y $v, w \in (V \cup B)^*$, ya que $e_2, \dots, e_r$ son contracciones, entonces $\text{cod}(f) = v' \otimes a \otimes b^r \otimes w'$, entonces $\text{cod}(f) \neq s$, pues $s \in V \cup B$.

Por lo tanto, $r = 0$ y $f = (p_1 \circ \cdots \circ p_s) \circ (c_1 \circ \cdots \circ c_t)$. $\square$

El colorario anterior nos brinda la ventaja principal de los pregrupos sobre las gramáticas anteriores; el análisis de la sintaxis mediante estas gramáticas es más eficiente, pues existe un pseudoalgoritmo para realizar las reducciones. Más aún, es ejecutable en tiempo polinomial [3].

6 DisCoPy

El objetivo de este capítulo es transformar las definiciones matemáticas definidas en los capítulos previos en información que puede ser interpretada mediante código en `Python`.

Para este fin, exploraremos la biblioteca `DisCoPy` (Distributional Compositional Python), introducida en [2] por Bob Coecke, Giovanni di Felice y Alexis Toumi, una herramienta de código abierto para el cómputo de categorías y diagramas de cables. La instalación de la paquetería `DisCoPy` se realiza con la siguiente línea de código

```
1 pip install discopy
```

El código presentado en este capítulo puede consultarse en el siguiente repositorio: <https://github.com/FerFerreyra/TesisFernando.git>.

Los códigos de este capítulo fueron obtenidos de la documentación de `DisCoPy`, [1] y [9] con modificaciones correspondientes a la versión actual de `DisCoPy` y al idioma español para su correcta ejecución.

6.1. Categorías

Como mencionamos anteriormente, los dos ingredientes básicos para definir una categoría $\mathcal{C}$ son una colección de objetos $\mathcal{C}_0$ y un conjunto de morfismos $\mathcal{C}_1$ que satisfacen las propiedades mencionadas con anterioridad.

En el contexto de `Python`, consideramos dos clases: la clase `Ob`, que contiene los objetos de la categoría, y la clase `Flecha`, que contiene los morfismos de la categoría, con los atributos `dom` y `cod` junto a dos métodos: `then` que efectúa la composición de dos flechas e `id` que genera la flecha identidad para cada objeto.

Antes de continuar, para señalar la naturalidad de introducir el concepto de categorías en el lenguaje de programación `Python`, definimos la categoría **Pyth** cuyos objetos son la clase de todos los tipos de `Python` y cuyas flechas son las clase de todas las funciones de `Python`.¹

El esqueleto de la implementación de una categoría en `Python` está dada de la siguiente forma.

¹Notemos que determinar qué es que dos funciones de `Python` sean iguales no es un concepto claro, incluso asumiendo que dos funciones son iguales si y sólo si producen los mismos resultados dado cualquier `input`, entonces verificar la asociatividad y propiedad de la identidad podrían fallar en programas no terminales, o ante la presencia del tipo `Undefined` o `None`. Sin embargo, no todo está perdido y podemos considerar los morfismos de **Pyth** excluyendo casos problemáticos.

```

1 class Ob: ...
2
3 class Arrow:
4     dom: Ob
5     cod: Ob
6
7     @staticmethod
8     def id(x:Ob) -> Arrow : ...
9     def then(self, other: Arrow) -> Arrow : ...

```

A continuación, proveemos de la implementación de una categoría libre en DisCoPy, recordemos que tiene como ingredientes básicos los objetos y generadores, o cajas, que forman los morfismos en la categoría. Comencemos con la clase `Ob` los cuales basta inicializar nombrándolos.

```

1 class Ob:
2     def __init__(self, name):
3         self.name = name

```

Continuamos con la clase `Arrow` que corresponden a los morfismos de la categoría.

```

1 class Arrow(Composable[Ob]):
2     def __init__(self, inside: tuple[Box, ...], dom: Ob, cod: Ob,
3         _scan: bool = True) -> None:
4
5     @classmethod
6     def id(cls: Type[Arrow], dom: Optional[Ob] = None) -> Arrow:
7
8     def then(self, *others: Arrow) -> Arrow:

```

Así cada morfismo es representado como una lista de cajas con un dominio y codominio. Notemos que si `_scan = True`, entonces verificamos que la composición de las cajas tenga sentido (es decir, que el codominio de una caja coincida con el dominio de la siguiente).

Y, finalizamos, con la clase `Box` de los generadores.

```

1 class Box(Arrow):
2     def __init__(self, name, dom, cod):
3         self.name, self.dom, self.cod = name, dom, cod
4         Arrow.__init__(self, dom, cod, [self], _scan=False)

```

Como ejemplo, podemos definir la categoría con los siguiente objetos y morfismos.

```

1 from discopy.cat import Ob, Box
2 w, x, y, z = map(Ob, "wxyz")
3 h, f, g = Box('h', w, x), Box('f', x, y), Box('g', y, z)

```

Es importante resaltar que la composición de funciones, `f.then(g)` o `h.then(f).then(g)` se representa como `f>>g` y `h >> f >> g`.

Con esta implementación, podemos representar la gramática del ejemplo 1 de la siguiente forma.

```

1 s0, x0, x1, x2, x3, s1 = map(Ob, "s0 x0 x1 x2 x3 s1".split())
2 A,B = Box('A', s0, x0), Box('B', s0, x0)
3 conoce = Box('conoce', x0, x1)
4 a = Box('a', x1, x2)

```

```

5 C,D = Box('C', x2, x3), Box('D', x2, x3)
6 quien = Box('quien', x3, x0)
7 _ = Box('.', x3, s1)
8 gramatica = [A, B, conoce, a, C, D, quien, _]

```

Como es esperable, queremos determinar cuando una oración es válida en la gramática, así que definimos la siguiente función.

```

1 def valida_gramatical(oracion, gramatica):
2     flecha = Arrow.id(s0)
3     bandera = True
4     oracion = oracion.split() + ['.']
5     for palabra in oracion:
6         es_Box = False
7         for Box in gramatica:
8             if Box.name == palabra:
9                 try:
10                    flecha = flecha.then(Box)
11                    es_Box = True
12                    break
13                except AxiomError:
14                    bandera = False
15                    break
16         if not es_Box:
17             bandera = False
18             break
19     return bandera

```

6.2. Categorías monoidales

A pesar de que en la exposición de este trabajo describimos las multicategorías de manera previa a las categorías monoidales, DisCoPy define los conceptos al revés. Los objetos generadores en la categoría monoidal libre se realizan mediante el constructor `Ty`.

```

1 from discopy import cat
2 class Ty(cat.Obj):
3     def __init__(self, *inside: str | cat.Obj):
4     def tensor(self, *others: Ty) -> Ty:

```

Es decir, los objetos serán objetos concatenados por la operación binaria tensor, denotada $\otimes$.

Los morfismos, por otra parte, son representados como los diagramas que generan, así están definidos en virtud de la proposición 8 de la siguiente manera

```

1 class Diagrama(cat.Arrow):
2     def __init__(self, dom, cod, boxes, offsets, layers):
3
4     def tensor(self, other: Diagram = None, *others: Diagram) ->
      Diagram:
5
6     @classmethod

```

```

7     def id(cls: Type[Diagrama], dom: Optional[Ob] = None) -> Diagram
8     :
9     def then(self, *others: Diagram) -> Diagram:

```

donde notamos que los métodos `tensor` y `then` corresponden a la composición paralela y secuencial respectivamente.

El último ingrediente para definir una categoría monoidal libre son los generadores, así está la siguiente implementación de la clase `Box`.

```

1     class Box(cat.Box, Diagram):
2         def __init__(self, name, dom, cod, **params):

```

Implementemos la siguiente categoría monoidal libre, cuyos objetos son

```

1 from discopy.monoidal import Ty, Box, Diagram
2
3 x = Ty("x")
4 y, z, w = Ty(*"yzw")

```

y con los siguientes morfismos generadores

```

1 f, g, h = Box('f', x, y), Box('g', z, w), Box('h', y, z)

```

Recordemos que los objetos en la categoría monoidal son “cadenas” de objetos generadores, así si a, b son objetos, denotamos como $a @ b$ su producto tensorial. De tal manera que su composición secuencial y paralela se representa de la siguiente manera.

```

1 f.then(h)
2 f @ g

```

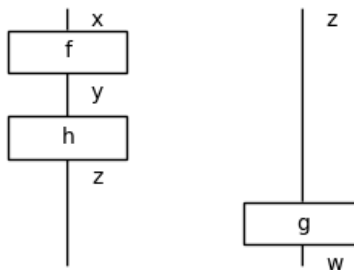
La clase de morfismos cuenta con el método `draw` que presenta los diagramas de cables de los morfismos en posición general. Por ejemplo, la siguiente línea de código

```

1 (f.then(h) @ g).draw(figsize=(5, 5))

```

produce el siguiente diagrama



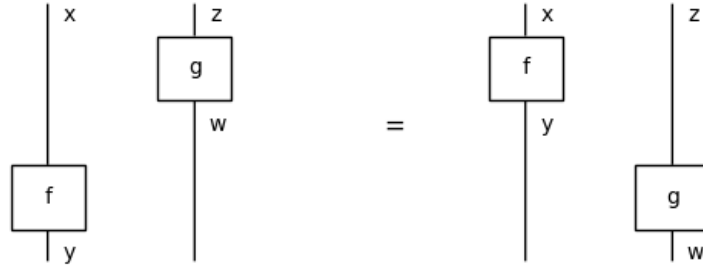
Incluso es posible representar ecuaciones del cálculo gráfico, por ejemplo, la ley de intercambio se obtiene de la siguiente forma.

```

1 Equation(
2     (f @ g).interchange(0, 1), (f @ g),
3     symbol="$=$").draw(figsize=(5, 2))

```

y produce el siguiente diagrama



Como mencionamos en el capítulo 4, las gramáticas monoidales resultan muchas veces poco útiles computacionalmente; sin embargo, esta clase de `DisCoPy` es sumamente eficiente para implementar el resto de gramáticas.

6.3. Multicategorías

Dentro de `DisCoPy`, las multicategorías se encuentran definidas dentro del módulo `grammar.cfg`. Los objetos son implementados con el mismo constructor que las categorías monoidales, pues recordamos que queremos que el dominio de las flechas sea una cadena de objetos.

La diferencia principal son la forma de los morfismos, tal como los definimos en el capítulo dos, son árboles cuya implementación es la siguiente en la clase `Tree`:

```
1 class Tree:
2     def __init__(self, root: Rule, *branches: Tree):
3
4     @staticmethod
5     def id(dom):
6         return Id(dom)
7
8     def __call__(self, *others) -> Tree:
```

donde el constructor de `Tree` recibe una raíz y sus hojas, junto a árboles tales que las composiciones en las raíces están bien definidas. La composición de árboles se implementa mediante el método `call`.

Por otro lado, los generadores de una multicategoría libre se implementan mediante la clase `Rule`

```
1 class Rule(Tree, thue.Rule):
2     def __init__(self, dom: monoidal.Ty, cod: monoidal.Ty, name:
3         str = None):
```

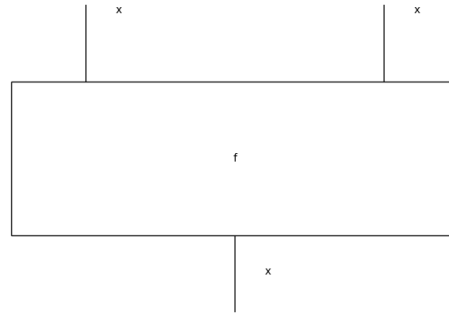
donde el constructor verifica que la longitud de `cod` es uno, así un elemento de `Rule` es una raíz con sus ramas. Consideremos la implementación de la siguiente multicategoría libre

```
1 from discopy.grammar.cfg import Ty, Tree, Rule
2
3 x, y = Ty('x'), Ty('y')
4 f, g, h = Rule(x @ x, x, name='f'), Rule(x @ y, x, name='g'), Rule(y
5     @ x, x, name='h')
```

Tal como la clase `Diagrama`, la clase `Tree` tiene asociado un método para generar los diagramas asociados a los morfismos. La siguiente línea

```
1 f.to_diagram().foliation().draw()
```

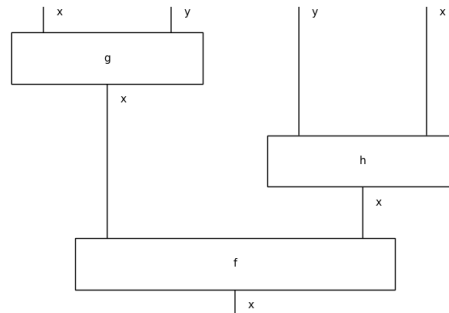
produce el siguiente diagrama



Recordemos que la composición en `Tree` ya no se da por medio del método `then`, sino con `__call__`. Un ejemplo de composición es el siguiente:

```
1 f(g, h).to_diagram().foliation().draw()
```

que produce el siguiente diagrama



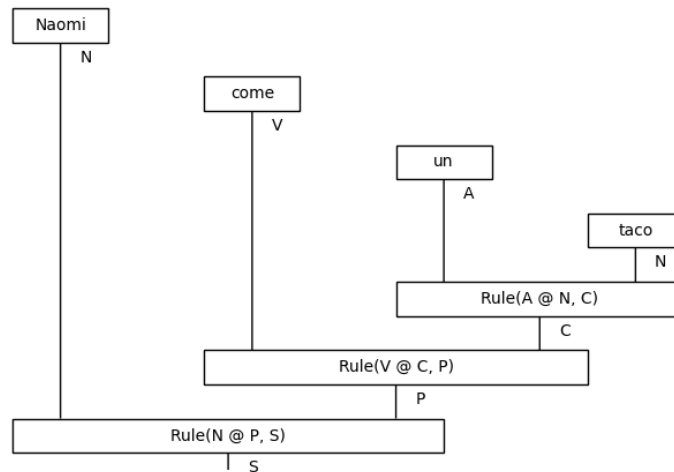
Antes de poder realizar derivaciones en `Tree`, necesitamos definir la clase `Word`

```
1 class Word(thue.Word, Rule):
2     def __init__(self, name: str, cod: monoidal.Ty, dom: monoidal.
    Ty = Ty(), **params):
```

que corresponde a una raíz con una sola hoja, que servirá para poder generar el léxico. Con lo anterior, podemos realizar el código del ejemplo 7 de la siguiente forma:

```
1 n, v, a = Ty('N'), Ty('V'), Ty('A')
2 c, p, s = Ty('C'), Ty('P'), Ty('S')
3 Naomi, come = Word('Naomi', n), Word('come', v)
4 un, taco = Word('un', a), Word('taco', n)
5 C, P = Rule(a @ n, c), Rule(v @ c, p)
6 S = Rule(n @ p, s)
7 sentence = S(Naomi, P(come, C(un, taco)))
8 sentence.to_diagram().foliation().draw()
```

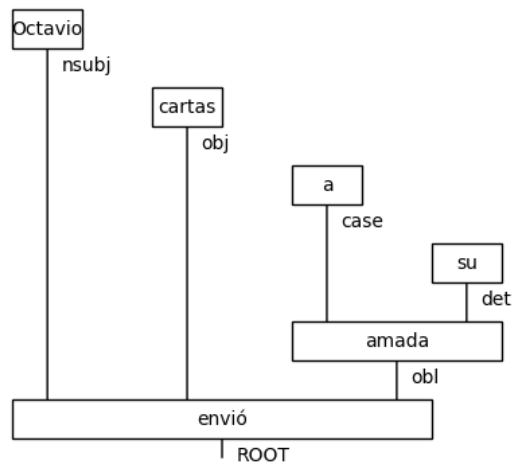
y produce el siguiente árbol



Por último, es importante mencionar que en el módulo `discopy.grammar` de `DisCoPy` se tiene la función `from_spacy` que permite generar árboles de derivación de oraciones con base en el análisis sintáctico realizado por la biblioteca para el procesamiento de lenguaje natural, `spaCy`. Afortunadamente, la librería es capaz de analizar oraciones en español, así que veamos el árbol obtenido al analizar la oración 10.

```
1 from discopy.grammar.dependency import from_spacy
2 import spacy
3
4 nlp = spacy.load("es_core_news_sm")
5 doc = nlp("Octavio envió cartas a su amada")
6
7 from_spacy(doc).to_diagram().draw(figsize=(4, 4))
```

que produce el siguiente árbol de derivación



6.4. Categorías bicerradas

Las categorías bicerradas se implementa en el módulo `discopy.closed`.

Recordemos que los objetos de una categoría bicerrada son de la forma a , a/b , $a \backslash b$ y $a \otimes b$. Así, los objetos se implementan de la siguiente forma

```
1 class Ty(monoidal.Ty): ...
2
3 class Over(Ty):
4     def __init__(self, left=None, right=None): ...
5
6 class Under(Ty):
7     def __init__(self, left=None, right=None): ...
```

Donde un objeto `Over(x,y)` representado como $x \ll y$, es el objeto x/y ; mientras que `Under(x,y)` representado como $x \gg y$ es el objeto $x \backslash y$. Recordemos que la categoría bicerrada se define como la categoría monoidal obtenida por los generadores añadiendo el `curry` y `uncurry` de cada función. Entonces implementamos los morfismos de la siguiente manera:

```
1 class Diagram(monoidal.Diagram)
2     def curry(self, n_wires=1, left=False) -> Diagram : ...
3
4     def uncurry(self) -> Diagram : ...
5
6 class Id(monoidal.Id, Diagram) -> Diagram : ...
7
8 class Box(monoidal.Box, Diagram) -> Diagram : ...
```

donde los argumentos de `curry` son un morfismo, el número de elementos a “currificar” y si es izquierdo o derecho. Tanto `curry` como `uncurry` se definen como funciones sobre los generadores, es decir, sobre `Box`.

Podemos entonces considerar los siguientes objetos y generadores de una categoría bicerrada

```
1 from discopy.closed import Ty, Box, Curry
2
3 x, y, z = map(Ty, "xyz")
4 f, g, h = Box('f', x, z << y), Box('g', x @ y, z), Box('h', y, x >>
    z)
```

Entonces, podemos aplicar `curry` y `uncurry` de la siguiente manera

```
1 f.uncurry()
2 g.curry()
3 h.uncurry(left=False)
```

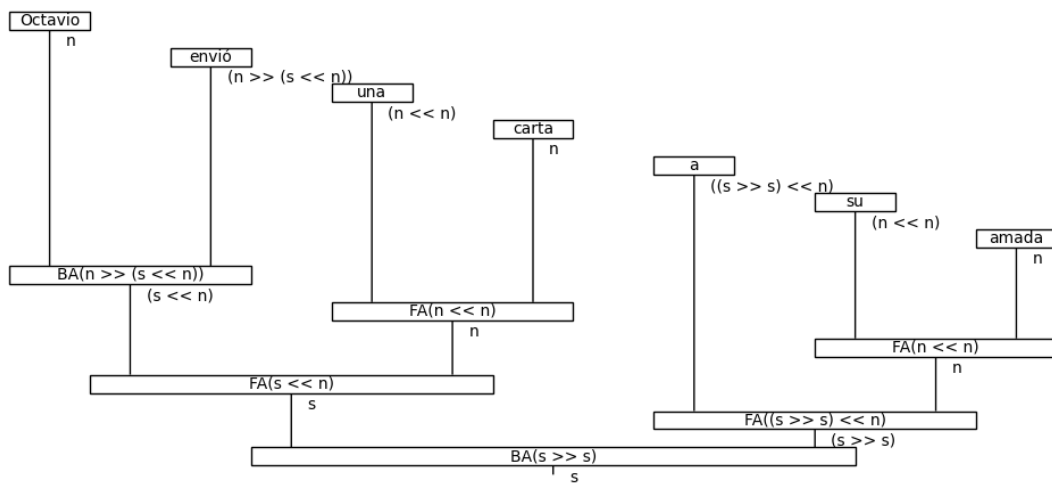
Para definir las gramáticas categoriales, *DisCoPy* tiene implementado el módulo `discopy.grammar.categorical` que cuenta con las funciones correspondientes a las reglas de inferencia. A continuación presentamos el código y derivación obtenida de los ejemplos 10 y 12.

```
1 from discopy.closed import Ty
2 from discopy.grammar.categorical import Word, BA, FA
```

```

3
4 s, n = map(Ty, 'sn')
5
6 Octavio = Word('Octavio', n)
7 envio = Word('envio', n >> (s << n))
8 una = Word('una', n << n)
9 carta = Word('carta', n)
10 a = Word('a', (s >> s) << n)
11 su = Word('su', n << n)
12 amada = Word('amada', n)
13
14 sentence = Octavio @ envio @ una @ carta @ a @ su @ amada\
15     >> BA(n >> (s << n)) @ FA(n << n) @ ((s >> s) << n) @ FA(n << n)
16     \
17     >> FA(s << n) @ FA((s >> s) << n)\
18     >> BA(s >> s)
19 sentence.draw()

```



```

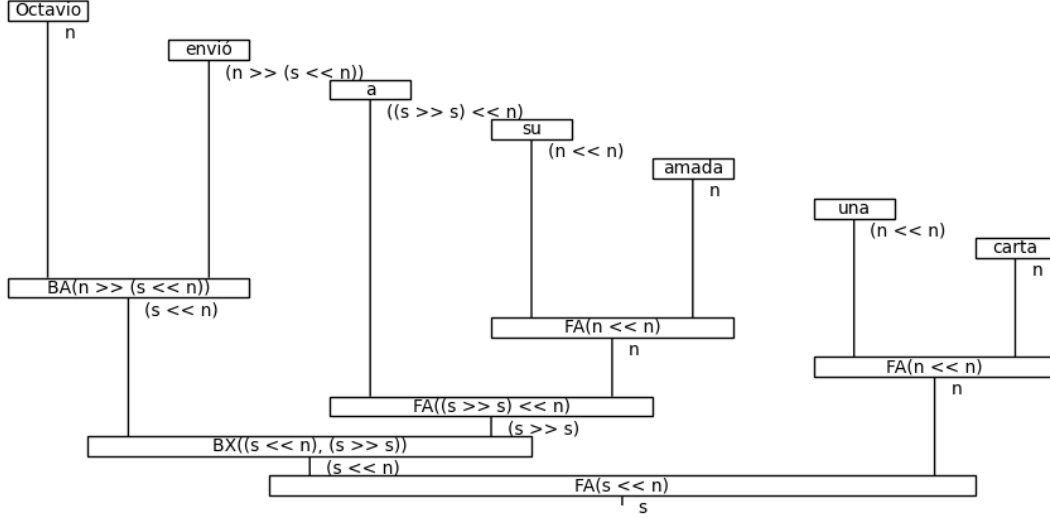
1 from discopy.closed import Ty
2 from discopy.grammar.categorical import Word, BA, FA, FX, BX
3
4 s, n = map(Ty, 'sn')
5
6 Octavio = Word('Octavio', n)
7 envio = Word('envio', n >> (s << n))
8 una = Word('una', n << n)
9 carta = Word('carta', n)
10 a = Word('a', (s >> s) << n)
11 su = Word('su', n << n)
12 amada = Word('amada', n)
13
14 sentence = Octavio @ envio @ a @ su @ amada @ una @ carta\
15     >> BA(n >> (s << n)) @ ((s >> s) << n) @ FA(n << n) @ FA
16     (n << n)\

```

```

16         >> (s << n) @ FA((s >> s) << n) @ n\
17         >> BX(s << n, s >> s) @ n\
18         >> FA(s << n)
19
20 sentence.draw()

```



6.5. Categorías rígidas

Para la implementación de categorías rígidas, **DisCoPy** utiliza la siguiente convención. Dado un conjunto de objetos G_0 , consideramos los objetos en la categoría rígida libre de la forma $(x, z) \in G_0 \times \mathbb{Z}$ donde definimos que $(x, z)^r = (x, z+1)$ y $(x, z)^l = (x, z-1)$. Notemos que esta definición es completamente compatible con la exhibida en el capítulo 5 pues tenemos la siguiente asignación: dado un elemento $x \in G_0$.

$$\overbrace{x^r \cdots r}^{n-\text{veces}} \mapsto (x, n) \quad x \mapsto (x, 0) \quad \overbrace{x^l \cdots l}^{n-\text{veces}} \mapsto (x, -n)$$

Así, primero se define la clase de tipos básicos

```

1 class Ob(cat.Ob):
2     def __init__(self, name, z=0): ...
3
4     def l(self) -> Ob: ...
5
6     def r(self) -> Ob: ...

```

con ello definimos lo elemento dentro de la categoría rígida libre.

```

1 class Ty(monoidal.Ty, Ob):
2     def __init__(self, *t):
3

```

```

4     def r(self) Ty(monoidal.Ty, Ob): ...
5
6     def l(self) Ty(monoidal.Ty, Ob): ...

```

Y los morfismos son de la forma de una categoría monoidal libre, diagramas, generados por cups y caps.

```

1 class Diagram(monoidal.Diagram):
2     def cups(left, right) -> Diagram: ...
3
4     def caps(left, right) -> Diagram: ...
5
6 class Box(monoidal.Box, Diagram):
7
8 class Id(monoidal.Id, Diagram):

```

Al igual que con las gramáticas libres de contexto y las gramáticas categoriales, Discopy cuenta con la instanciación de la librería `discopy.rigid` para pregrupos. En ella, podemos definir las evaluaciones y coevaluaciones de la siguiente manera

```

1 from discopy.grammar.pregroup import Ty, Cup, Cap
2
3 n = Ty('n')
4 Cap(n, n.l)
5 Cap(n.r, n)
6 Cup(n.l, n)
7 Cup(n, n.r)

```

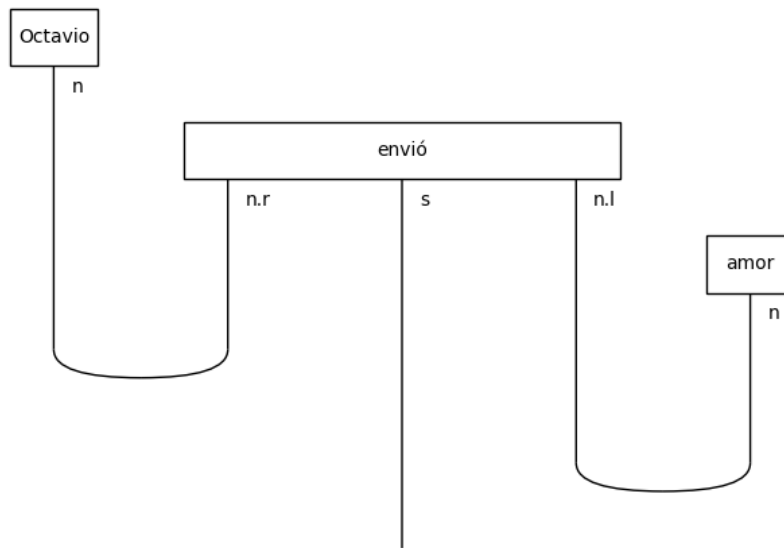
Ya que sabemos implementar los pregrupos, hagamos una derivación sencilla.

```

1 from discopy.grammar.pregroup import Ty, Word, Cup
2
3 s, n = Ty('s'), Ty('n')
4 Octavio, amor = Word('Octavio', n), Word('Eliza', n)
5 envio = Word('envio', n.r @ s @ n.l)
6
7 sentence = Octavio @ envio @ amor >> Cup(n, n.r) @ s @ Cup(n.l, n)
8 sentence.draw()

```

que genera el siguiente diagrama



Finalizaremos este capítulo y este texto con el recordatorio que toda categoría rígida es bicerrada y, por lo tanto, existe el `curry`.

A continuación se mostrará la implementación en gramáticas de pregrupos de un analizador basado en `Spacy`. Además, los probaremos con nuestra ya conocida oración “Octavio envió una carta a su amada”.

```

1 from discopy.grammar.pregroup import Ty, Id, Box, Diagram
2 import spacy
3
4 def curry(diagram, n_wires=1, left=False):
5     if not n_wires > 0:
6         return diagram
7     if left: #El curry se realiza por la izquierda
8         wires = diagram.dom[:n_wires]
9         #Sustituimos los primeros cables por sus caps para realizar el
10        curry
11        return Diagram.caps(wires.r, wires) @ Id(diagram.dom[n_wires:])
12        >> Id(wires.r) @ diagram
13        wires = diagram.dom[-n_wires:]
14        #Sustituimos los ultimos cables por sus caps para realizar el
15        curry
16        return Id(diagram.dom[:-n_wires]) @ Diagram.caps(wires, wires.l)
17        >> diagram @ Id(wires.l)
18
19
20 def find_root(doc):
21     for token in doc:
22         if token.dep_ == 'ROOT':
23             return token
24
25
26 def doc2rigid(word):
27     children = word.children
28     #Si la raiz no tiene hijos, terminamos
29     if not children:

```

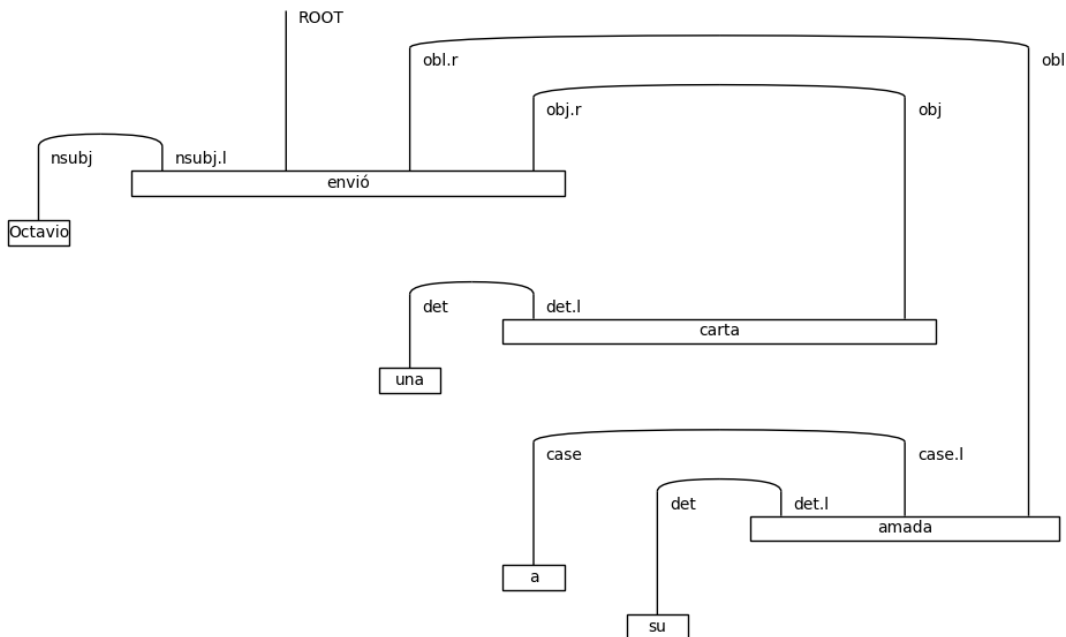


```

24     return Box(word.text, Ty(word.dep_), Ty())
25     #Generamos los objetos por los hijos a la izq y a la derecha
26     #y obtenemos la clasificacion sintactica de la palabra en la
    oracion.
27     left = Ty(*[child.dep_ for child in word.lefts])
28     right = Ty(*[child.dep_ for child in word.rights])
29     #Definimos el generador obtenido con la palabra actual
30     box = Box(word.text, left.l @ Ty(word.dep_) @ right.r, Ty(),
31     data=[left, Ty(word.dep_), right])
32     top = curry(curry(box, n_wires=len(left), left=True), n_wires=len(
    right))
33     #Realizamos la llamada recursiva para seguir explorando la oracion
    .
34     bot = Id(Ty()).tensor(*[doc2rigid(child) for child in children])
35     return top >> bot
36
37 def doc2pregroup(doc):
38     root = find_root(doc)
39     return doc2rigid(root)
40
41 nlp = spacy.load("es_core_news_sm")
42 doc = nlp("Octavio envio una carta a su amada")
43
44 doc2pregroup(doc).draw()

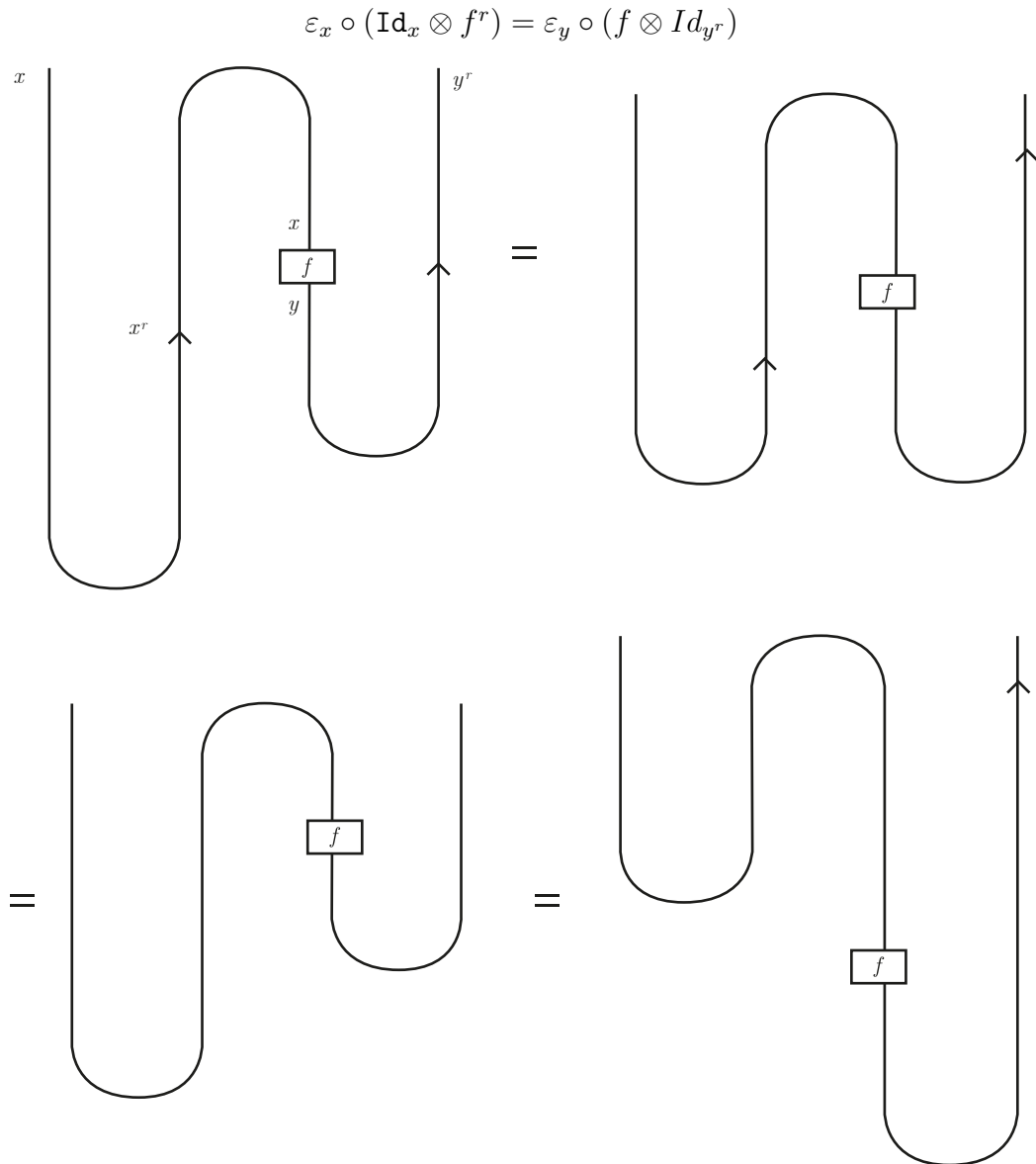
```

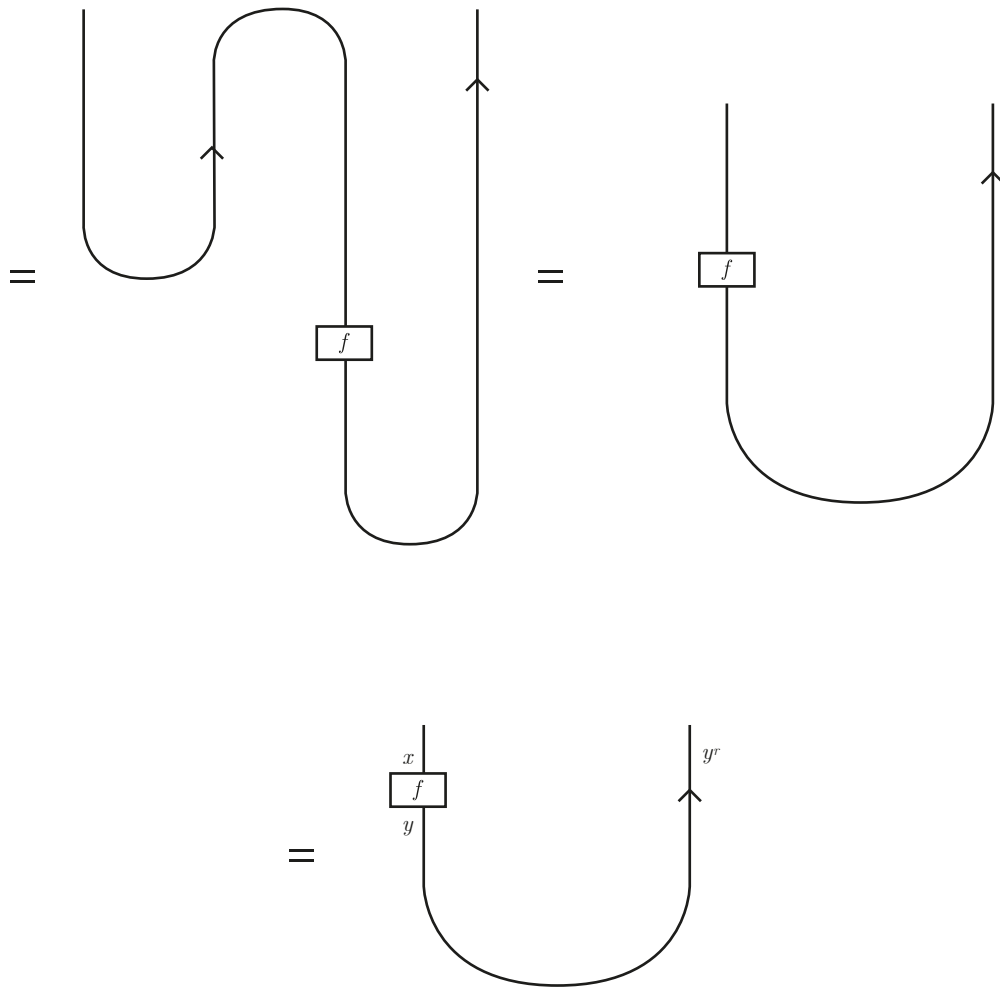
que produce el siguiente diagrama



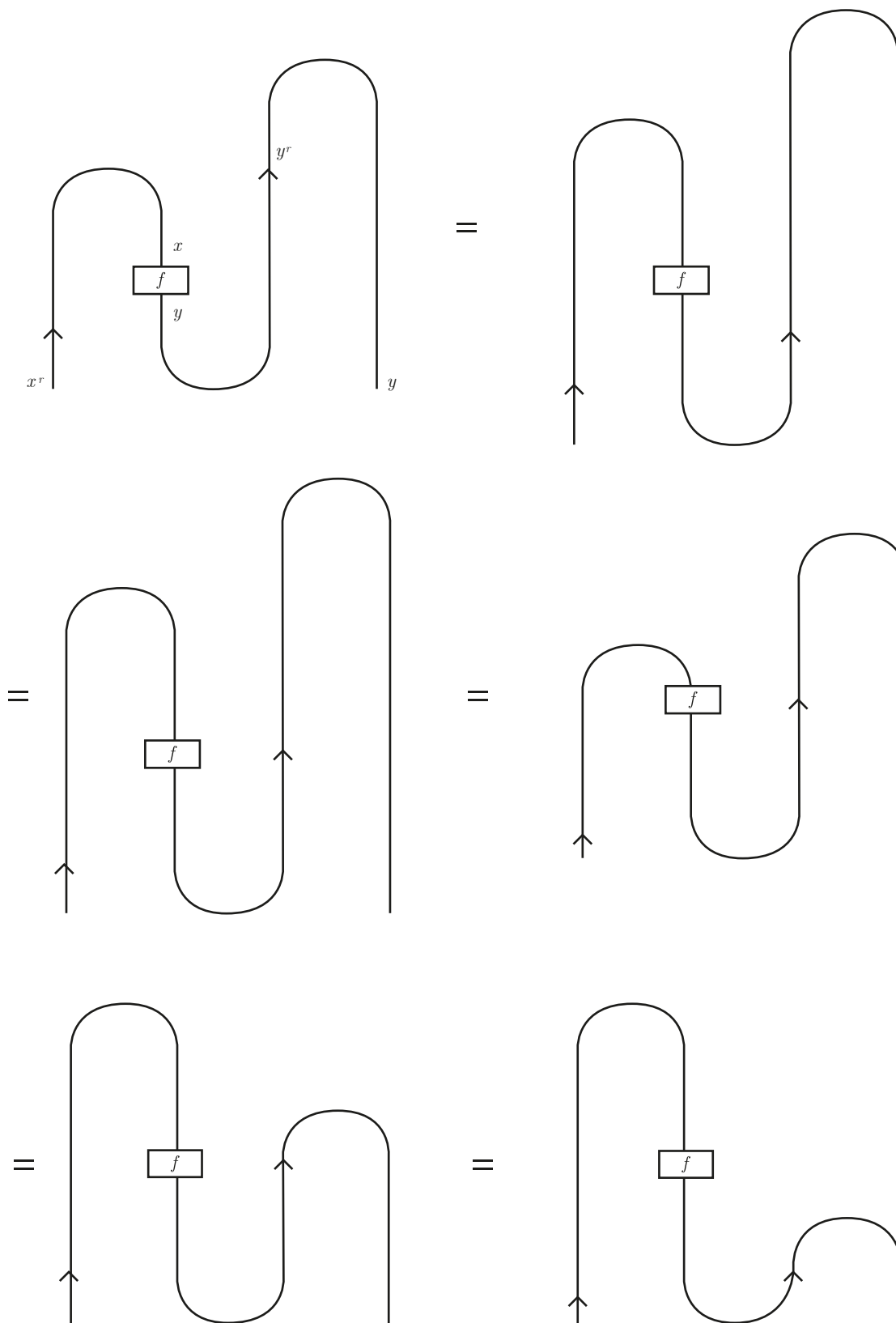
Apéndice. Demostraciones con diagramas

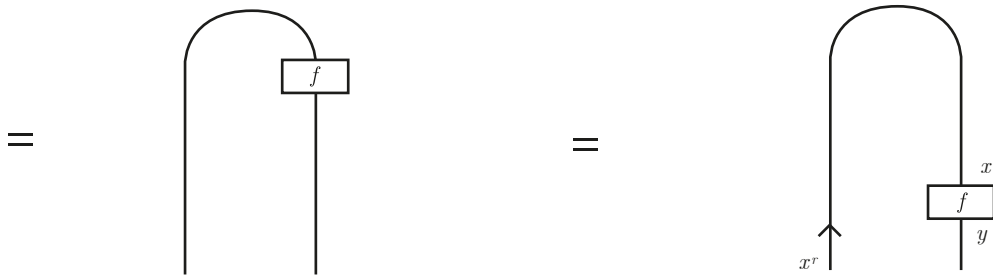
Presentamos los diagramas asociados a distintas igualdades formuladas en el Capítulo 5, con el propósito de mostrar la claridad y eficiencia que ofrecen.



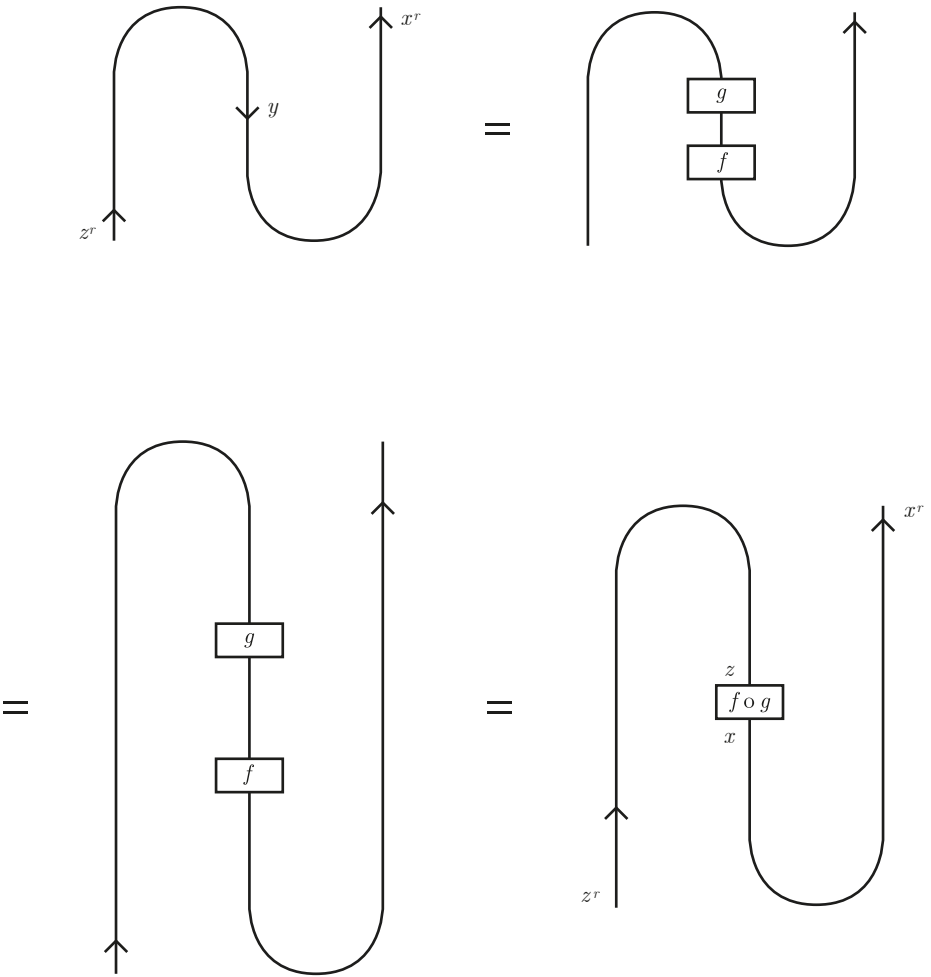


$$(f^r \otimes \text{Id}_y) \circ \eta_y = (\text{Id}_{x^r} \otimes f) \circ \eta_x$$

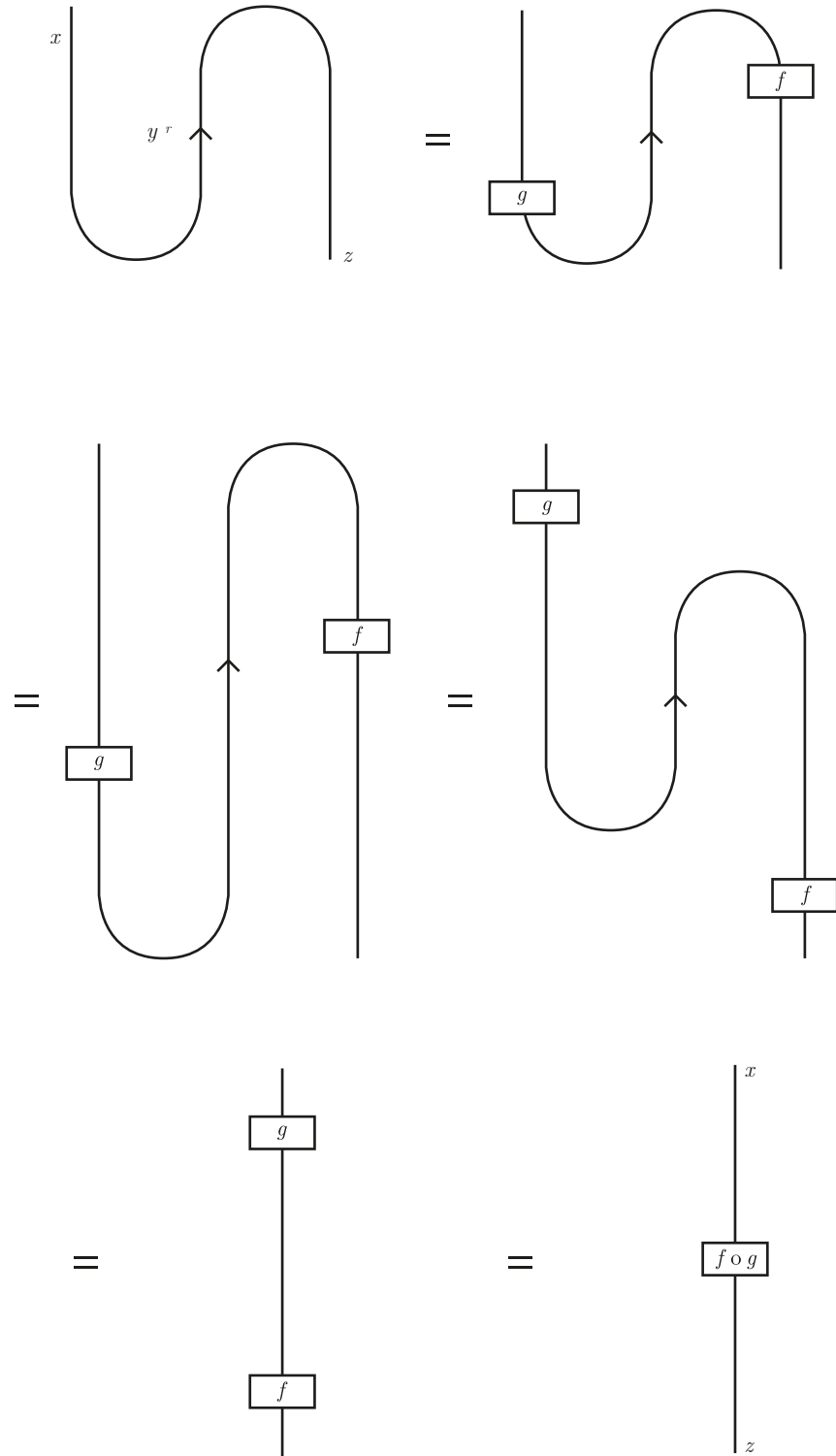




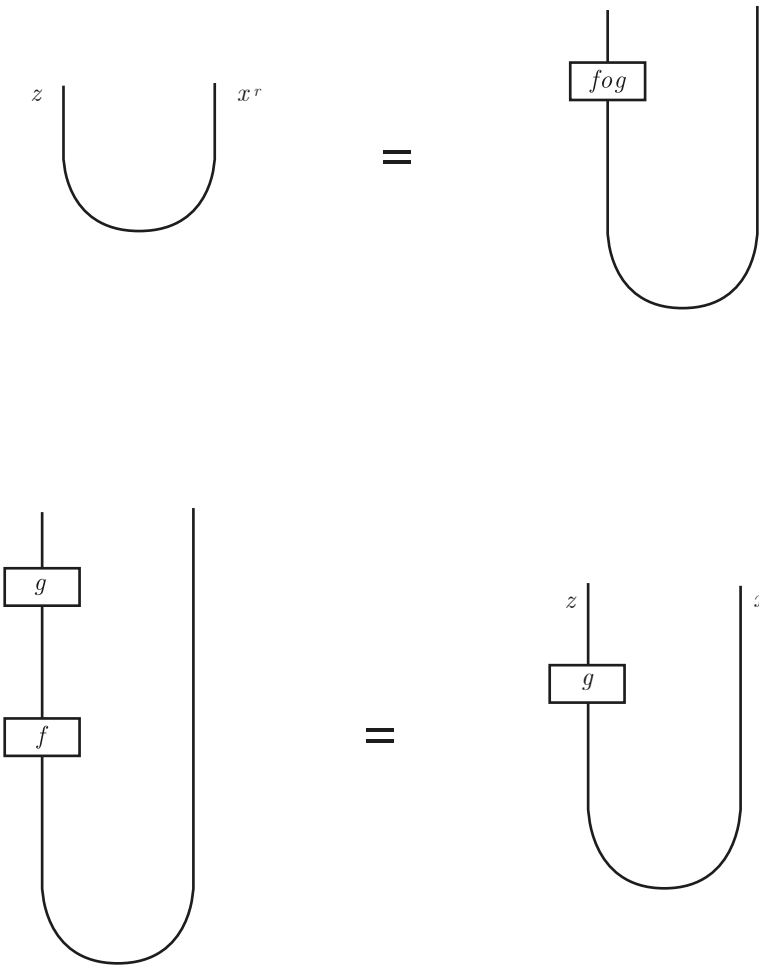
$$(\text{Id}_{z^r} \otimes \varepsilon_f) \circ (\eta_g \otimes \text{Id}_{x^r}) = (f \circ g)^r$$



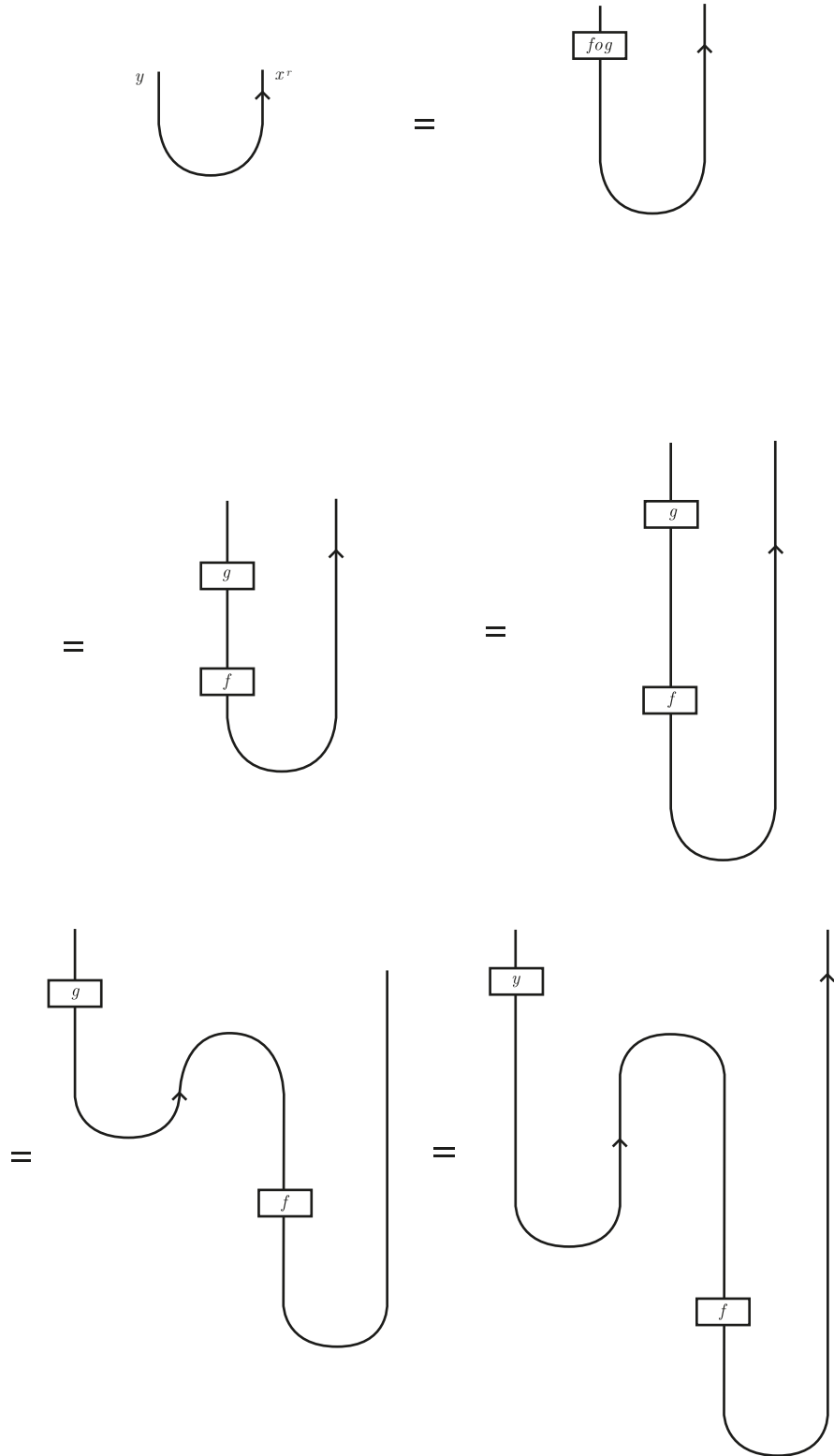
$$(\varepsilon_g \circ \text{Id}_z) \circ (\text{Id}_x \circ \eta_f) = f \circ g$$

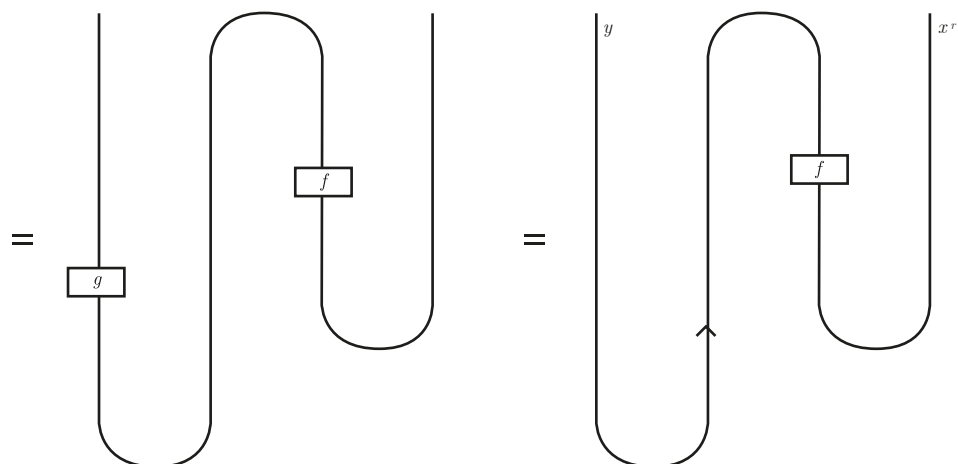
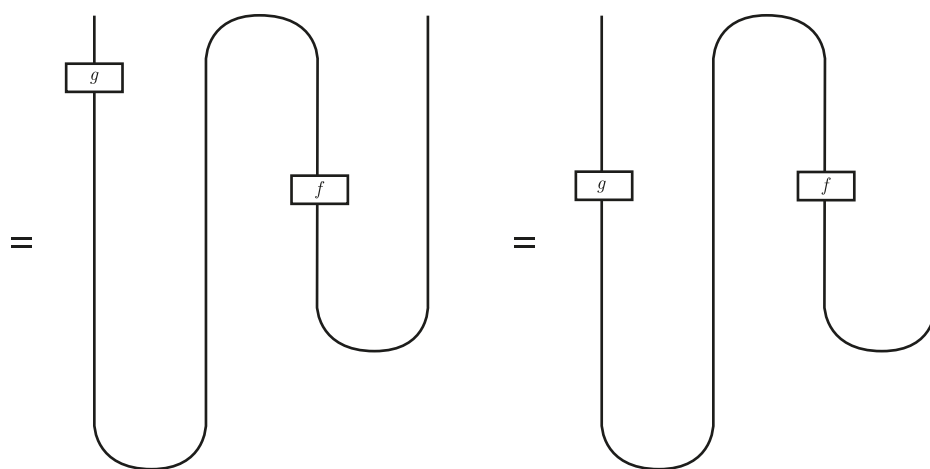
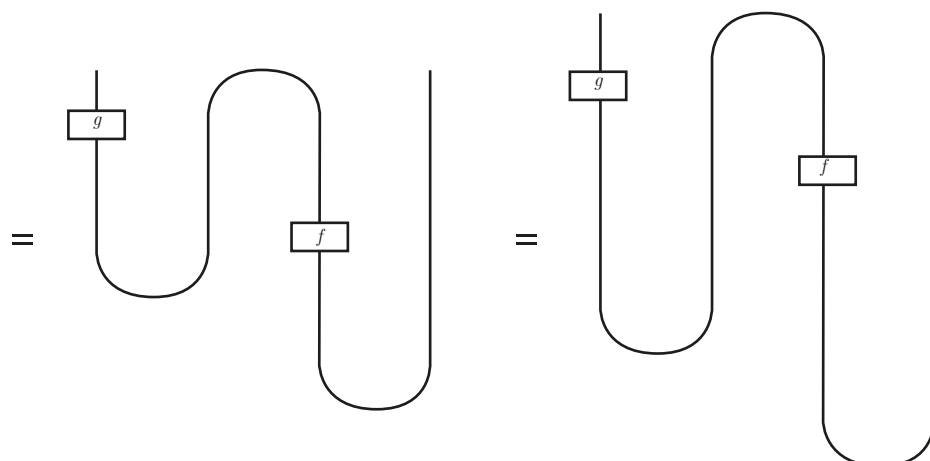


$$\varepsilon_{f \circ g} = \varepsilon_f \circ (g \otimes \text{Id}_{x^r})$$



$$\varepsilon_{f \circ g} = \varepsilon_g \circ (\text{Id}_y \otimes f^r)$$





Bibliografía

- [1] Giovanni de Felice. Categorical tools for natural language processing, 2022. Tesis de doctorado, Universidad de Oxford.
- [2] Giovanni de Felice, Alexis Toumi, and Bob Coecke. Discopy: Monoidal categories in Python. *Electronic Proceedings in Theoretical Computer Science*, 333:183–197, February 2021.
- [3] Sylvain Degeilh and Anne Preller. Efficiency of pregroups and the French noun phrase. *J. Logic Lang. Inf.*, 14(4):423–444, October 2005.
- [4] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to automata theory, languages, and computation, 2nd edition*, volume 32. ACM Press, New York, NY, USA, 2001.
- [5] Agnieszka Kułacka. Natural language versus regular language. *Kształcenie Językowe*, 2012.
- [6] J. Lambek. Type grammar revisited. In Alain Lecomte, François Lamarche, and Guy Perrier, editors, *Logical Aspects of Computational Linguistics*, pages 1–27, Berlin, Heidelberg, 1999. Springer Berlin Heidelberg.
- [7] Roger Levy, Evelina Fedorenko, Mara Breen, and Ted Gibson. The processing of extraposed structures in English. *Cognition*, 122(1):12–36, 2012.
- [8] Saunders Mac Lane. *Categories for the Working Mathematician*. Springer New York, 1978.
- [9] Alexis Toumi. Category theory for quantum natural language processing, 2022. Tesis de doctorado, Universidad de Oxford.
- [10] A. Joyal y R. Street. Planar diagrams and tensor algebra. 1988. Manuscrito no publicado. Consultado en <https://web.science.mq.edu.au/~street/PlanarDiags.pdf>.